# 系统分析与设计复习笔记

dh

# 00-课程介绍

需求获取-需求分析-可行性分析-系统分析-设计与编码实现-软件测试-软件配置管理-软件维护-软件项目管理

SDLC（软件开发生命周期）：分析阶段、设计阶段、实现阶段、维护阶段

# 01-概述

## 基本概念

系统：一组为实现共同目标而相互关联、相互作用的部件

- 系统是由若干个部件组成 （系统可以划分为子系统，子系统是系统的基本组成元素）
- 系统的部件之间是相互关联、相互作用的
- 系统有一个实现目标 （目标是系统功能的综合表现）

信息系统：一种专门类型的系统，用于对信息进行采集、输入、处理、输出、存储、传播以及对信息进行管理

- 信息系统功能（6点）：输入、处理、输出、存储、传播、管理
- 信息系统是一种人造系统
- 信息系统是一种自动系统

信息系统的组成部分：软件、硬件、数据、过程和人

## 信息系统的分类

## 系统开发角色分工及任务

系统分析员任务

了解客户的需求 - 建立分析模型 - 了解系统环境与平台 - 建立设计模型 - 编写分析与设计文档（系统分析说明书/系统设计说明书）

## 系统开发方法介绍

结构化开发方法 - 面向对象开发方法 - 基于UML的面向对象开发方法

系统分析

- 需求获取和描述
- 需求分析与建模
- 方案评估及选择

系统设计:

- 系统架构设计
- 系统概要设计

- 系统详细设计
- 数据库设计
- 用户界面设计

## UML统一建模语言回顾

UML — Unified Modeling Language

统一标准:已成为面向对象的标准化的统一的建模语言

视图是比表达系统某一方面特征的UML建模元素的子集，它是由一个或者多个图组成的对系统某个角度的抽象

Use-Case View(用例视图)

用途：描述系统应该具备的功能，被称为参与者（执行者）的外部用户所能观察到的功能

用例视图是几个视图的核心

用例视图包括用例图

Logical View(逻辑视图)

用途：描述用例视图中提出的系统功能的实现

逻辑视图既描述系统的静态结构，也描述系统内部的动态协作关系

包含的UML图：类图、对象图、状态图、时序图、协作图、活动图

静态结构在类图和对象图中描述

动态模型在状态图、时序图、协作图、活动图中描述

Process View(进程视图)

用途:考虑资源的有效利用、代码的并行执行以及系统环境中异步事件的处理

解决在并发系统中存在的通信和同步问题

包含的UML图：状态图、协作图、组件图、活动图

Implementation View(实现视图)

用途:描述系统的实现模块以及它们之间的依赖关系

包含UML图：组件图

Deployment View(部署视图)

用途:显示系统的物理部署，并描述位于节点实例上的运行组件实例的部署情况

包含UML图：组件图

UML中的关系

关联 依赖 泛化 实现 聚合/组合

类的状态图在状态之间会在状态之间标注转移条件

# 02-需求分析与需求获取

## 软件需求的定义

为用户解决问题以达到特定目标所需的能力

系统或系统构件要满足的合同、标准、规范或其他正式文档所需具备的能力

软件需求要求以一种**清晰、简洁、一致且无二义性**的方式描述期望，是开发过程中对未来系统的约束

需求通常用于表达做什么，而不是描述如何做（做什么与怎么做）

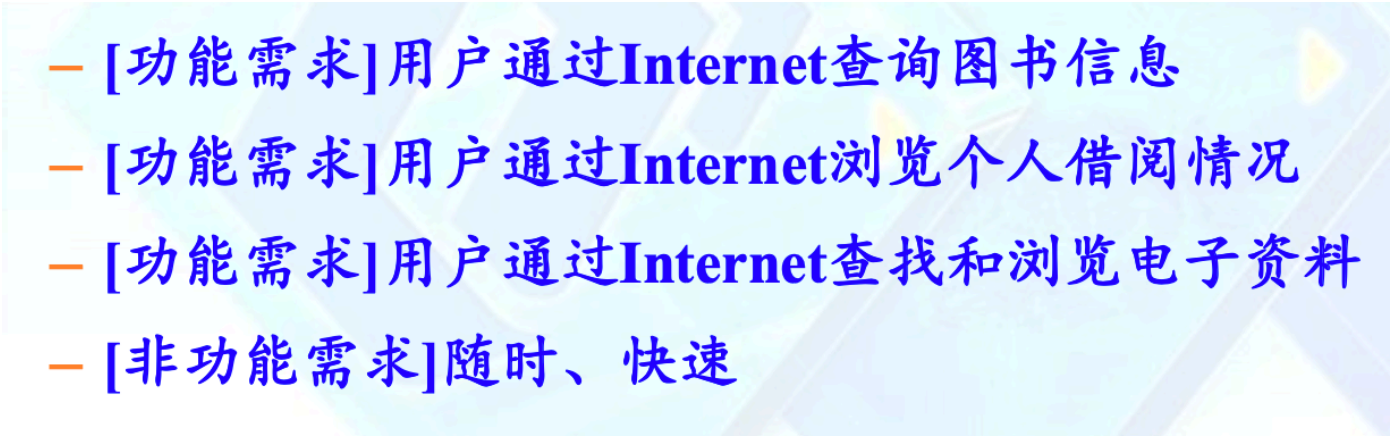
错误的需求会产生扩散效应和修复代价

## 需求的分类

业务需求（愿景需求）：客户对系统的高层次目标要求，定义了项目的远景和范畴

用户需求：从用户角度描述的系统功能需求与非功能需求，只涉及系统的外部行为（用户角度）而不设计内部特性（开发者角度）

功能性需求与非功能性需求区分




功能需求：系统应该提供的功能或服务，通常涉及用户或外部系统与该系统之间的交互，不考虑系统内部的实现细节。功能需求描述了系统所展示的可观察的行为，并且大多数是处于执行者-系统响应顺序的环境中。

非功能需求：从各个角度对系统的约束和限制，反映了客户对软件系统质量和性能的额外要求。

业务规则：对某些功能的可执行性或内部执行逻辑的 一些限定条件

通常表达为“如果...，那么...”的形式

通常是一些容易发生变化的功能

数据定义：当客户描述一个数据项或一个复杂的业务数据结构的格式、 允许值或缺省值时，他们正在进行数据定义。通过收集用户当前正在使用的各类数据表格、单据等，基于它们来获得对数据的详细定义。

约束条件：系统设计和实现时必须满足的限制条件，对 其进行权衡或调整是相当困难的，甚至是不可能的。是一种类型 的非功能需求，必须把这些需求写入软件需求规格说明。

来源:法规政策、硬件/资源限制、开发语言等

外部接口需求：描述系统与其所处的外部环境之间如何进行交互 （各类不同的接口需求）

业务需求和用户需求的区分

- 业务需求：
  - 用户能有效地纠正文档中的拼写错误
- 用户需求：
  - 找出文档中的拼写错误并通过一个提供的替换项列来供选择替换拼错的词”

好的需求应具备的特征与产生不合格需求的原因

好需求与坏需求

- 考虑以下需求是否满足“好需求”的标准，如不是，该如何修正？
  1. 在用户每次存钱时系统将进行信用检查
  2. 如果用户试图透支，系统将采取适当的行动
  3. 系统将尽可能快的响应所有有效的请求
  4. 系统允许立即使用所存资金
  5. 只有在手工验证所存资金后，系统才能允许使用它

1. 在现实情况中，用户存钱时并不需要信用检查，因此这个需求描述是错误的
2. “适当的行动”对不同的人来说有不同的解释，显然是歧义的
   改正：如果用户试图透支，系统将显示错误信息并拒绝取款操作
3. “尽快”是不可验证的，应该给出具体数量值
   改正：系统将在20秒内响应所有有效的请求
4. 与5是矛盾的

## 需求获取方法

# 03-需求建模

## 模型与建模

分析员需要一组模型来充分描述信息系统的需求

不同的模型从不同的角度描述需求或解决方案

三种常见的模型：数学模型、描述模型、图形模型

## 事件与系统需求

事件的分类：外部事件、临时事件、状态事件

外部事件：系统之外发生的事件，通常是由外部实体或系统参与者触发的（外部实体触发）

临时事件：由于到达某一时刻所发生的事件（定期触发的事件或临时事件）

状态事件：当系统内部发生了需要处理的情况所引发的事件（系统状态发生改变的事件）

**外部事件实例：**

- 外部实体的需要触发一个事务处理，如客户下订单
- 外部实体想获得某些信息，如客户查询帐户信息
- 数据发生改变需要更新，如客户地址变更
- 管理部门想获取某些信息，如管理者需要销售情况分析报告

**临时事件实例：**

- 所需的内部输出结果，如定期报表
- 所需的外部输出结果，如定期结算单、帐单等
- 突发的情况，如系统掉电、内存溢出、读写错误等

**状态事件实例：**

- 所有外部事件和临时事件都要考虑是否有状态事件发生，如销售引起库存变化、签收引起订单状态变化等

区分事件和随事件发生的一部分交互行为

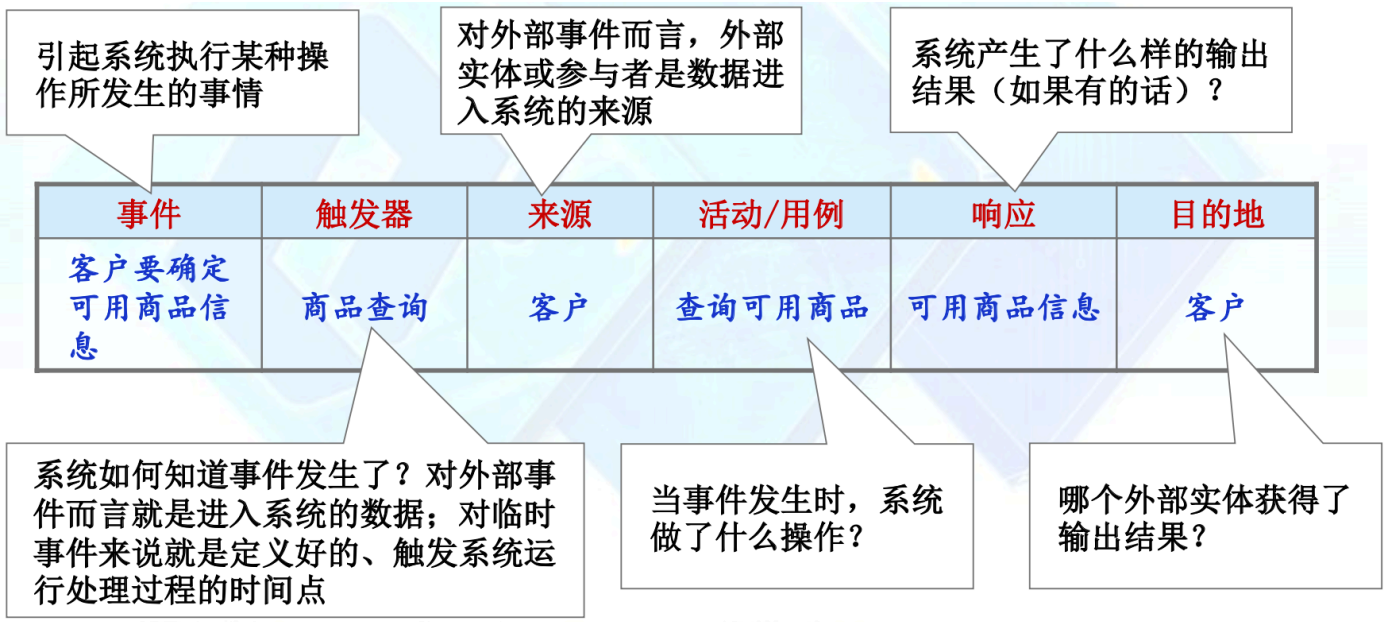
只有对系统产生直接影响的行为才能称之为事件，导致影响系统的一个事件的一系列行为，不能都算作“事件”!

跟踪事务处理生命周期可以得到一系列事件，导致许多事件的某一特定客户的“事务”处理序列，都算作“事件”!

处理某个事务的序列内容都可以算作为是事件

特别细节的事件在系统分析阶段不予考虑，特别细节的事件是时间，但是在设计阶段之前可以忽略

事件列表属于“描述性模型”




## 事物与系统需求

在传统的和面向对象的开发方法当中，事物分别代表的构成系统存储信息的相关数据和在系统当中相互交互的对象

事物之间关系

根据每件事物的关联数目来理解每种关系的本质非常重要

基数（重数）：发生在事物间关联的数目

二元关系：两种不同类型事物之间的关系

一元（回归）关系：同一类型的两个事物间的关系

事物的属性

属性：有关事物的一条特定信息

标示符（关键字）：能唯一标志事物的一个属性

复合属性：包含许多相关属性的属性

数据实体和对象

传统的系统开发方法，事物被称为数据实体

面向对象的系统开发方法，事物被称为对象

数据实体与过程是分离

对象在操作时，将数　据和处理数据的方法封装成一个单元

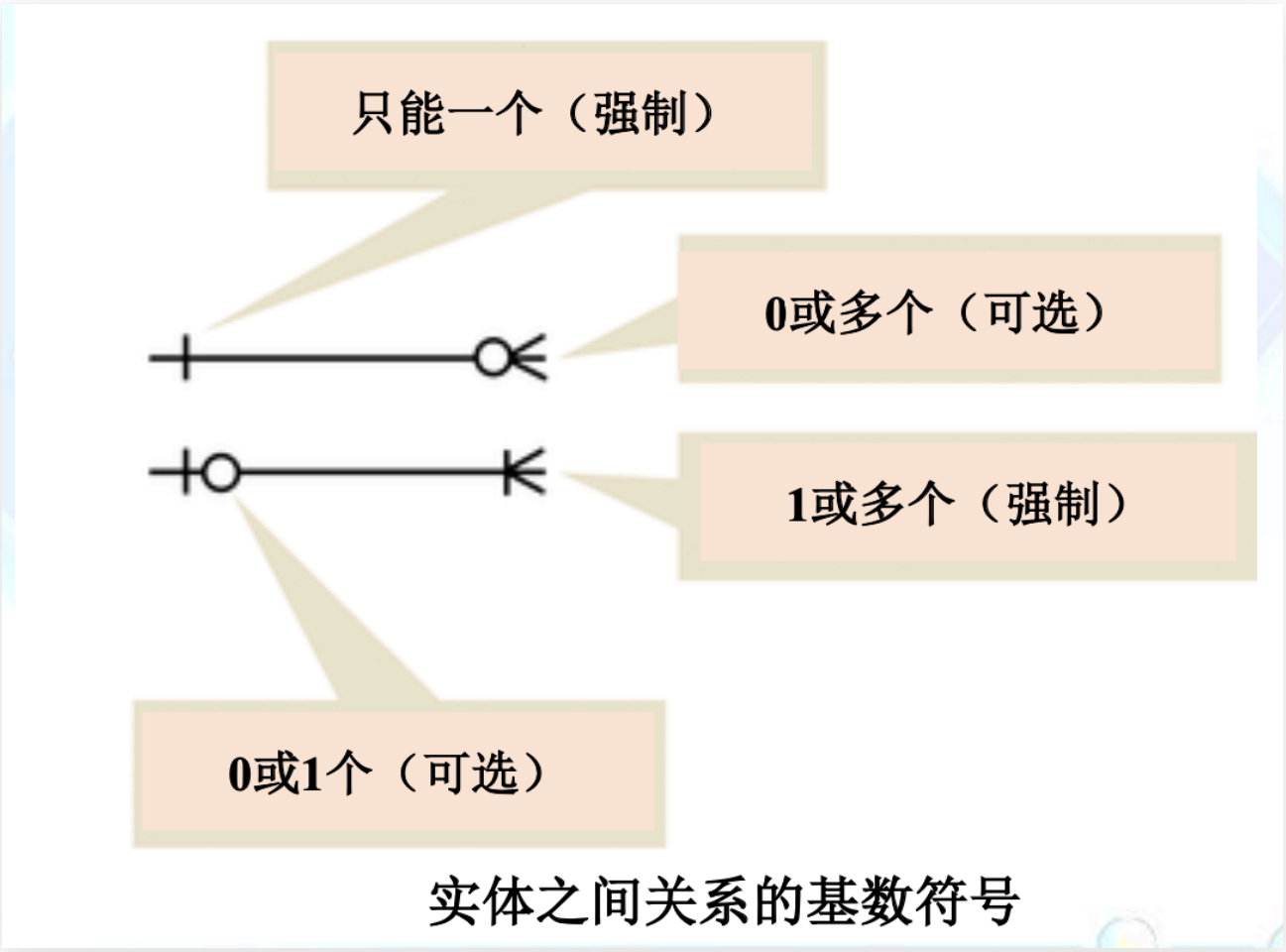
要让某个对象执行某种操作，可以让另外一个对象给该 对象发送一个消息

## 实体——关系图

传统的系统开发方法将重点集中在数据存储需求上

数据存储需求包括数据实体、数据实体的属性以及它们之间的关系

实体之间关系的基数符号




带有属性的扩展ERD图（用*表示标识符或关键字）

ERD概念：对ERD的细化工作就是对多对多的关系进行处理

　由于关系数据库当中不能直接实现多对多的关系，所以需要建立一个单独的表，用来消除多对多的关系

为解决多对多的关系而人为添加的数据实体被称为关联实体，关联实体一定包含两端数据实体的关键字

## 类图

面向对象的方法强调对系统中所包含事物的理解

面向对象的方法给事物建立的模型即是类图

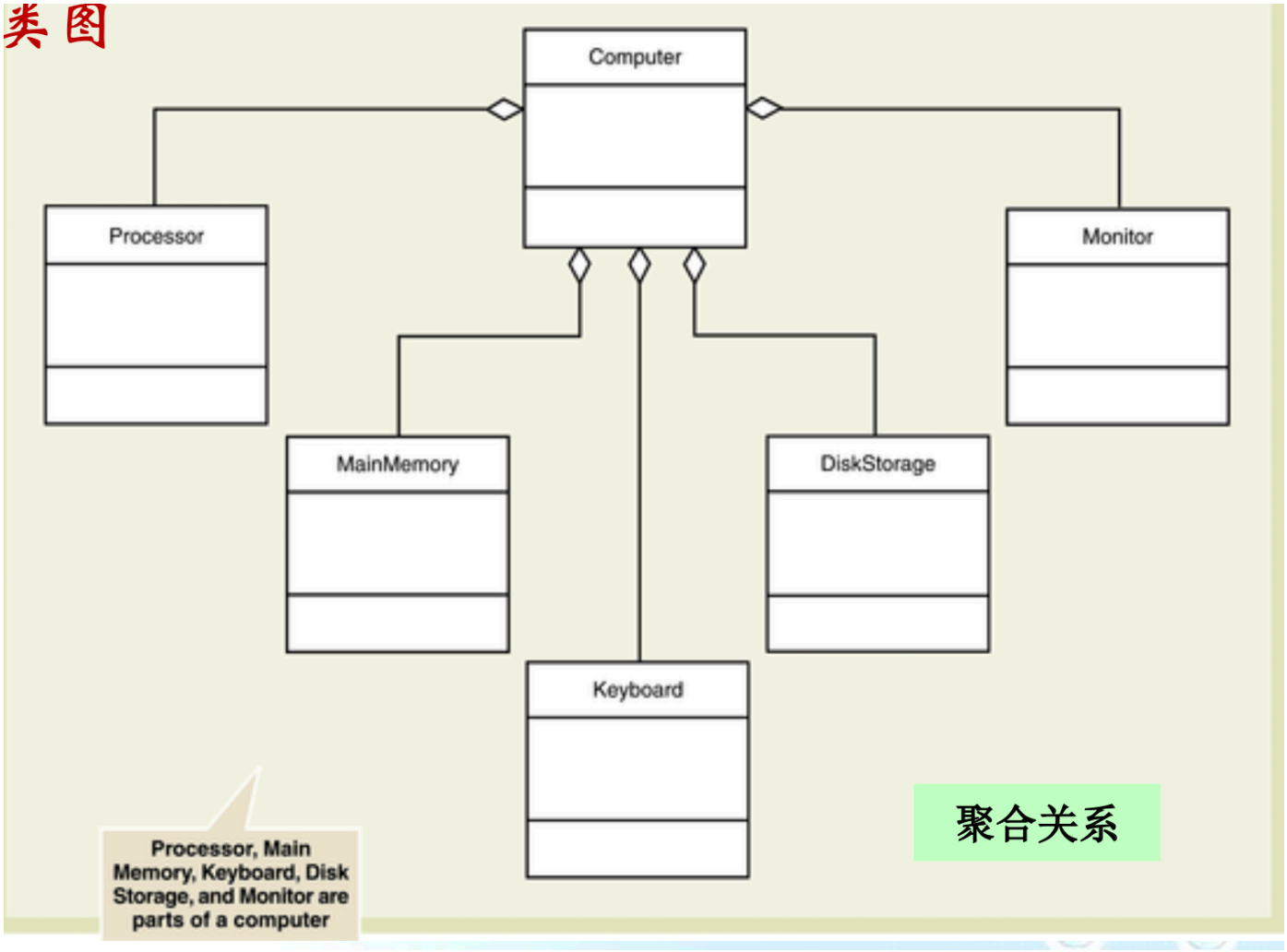
类和实体之间是存在有明显区别的

泛化/具体层次图：把类按照从最概括的父类到最具体的子类的顺序进行排列的层次图（继承层次图）

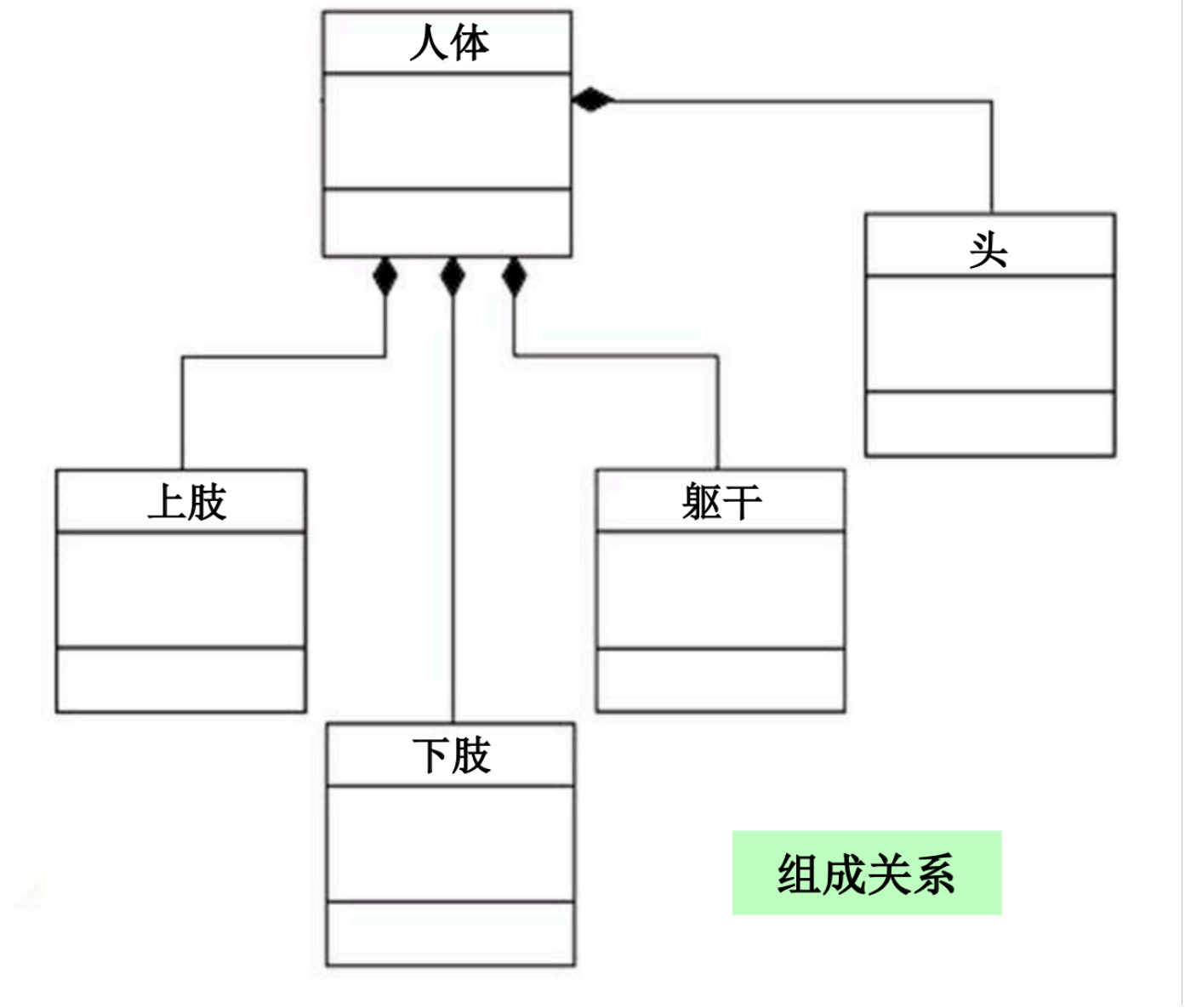
继承：允许子类共享其父类的所有的特征

整体/局部层次图：按照类之间的关联组件将类进行结构化分解的层次图

聚合:对象及其各个部分之间的一种整体-局部关系




组成:对象及其与它不可分割的各个部分之间的一种 整体-局部关系




组成关系比聚合关系具有更强的约束性

类图：name - attributes - methods

几种类

抽象类:一种不能被实例化(即不能创建对象)的类，仅为了使其子类能够继承他的属性与方法

具体类:能够被实例化的类

关联类:一种表示两种不同类之间多对多的关系的类

事件、事物抽取实例

## 系统需求建模：事件和事物

### 事件、事物抽取实例

**Waiters on Call – 餐馆送货系统– 事件与事物的抽取**

1.在发生如下事件时，系统可以进行处理：

- 客户打电话下订餐订单
- 客户打电话修改订单
- 送货司机汇报工作情况
- 送货司机上交一天的收入

2.在特定时间内系统产生所需信息（临时事件）：

- 生成日结算存款单
- 生成周末餐馆支付帐单
- 生成周销售报表
- 生成月财务报表

3.系统还需要处理以下事件：

- 新餐馆加盟
- 餐馆更改了菜单
- 雇佣了新的送货司机（送餐员）
- 送货司机（送餐员）离职

4.需要存储的信息（事物）：

- 餐馆
- 菜单
- 客户
- 订单
- 订单支付帐单
- 司机（送餐员）



“我”是系统的使用者，系统需要处理的事件是其他外部实体的行为/事件

在进行抽取时，要注意报表的相关事件，即临时事件（在特定时间内系统产生所需信息）

## 用户故事

在敏捷开发当中，需求表述为一组用户故事

在传统的oo分析方法中，需求被表述为一组用例

用户故事和用都是对需求的描述形式

用户故事：对软件用户或所有者有价值的功能性的简明的书面描述

从用户角度进行描述软件功能：角色、目标/活动、商业价值

作为一个<角色>, 我想要<活动>, 以便于<商业价值>

好的用户故事应该具备的特征：INVEST

用户故事支持验收测试

在每个用户故事背后，列出未来用户测试时可能使用的“测试用例”， 作为该故事是否已被完整实现的基本标准

敏捷开发的基本思想：在写代码之前先写测试(Test-Driven Development, TDD)

用户故事支持敏捷迭代计划

针对识别出的每一个故事，使用Story Point估算其工作量

对用户故事排列优先级-安排负责人-汇聚为迭代计划并发布-开发过程中监控进度

# 用例建模

用例：表示系统所提供的服务或可执行的某种行为

用例是站在用户角度定义软件系统的外部特性

用例的四大特征

行为序列（用例由一组可产生某些特定结果的行为构成 这些行为是不可再分解的）

系统执行（系统为外部角色提供服务）

可观测到的、有价值的结构（用例必须对用户产生价值）

特定的角色（某人、某台设备等能够触发某些行为）

用例方法的基本思想：用户不想了解系统的内部结构和设计，所关心的是系统所能提供的服务，即被开发的系统将是如何被使用的

用例模型的构成元素：

参与者(Actor) :存在于被定义系统外部并与该系统发生交互的人或其他系统，代表系统的使用者或使用环境

用例(Use Case):表示系统所提供的服务或可执行的某种行为

通讯关联(Communication Association) :用于表示参与者和用例之间的对应关系，它表示参与者使用了系统中的哪些服务(用例)、系统所提供的服务(用例)是被哪些参与者所使用的

通讯关联

通讯关联表示的是参与者和用例之间的关系

箭头表示在这一关系中哪一方是对话的主动发出者，箭头所指的方向是对话的被动接受者

如果不强调在对话当中的主动与被动关系，可以使用不带箭头的关联实线

通讯关联不表示在参与者和用例之间的信息流，并且信息流是双向的，与通讯关联箭头所指的方向没有关系

用例有用例规约进行描述，不等于 椭圆 + 姓名

用例方法的优点

系统被看作为是一个黑箱，不关心如何进行具体功能的实现

用例方法描述了系统有哪些使用者（Actor）以及系统和参与者之间的服务（Use Case）

用例模型容易构建且容易阅读

站在用户的角度，从系统外部来描述功能

帮助系统的最终用户参与到需求分析的过程当中，使其需求更容易表达出来

# 用例建模的基本过程

- 识别并描述参与者

  特殊的参与者（系统时钟）

  有些操作不是由外部的人或系统触发的

  抽象出系统时钟 系统系统会触发周期性任务（系统内的定时操作）

- 识别用例，并给出简要描述

  参与者需要系统提供什么样的服务

  参与者如何使用系统

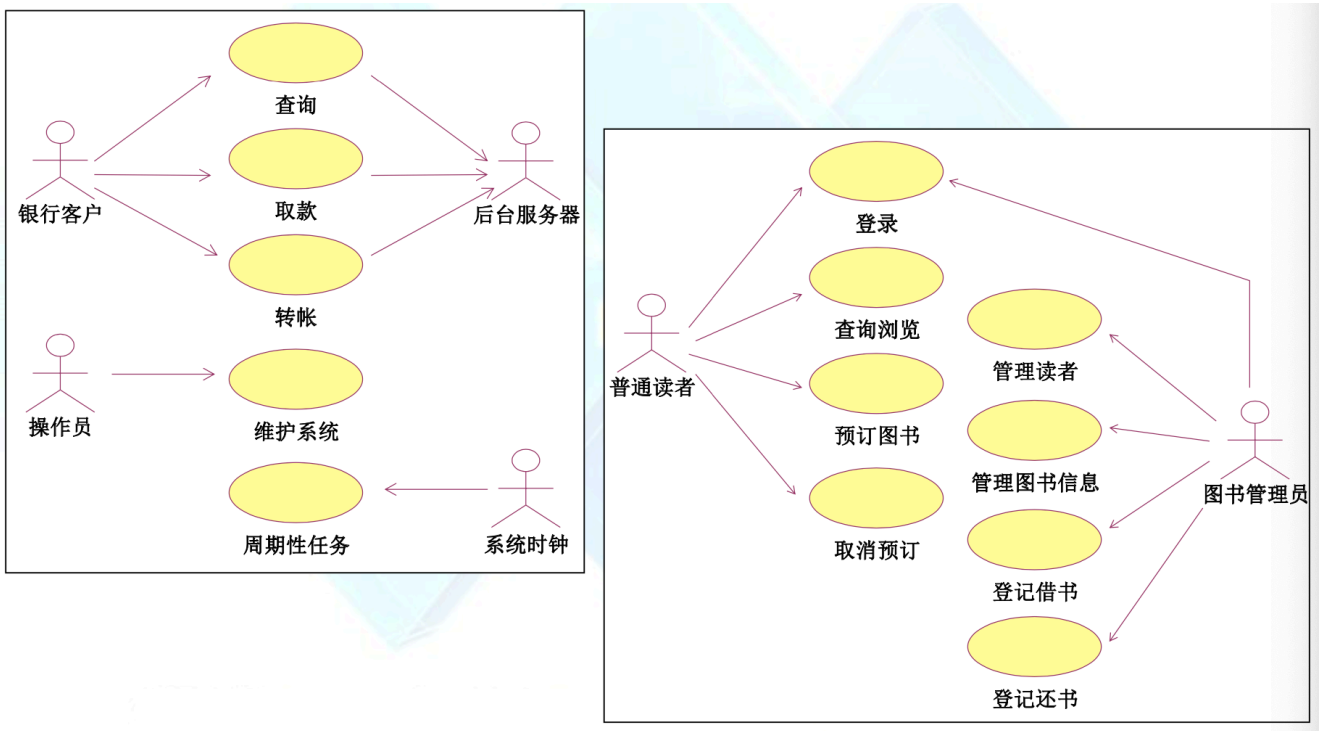
  几个注意点：

  用例必须是由某一个actor触发而产生的活动，即每个用例至少一个actor

  如果存在与actor不进行交互的用例，需要将其并入其他用例，或者是检查该用例相对应的参与者是否被遗漏

  每个参与者也必须至少涉及到一个用例

- 识别参与者和用例之间的通讯关联




如图所示，通讯关联也可以由用例指向参与者

- 给出每一个用例的详细描述

  单纯的用例图并不能描述完整的信息，需要用文字描述不能反映在图形上的信息（用例规约）

  事件流可以分为常规流和备选流（处理异常或偶尔发生的一些情况）

  对常规事件流

  每一个步骤都需要用数字编号以清楚地标明步骤的先后顺序

  用一句简短的标题来概括每一步骤的主要内容

  对每一步骤，从正反两个方面来描述 （参与者向系统提交了什么消息，系统作出什么样的响应）

对备选事件流

备选事件流的格式与基本流的格式基本保持一致

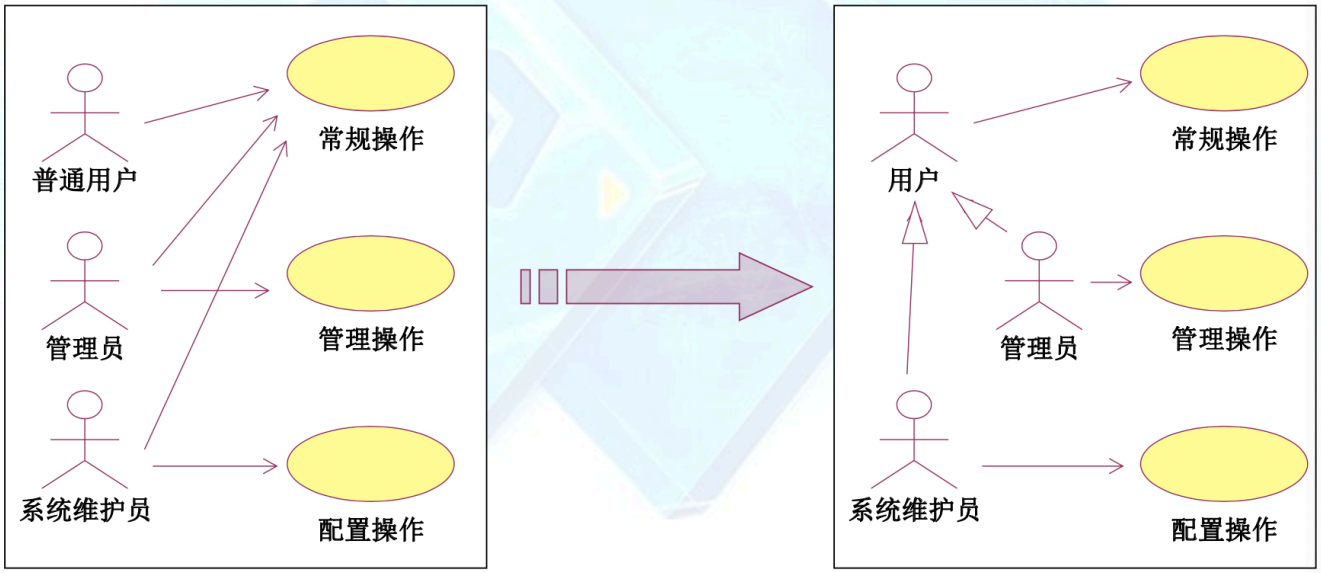
备选事件流有四个有机组成部分 （起点、条件、动作、恢复）

- 细化用例模型

抽取公共信息，使得用例模型更易于维护

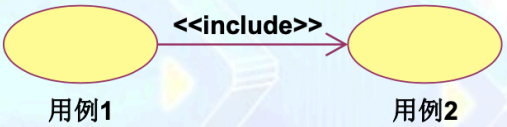
参与者与参与者之间的泛化

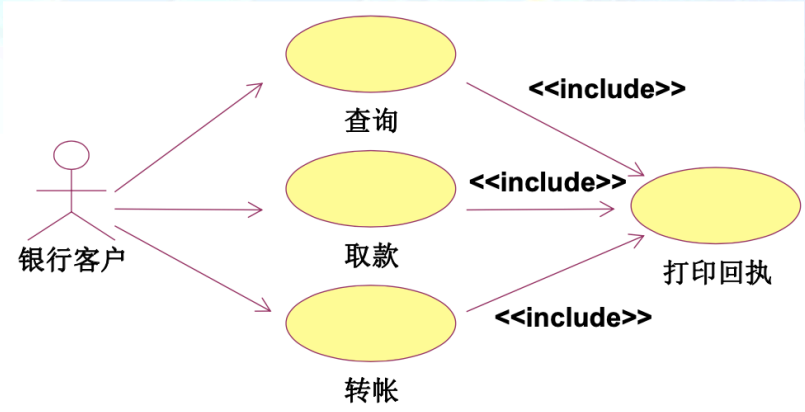
- 参与者之间也可以有泛化(Generalization)关系




用例和用例之间的包含

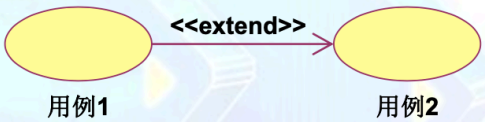
- “包含关系”是通过在关联关系上加入<<include>>标记来表示
- 语义：用例1会用到用例2，用例2的事件流将被插入到用例1的事件流中

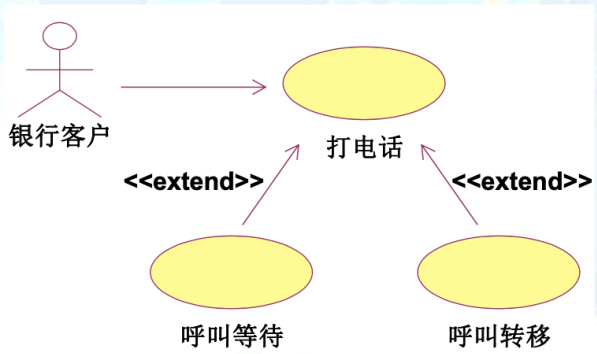



用例和用例之间的扩展

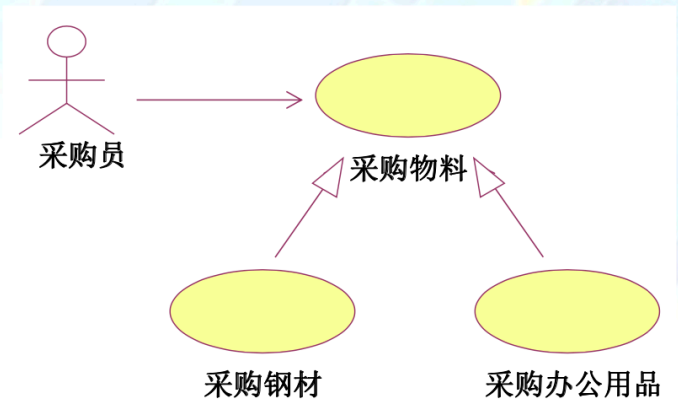
- “扩展关系”是通过在关联关系上加入<<extend>>标记来表示
- 语义：用例2在某些特定情况下会用到用例1，此时，用例1的事件流将被插入到用例2的事件流中




用例和用例之间的泛化

- 当多个用例共同拥有一种类似的结构和行为的时候，可将它们的共性抽象成为父用例，其他的用例作为泛化关系中的子用例
- 子用例继承了父用例所有的结构、行为和关系



## 需要注意的问题

- 用例的粒度

  用例识别的标准:actor与系统之间的一次独立交互（独立存在的功能体）

  如果多次交互总是同时发生且不会单独发生，可合并为一个用例

  对过大的用例进行拆分

- 用例是actor与系统的交互

  actor与actor在现实当中的交互不应包含在use case中

- actor与外部系统

  系统自己做的事情，不是单独的用例:系统的行为受到actor的触发(可能是系统时钟)

  与系统发生作用的外部系统是单独的用例

## 用例模型的提交物

- 用例模型
- 每个用例的详细描述
- 术语表：所用到的术语说明
- 补充规约：非功能性需求的说明

## 活动图和泳道图

提供一种可视化的流程图方式，对use case的事件流进行直观展示，以便于读者更好的理解

两种形式:

- 传统的活动图:只涉及一个参与者
- 泳道图(swim-lane diagram):侧重于描述多个参与者的活动之间的交互关系

活动图的基本要素

- 起始点，结束点
- 活动；决策点
- 活动之间的时序连接；活动之间的并发点
- 泳道

## 用例分析的重要概念

用例分析：一种把现实世界的需求捕获下来的方法

用例图：用例分析的UML模型图

涉众：与未来系统有利益相关的一切人或事(外部系 统)，涉众的利益诉求会影响系统的建设结果

参与者：涉众中的一部分，直接对未来系统发出动作， 或直接从系统中接收反馈

主角：处于系统边界之外的参与者

业务工人：一种参与者，处于系统边界之内，直接参与业务过程，俗称“人肉”

用例：一个用例定义了一组用例实例，其中每 个实例都是系统所执行的一系列操作，这些操作 生成特定主角可以观测的值

用例就是一件事情，要完成这件事情， 需要做一系列的活动，并产生期望的结果

用例实例（场景）：做一件事情可以有很多种不同的方法和步骤，每种方法和步骤称之为用例场景，一个场景就是一个用例实例

业务用例：用于业务建模的用例，定义业务范围

概念用例：获取业务用例中的核心业务逻辑步骤

系统用例：用于功能需求分析的用例，定义系统范围

用例粒度 ： 用例内涵的大小

边界：用例分析工作的范围，可以是组织/系统/领域

粒度与边界：边界定义的大小与用例粒度大小成正比关系，在边界不变的情况下，边界内的用例粒度是不变的

用例分析的目标：最终在找到系统用例，完成功能性需求分析

业务用例 -> 概念用例 -> 系统用例

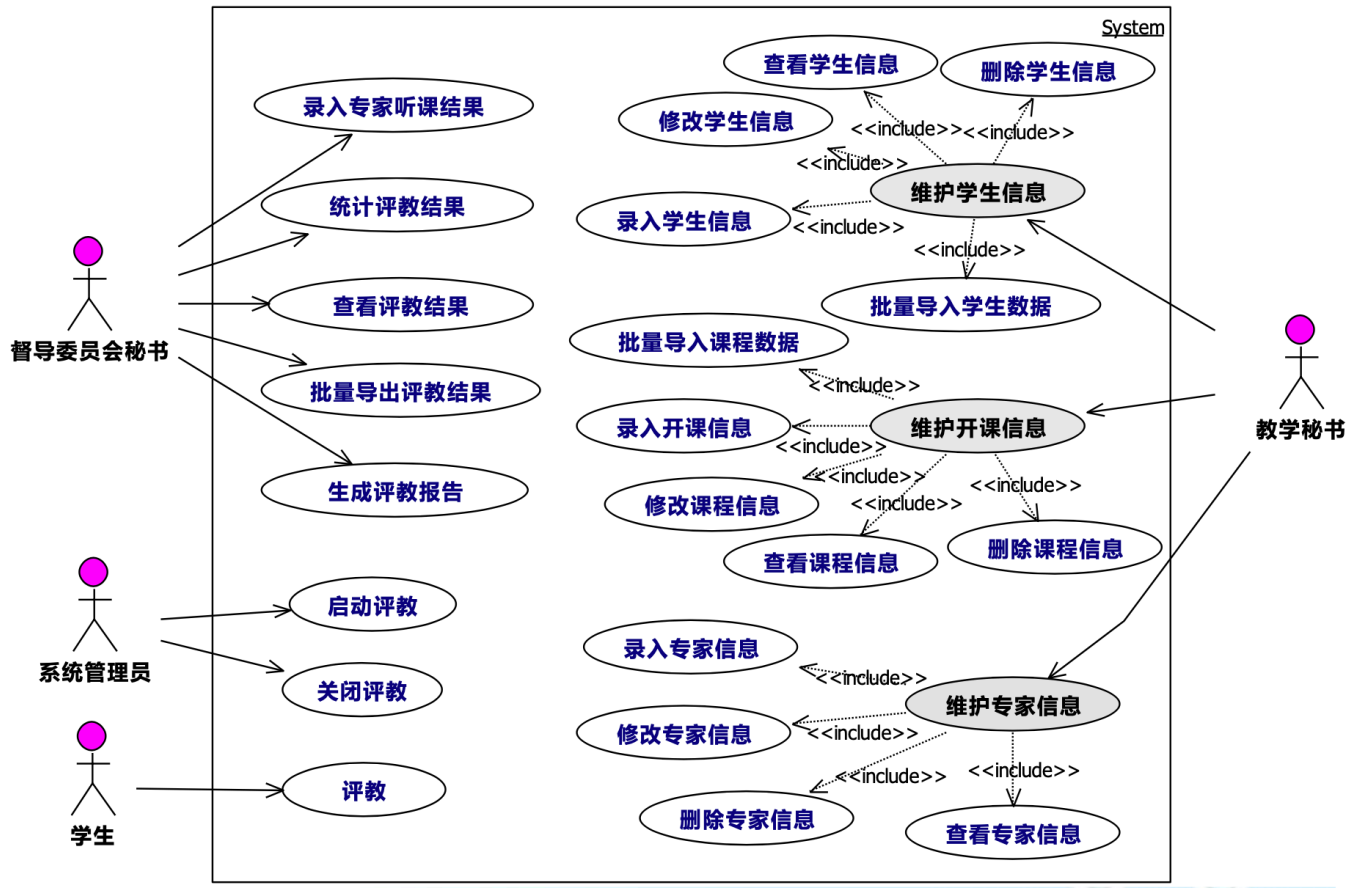
用例的命名：动宾词组构成

用例识别：

(1) 确定参与者(主角)，即确定了边界;

(2) 找出相关用例，方法是以该用例是否完成了参 与者的某个完整目标为依据

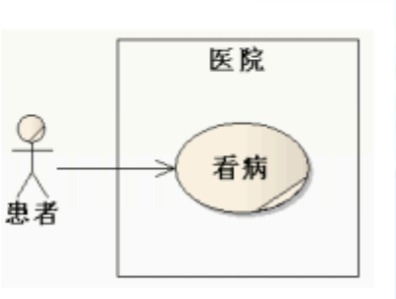
# 用例分析的重要概念

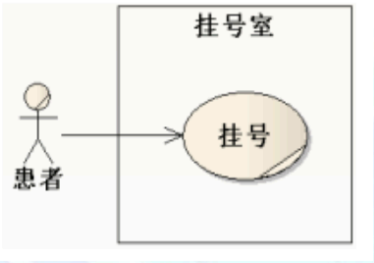


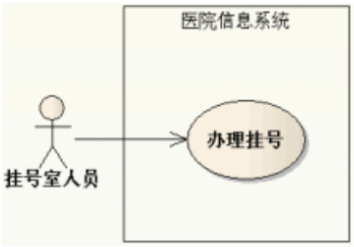



维护信息当中include有增删改查的模块

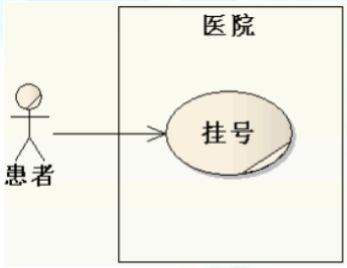
# 边界 VS 用例及粒度

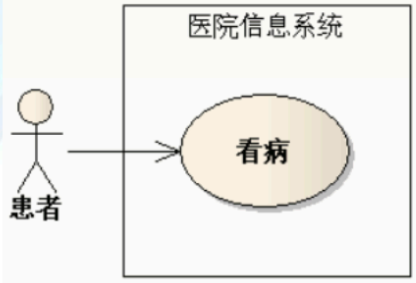


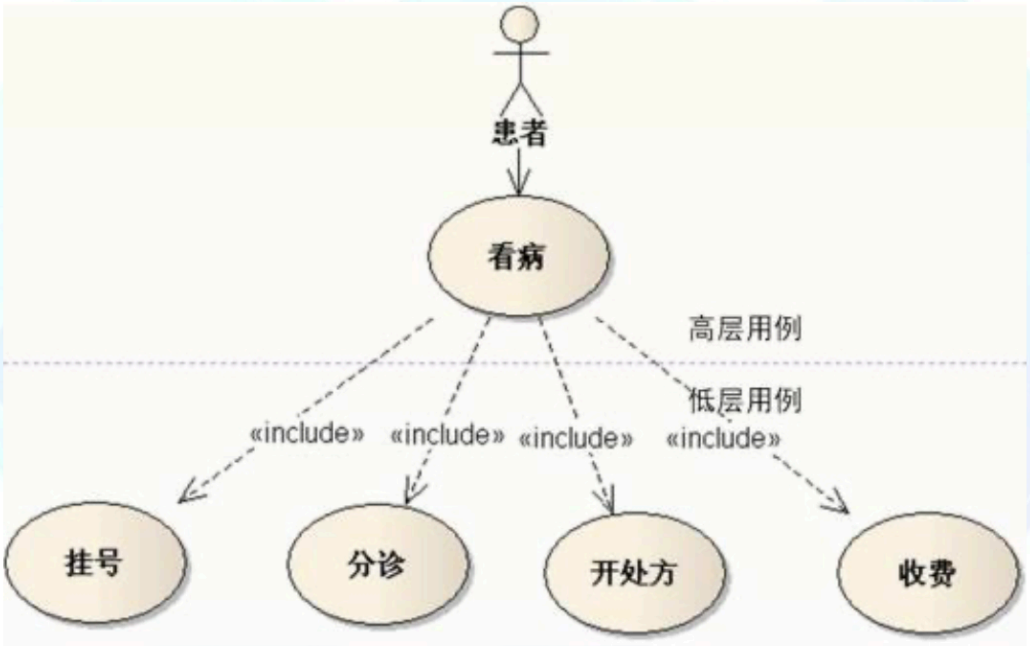




正确的边界与正确的用例 √



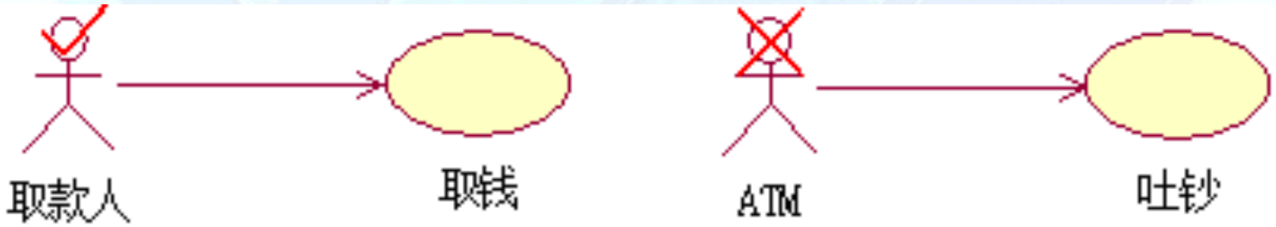




正确的边界、错误的用例 ×



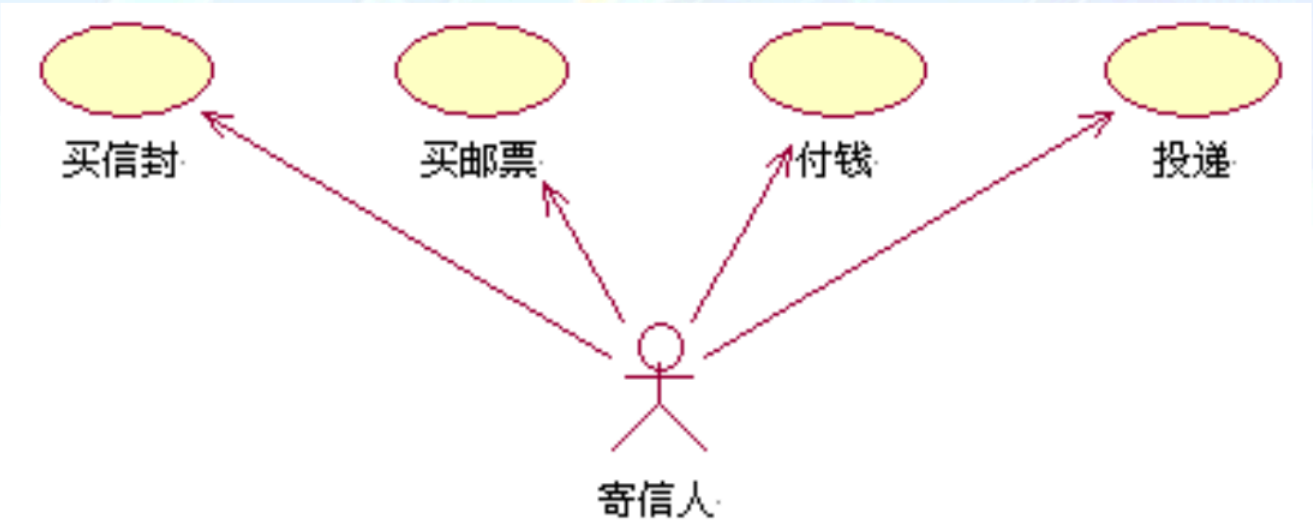

边界混乱：上面的边界是“医院”，下面的边界是“医院管理系统”

填写取款单不是取款人的目的，因此不是用例

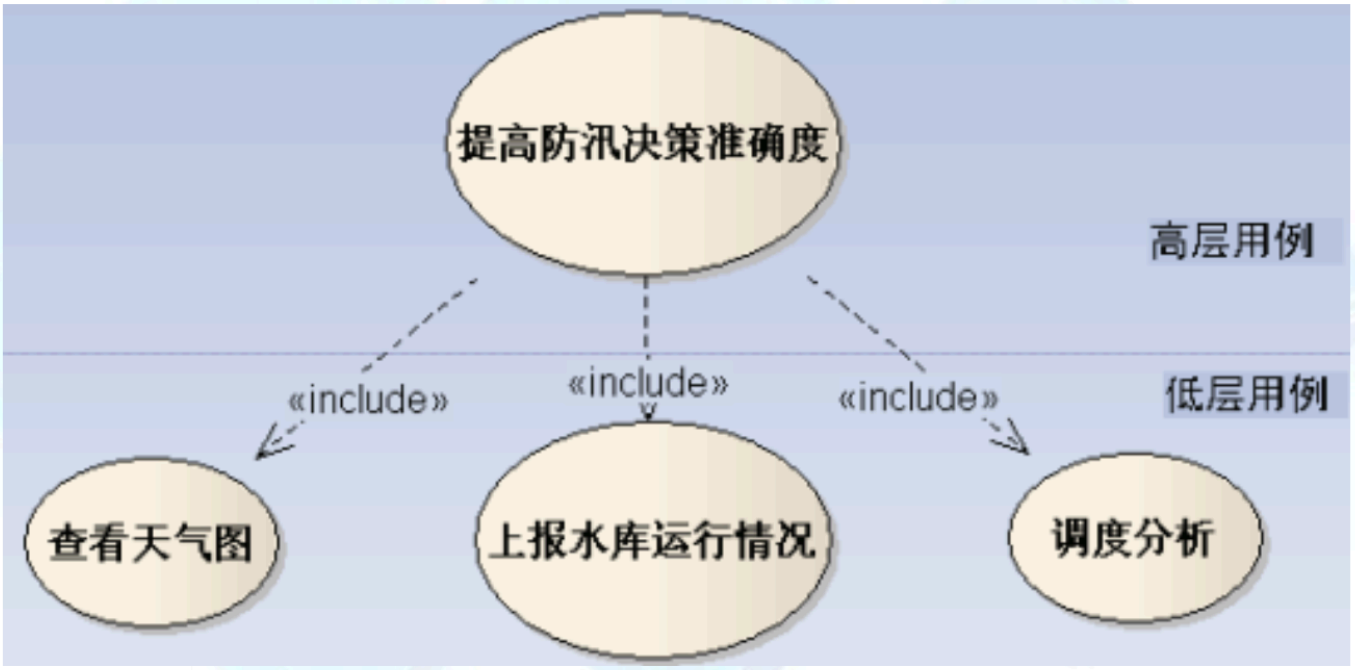



ATM是没有吐钞的愿望的，因此不能驱动用例

以完整目标作为用例！！！




不能以步骤作为用例！！！




不要把领导的愿望作为用例

# 04-系统分析

## 结构化方法与面向对象方法

结构化方法利用算法解决问题，同时寻找适当的方法存储数据

面向对象方法的系统由能够完成一组相关任务的对象组成，对象有其所负责的任务，当其依赖不负责的任务时，需要访问其他对象。一个对象不能直接操作另一个对象内部的数据，它也不能使其它对象直接访问自己的数据。不同对象之间的交流都必须通过方法调用。

面向过程(事件)的设计思路是首先分析问题的步骤。

面向对象的设计思路是分析与问题有关的对象。

## 结构化开发方法

- 起源时间：20世纪60年代产生软件危机，70年代出现“结构化开发方法”
- 思想方法：自顶向下、问题分解、分而治之、由分到合；
  软件/程序 = 数据结构 + 算法，软件结构化、模块化、层次化
- 表达模型：分析模型（系统流程图、数据流图、 ERD、数据字典）
  设计模型（系统结构图、程序流程图、ERD）
- 分析与设计线索：
  面向过程（处理） – 过程驱动
  面向数据 – 数据驱动
- 优　　点：思维自然，符合人们思考问题的方式；总体可控性强；适合偏重数学计算方面的项目
- 缺　　点：不容易描述客观世界的需求；耦合性相对高

## 基于UML的面向对象开发方法

- 起源时间：20世纪90年代末
- 思想方法：开发一整套系统分析与设计模型及描述方法，从形式上和概念上统一描述软件系统
- 表达模型：用例图、活动图、状态图、序列图（协作图）、类图、组件图、包图、部署图等
- 优　　点：统一标准；容易交流；建模工具较多；适合大型信息系统发
- 缺　　点：模型不容易掌握

## 结构化的系统分析方法

结构化分析方法(SA):将待解决的问题看作一个系统，从而用系统科 学的思想方法(抽象、分解、模块化)来分析和解决问题

自顶向下的分解(top-down)

帮助开发人员定义系统需要做什么(处理需求)，系统 需要存储和使用哪些数据(数据需求)，系统需要什么样的输入和输出以及如何把这些功能结合在一起来完成任务

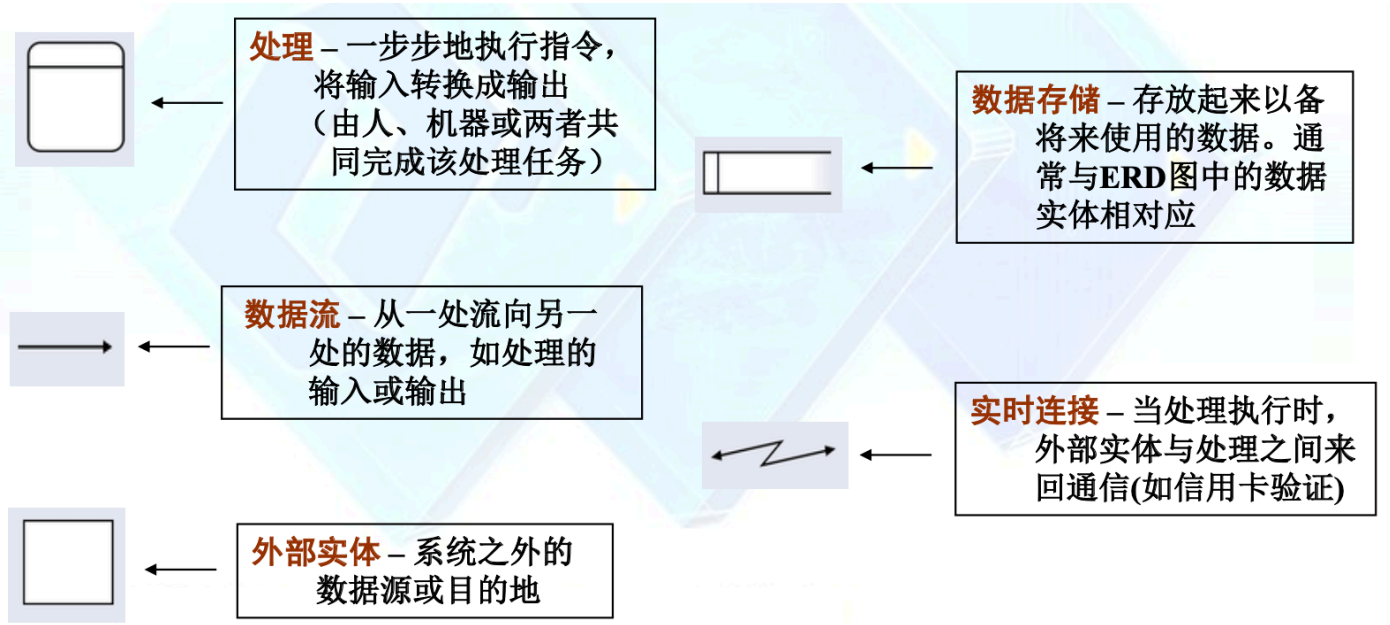
基于数据流的需求分析建模 -- DFD：数据流图(DFD图)

实体-关系图(ERD，IDEF1X图)

# 数据流图

数据流图：用处理、外部实体、数据流以及数据存储（DFD）来表示系统需求的图表

- 图形元素少且符号简单易懂
- 较充分表达系统的主要需求：输入、输出、处理和数据存储




从外部实体出发的数据流可以是

外部实体给出的”处理请求“，即一个事件

外部实体给出的”输入数据“

指向外部实体的数据流一般是处理的反馈或处理结果

DFD图可以描述高层次的具有高度概括的系统处理 也可以描述低层次的具有更详细分解的系统处理

抽象层次:把系统分解成一个逐步细化的分层集合的建模技术

DFD图：关联DFD（顶层DFD） - 0层DFD - 1层DFD

## 关联DFD图

在单个处理符号中概括系统内所有处理活动的DFD

关联图在表达系统边界时用处很大

系统的范围是通**过单个的处理和外部实体所表示的事物**来定义的

数据存储不画在关联图中是因为它本身被认为是系统内部的内容

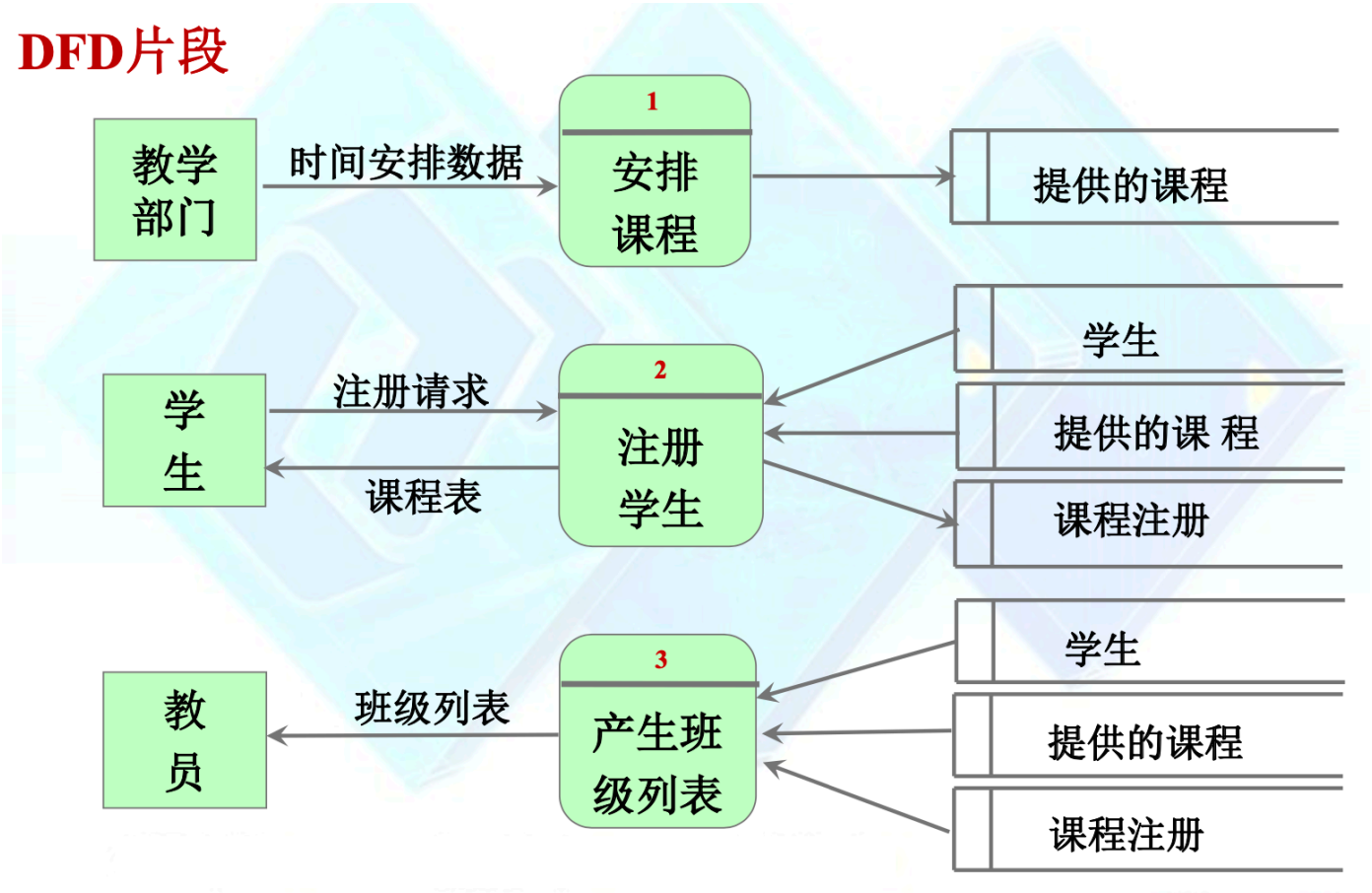
当一个系统响应事件较多时，常常将系统分成多个子系统，并为 每个子系统创建一张关联图

数据存储是不画在关联DFD图当中的

## DFD片段

用一个单一处理符号表示系统响应一个事件的DFD

- 在DFD片段当中，展示了处理、外部实体和内部数据存储之间的交互细节
- 每个DFD片段仅显示要响应该事件的相关的那些数据存储
- 一个DFD片段是为事件表中的每一个事件创建的




## DFD的0层图

将一个系统或子系统的所有DFD片段组合到一个单个的DFD图中，这样的DFD图称为事件分离的系统模型/0层图

0层DFD图:“处理”的编号为i

分解“处理”-- 理解“处理”的细节: 一个DFD片段中的处理也可以包含多个更小的处理有时分析员需要将该DFD片段进一步细化，生成更详细层次的DFD图

结构化的系统分析 DFD建模–基于数据流的系统分析

## DFD的层次

分解“处理”-- 理解“处理”的细节: 一个DFD片段中的处理也可以包含多个更小的处理

有时分析员需要将该DFD片段进一步细化，生成更详细层次的DFD图

1层DFD图:将0层DFD中的处理进一步细化等到的DFD图 “处理”的编号为“i.j”

2层DFD图:将1层DFD中的处理进一步细化等到的DFD图 “处理”的编号为“i.j.k”

## DFD的质量评估

高质量的DFD的要求

- 可读性强
- 内部一致
- 能够准确描述系统需求

措施

- 最小化复杂度

  使每幅DFD图尽量简单易懂，避免信息超量

  信息超量即当太多信息同时显现时所发生的难以理解的情况

  可以采用分层结构将DFD划分为小的且相对独立的子集（便于进行逐级阅读、考察DFD）

  构建DFD图的7±2原则

  单个DFD中不应该有超过7±2的处理

  单个DFD中不应该有超过7±2个数据流进出同一个处理/数据存储

  接口最小化：DFD中各个元素之间的连接数越来越少

- 保证数据流一致性

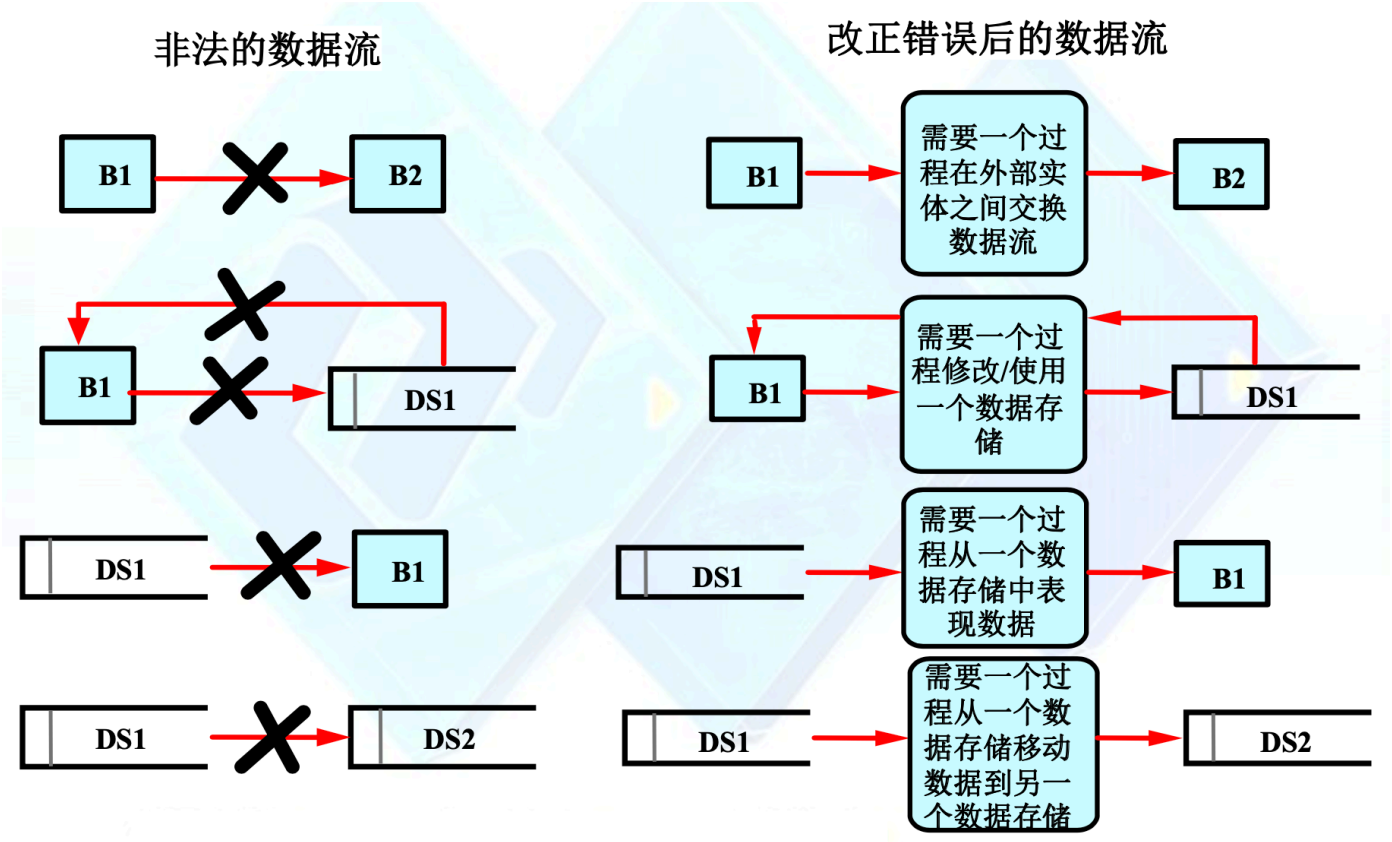
  数据流的一致性表现在三个方面

  - 一个处理和该处理被详细分解后在数据流内容应该一致
  - 对一个处理，有数据流入必须要有响应的数据流出
  - 对一个处理，有数据流出必须要有响应的数据流入

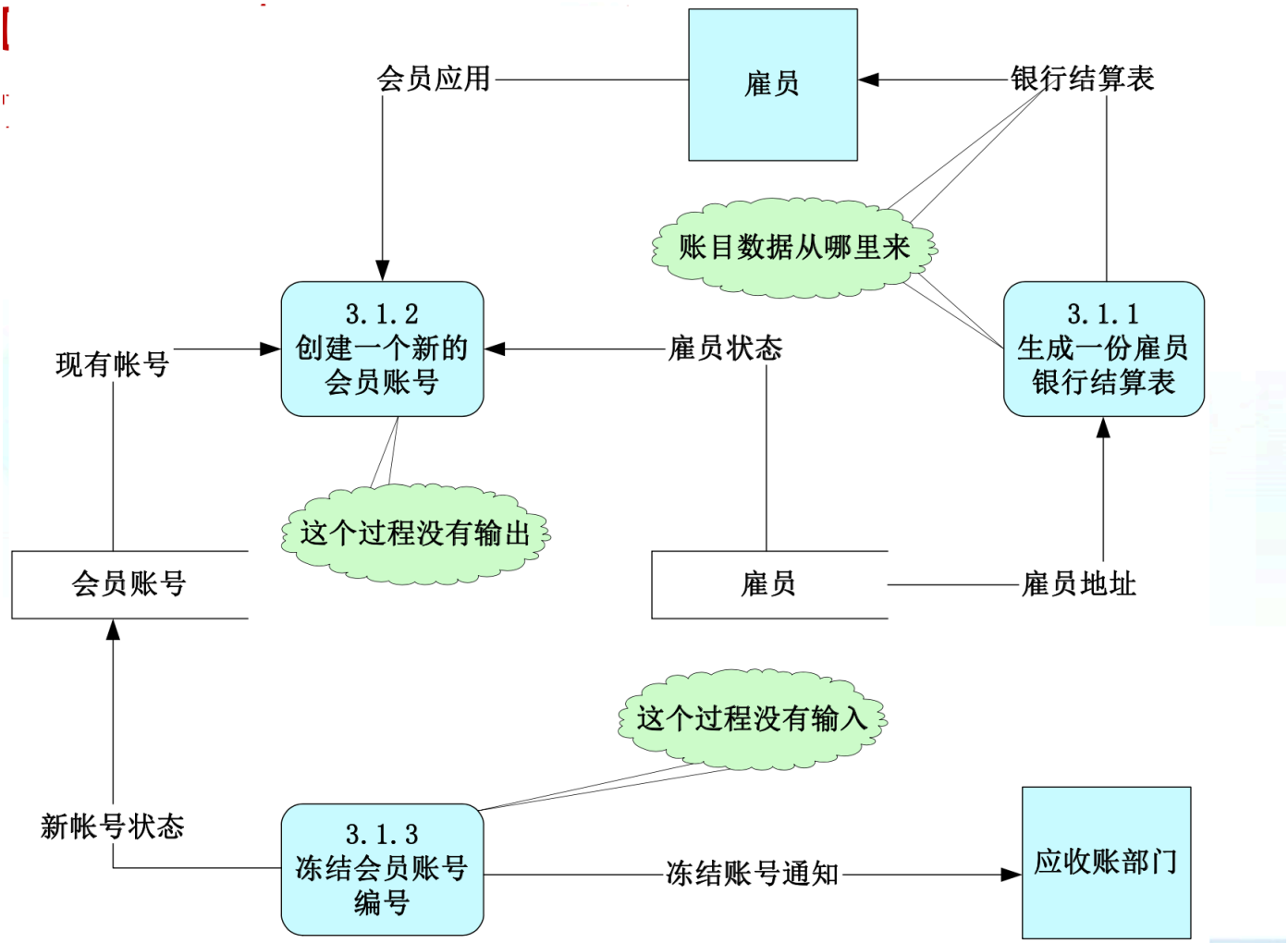
  黑洞：带有输入数据的但并**不用其产生输出数据**的处理或数据存储（带有不必要数据输入的处理）

  奇迹：没有足够数据元素作为输入或产生来源的一个处理或数据存储（有不可能的数据输出的处理）

数据流图中典型错误总结




两个元素之间进行数据流的传输（交换、存储、表现、移动）需要一个过程/处理。






数据流图中典型错误例子

有输出无输入，有输入无输出，输入输出二者不对等

## DFD模型总结

在关联图中不画数据存储

数据流不反映处理顺序，显示数据通过系统的流程，因此处理可以并行工作。

处理/数据存储既要有输入也有输出

若输入数据流不完全用来产生输出数据流，称之为黑洞

若输出数据流不完全依赖于输入数据流，称之为奇迹

在传统方法中，DFD图在一个图中描述了三个元素：

- 处理 -- 每个最底层“处理”需要详细描述其细节
- 数据流 -- “数据流”包含的数据元素需要定义
- 数据存储 -- “数据存储”需要在ERD中定义

其中数据流和数据存储可以合并用“数据字典-DD”来进行描述

## DFD细节内容描述

- “处理”细分解，层层分解，直到可详细描述细节
- 结构化语言/伪代码

  一种描述“处理”的规范方法，将结构化编程技术和叙述性英语结合起来

  结构化英语适合用来描述处理步骤清楚、控制逻辑相对简单的情况

  结构化语言的特点：

  介于自然语言和形式语言之间的语言
- 决策表/决策树

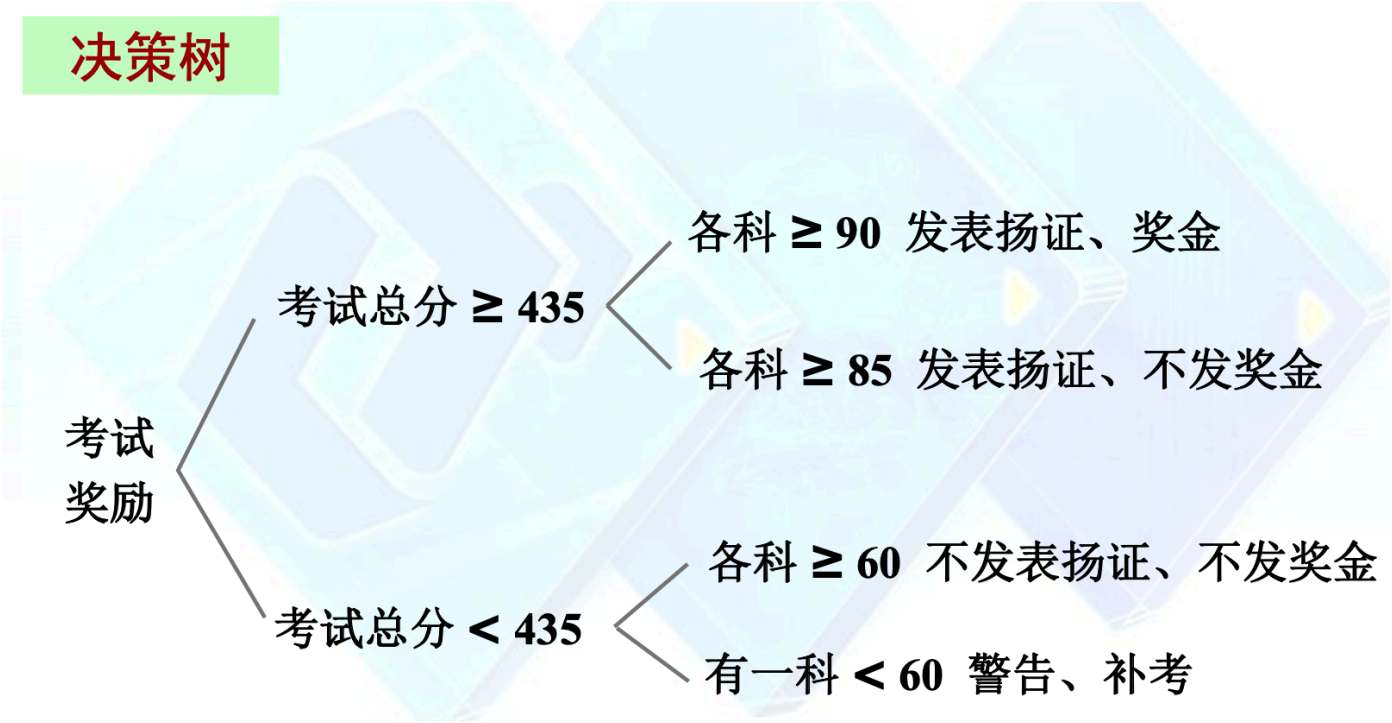
  决策表:“处理”逻辑的一种表格形式的表示方法，其中包括决策变量、决策变量值、行为或公式

  决策树:用树形结构组织起来的线条对“处理”逻辑进行图形化的描述

### 决策表

| | | 规　则 | | | |
|---|---|---|---|---|---|
| 条件 | 考试总分 | ≥ 435 | ≥ 435 | < 435 | < 435 |
| | 单科成绩 | ≥ 90 | ≥ 85 | ≥ 60 | < 60 |
| 动作 | 发表扬证 | √ | √ | × | × |
| | 发奖金 | √ | × | × | × |
| | 警告 | × | × | × | √ |

## 决策树




## 数据字典（DD）

数据字典是数据分析的描述模型，包括：数据项定义、数据结构定义、数据流定义、数据存储定义

广义的数据字典，也可以包含对“处理”的描述

数据项：数据的基本单位

数据结构：由数据项组成，它给出了数据基本结构单位

数据流：数据结构在系统中的流通路径

　数据流是数据元素的集合，数据流定义就是列出其包含的所有数据项

数据元素还需进一步的定义：说明类型、长度等，一般在设计阶段详细定义

数据处理(广义DD)：给出处理的流程和说明信息

## 数据分析(ERD、IDEF1X)

对DFD中的数据存储的分析非常重要，单独进ERD建模

数据存储需求包括**数据实体、数据实体的属性以及它们之间的关系**

对ERD进行细化的工作，即处理多对多的关系以及细化属性说明

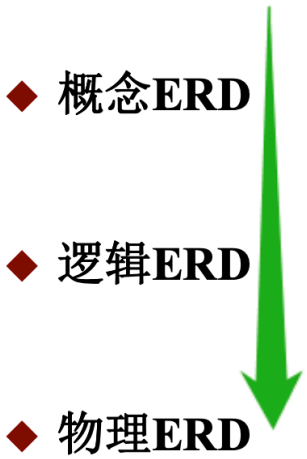
由于关系数据库中不能直接实现多对多的关系，因此必须建立一个单独的表，来消除多对多的关系

关联实体：（多对多关系的过渡）

　为解决多对多的关系而人为增加的数据实体，它一定包含了两端的关键字

　多对多 —> 多对一+一对多

实体关系图 – ERD




| ERD形式 | 概念ERD | 逻辑ERD | 物理ERD |
| --- | --- | --- | --- |
| 实体（名称） | √ | √ | √ |
| 关系 | √ | √ | √ |
| 关联实体 | | √ | √ |
| 属性（名称） | | √ | √ |
| 属性的类型、长度、说明 | | 随意 | √ |
| 主键 | | | √ |
| 外键 | | | √ |

概念ERD - 逻辑ERD - 物理ERD （2-4-7）

## 面向对象的分析方法概述

分析业务领域，找出问题解决方案，发现对象，分析对象的内部构成和外部关系，建立软件系统的对象模型

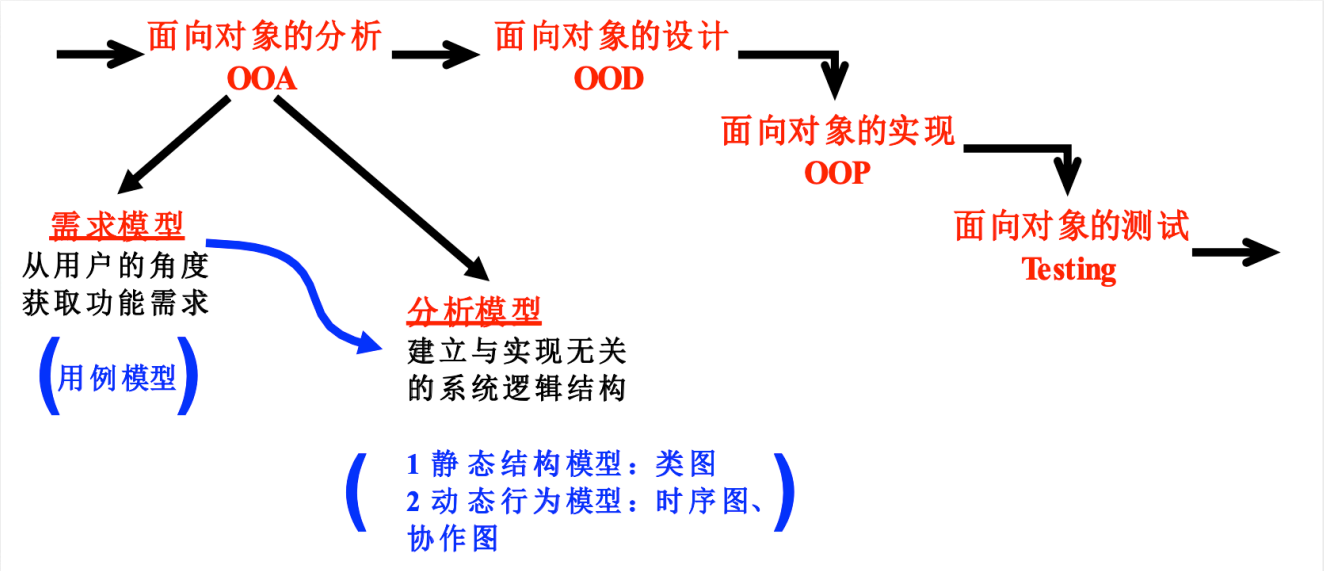
着重分析业务领域和系统责任，建立**独立于实现**的OOA模型，暂时忽略与系统实现有关的问题

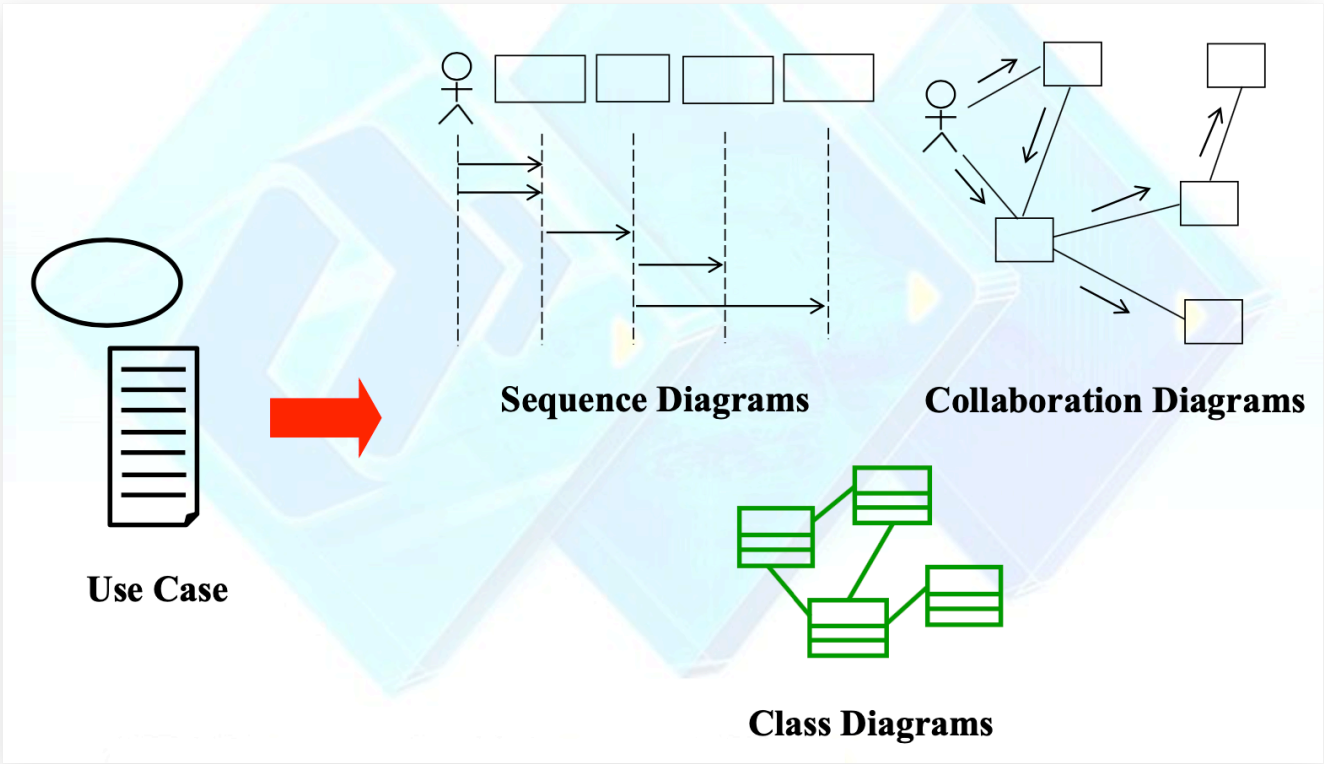
主要使用5种图描述完整的系统需求：用例图、类图、时序图、协作图、状态图

面向对象的分析模型由三类模型构成

- 功能模型：从用户的角度获取功能需求，由用例模型表示
- 静态结构模型（分析对象模型）：描述系统的概念实体，由类图表示
- 动态行为模型：描述对象之间的交互行为，由时序图和协作图表示

静态结构模型和动态行为模型简历与实现技术无关的系统逻辑结构




## 面向对象分析的过程

- 第一阶段：业务领域分析

  分析应用领域的业务范围、业务规则和业务处理过程，确定系统的责任、范围和边界，确定系统的需求

  在分析中需要着重对系统与外部的用户和其他系统的交互进行分析，确定交互的内容、步骤和顺序

  业务用例模型 → 系统用例模型

- 第二阶段：发现和定义对象和类

  识别对象和类，确定它们的内部特征:属性与服务操作

  这是一个从现实世界到概念模型的抽象过程，而抽象是面向对象分析的基本原则

- 第三阶段：识别对象的外部联系

  识别对象与对象、类与 类之间的各种外部联系

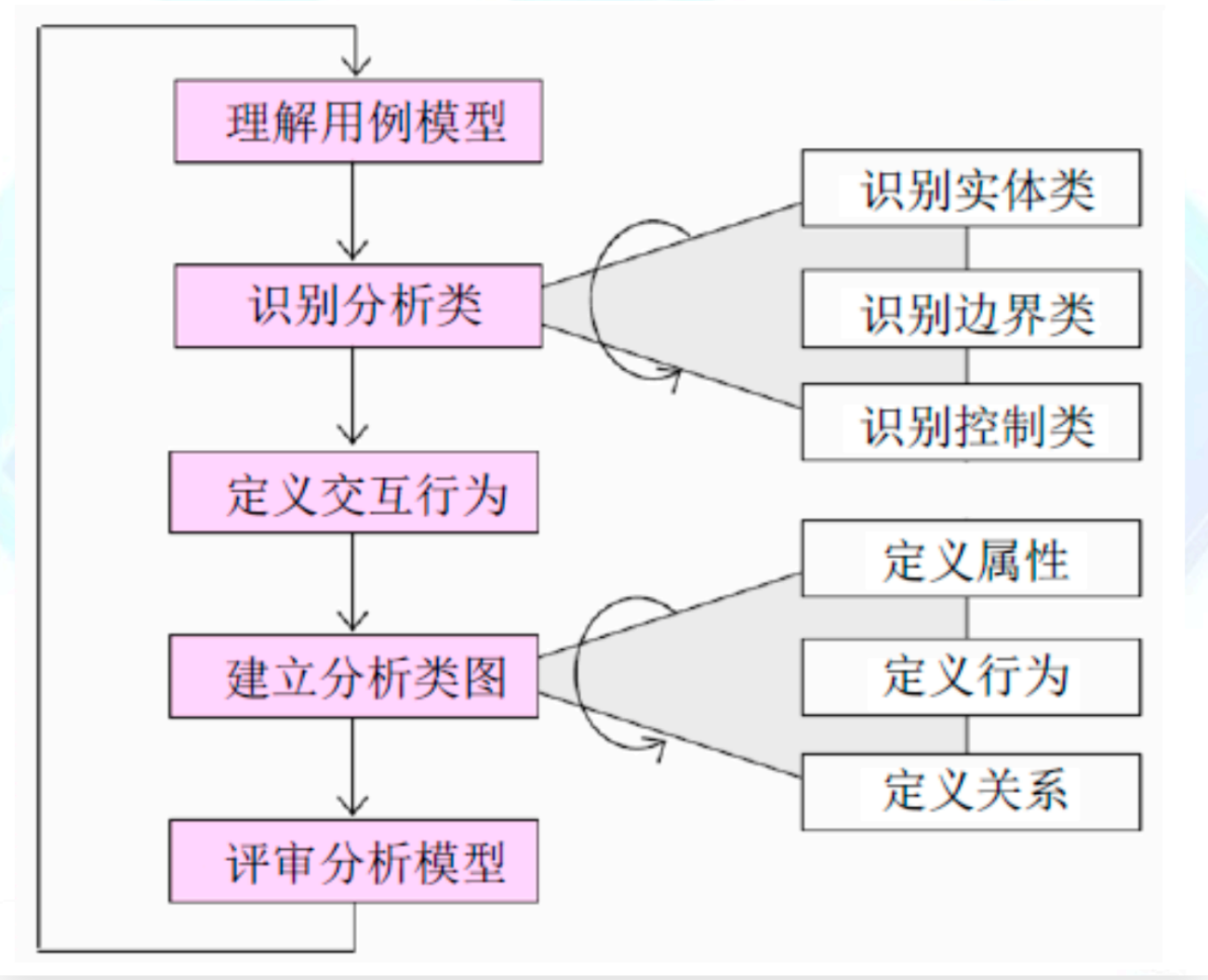
  对象和类是现实世界中的事物的抽象，它们之间的联系也要从分析现实世界事物的各种真实的联系中获得

- 第四阶段：建立系统的静态结构模型

  分析系统的行为，建立系统的静态结构模型，并将其用图形和文字说明表示出来，如绘制类图、对象图、系统与子系统结构图等，编制相应的说明文档

- 第五阶段：建立系统的动态行为模型

  分析系统的行为，建立系统的动态行为模型，并将其用图形和文字 说明表示出来，如绘制活动图、交互图、状态图等，编制相应的说 明文档




## 建立静态结构模型

建立静态结构模型的基本分析过程

- 从用例模型入手，识别分析类
- 描述各个类的属性
- 定义各个类的操作
- 建立类之间的关系

- 绘制类图

## 什么是分析类

分析类是概念层次上的内容，用于描述系统中较高层次的对象

分析类直接与应用逻辑相关，而不关注于技术实现的问题

分析类的类型

实体类：表示系统存储和管理的持久性信息

边界类：表示参与者与系统之间的交互

控制类：表示系统在运行过程中业务控制逻辑

## 边界类

描述外部的参与者与系统之间的交互

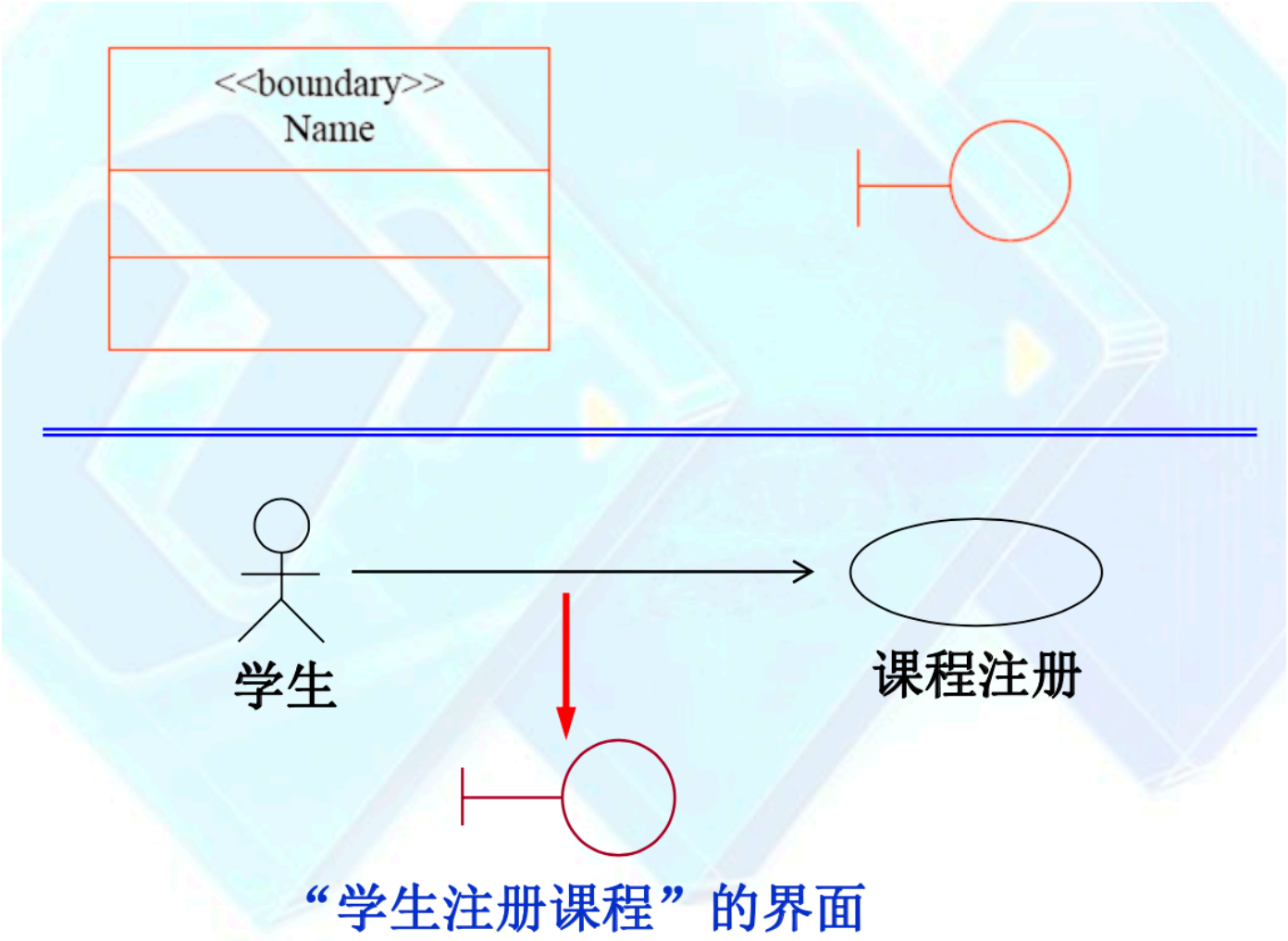
目的：将用例的内部逻辑和外部环境进行隔离，使得外界的变化不会影响到就内部的逻辑部分

类型：用户界面、系统接口、设备接口

对用户界面而言，描述用户与系统的交互信息，而不是用户界面的显示形式

对系统接口/设备接口而言，描述通信协议，但不必说明协议如何实现

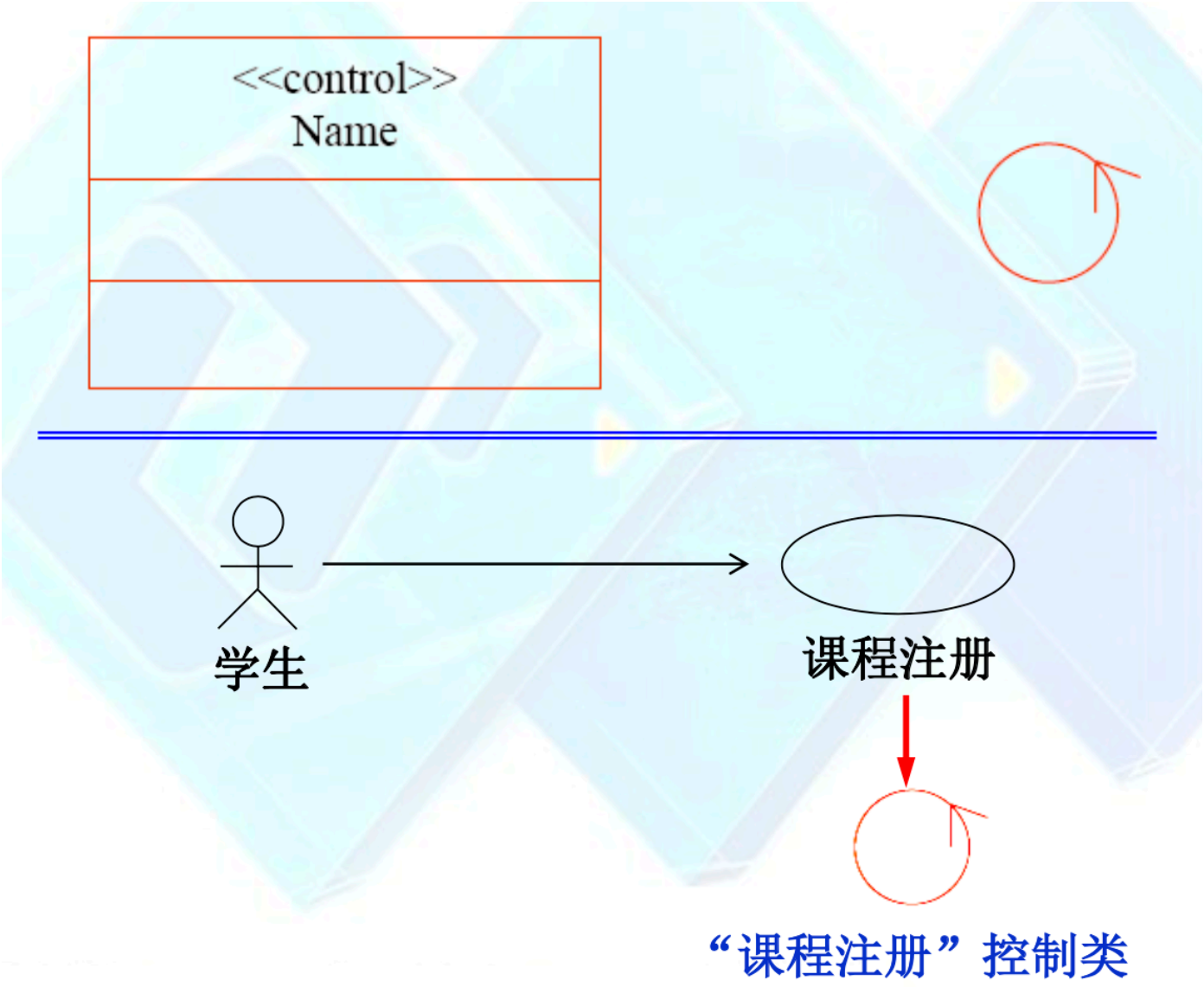
（注重于如何去进行描述，而不注重代码的具体实现）




**控制类**

描述一个用例所具有的事件流的控制行为， 本身并不处理具体的任务，而是调度其他类来完成具体的任务

实现了对用例行为的封装，将用例的执行逻辑与边界和实体进行隔离，使得边界类和实体类具有较好的通用性

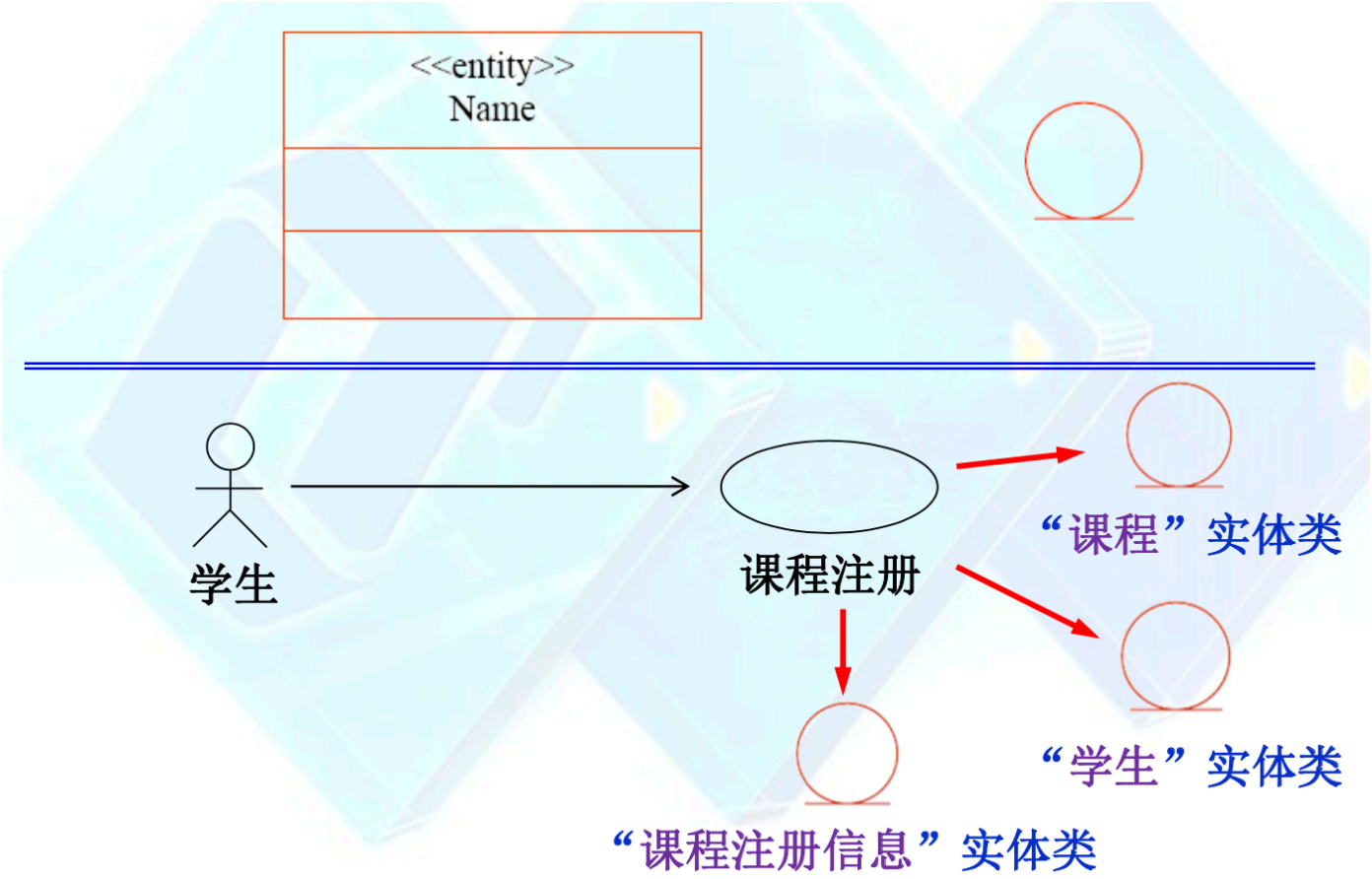



**实体类**

描述必须存贮的信息及其相关行为

对系统的核心信息建模，通常这些信息需要长久的保存

通常对应现实世界中的“事物”



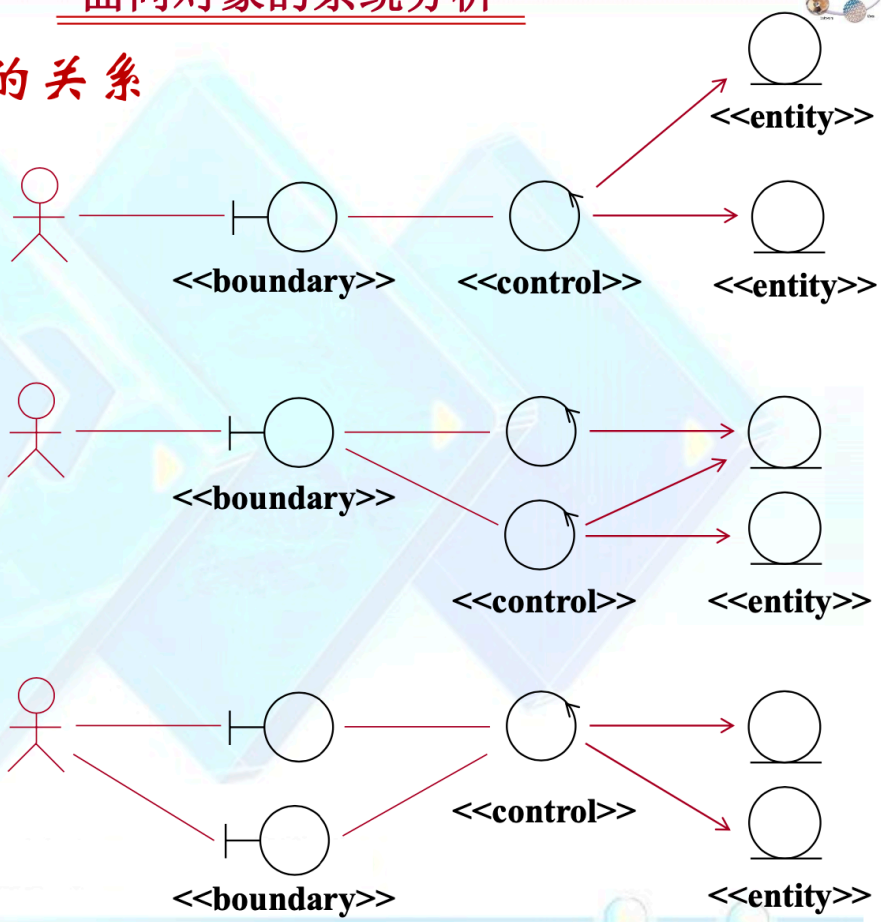

三种分析类之间是 “关联”关系

三种分析类之间是 多对多(m:n)关系

# 面向对象的系统分析

## 三种分析类之间的关系

- 三种分析类之间是“关联”关系(association)
- 三种分析类之间是多对多(m:n)关系：
  - 一个边界类可以与多个控制类相关联
  - 一个控制类可以与多个边界类相关联、与多个实体类关联
  - 一个实体类可以与多个控制类相关联






### Step1：识别分析类

- 重用或修改现有的模型
- 使用分类列表（不同领域不同的分类列表）
- 对用例文本进行语法分析

识别边界类：

通常，一个参与者与一个用例之间的交互或通信关联对应一个边界类（UI界面/接口）

在分析阶段，力求使用用户的术语描界面

边界类实例的生命周期并不仅限于用例的事件流，如果两个用例同时与一个参与者交互，那么它们有可能会共用一个边界类，以便增加边界类的复用性

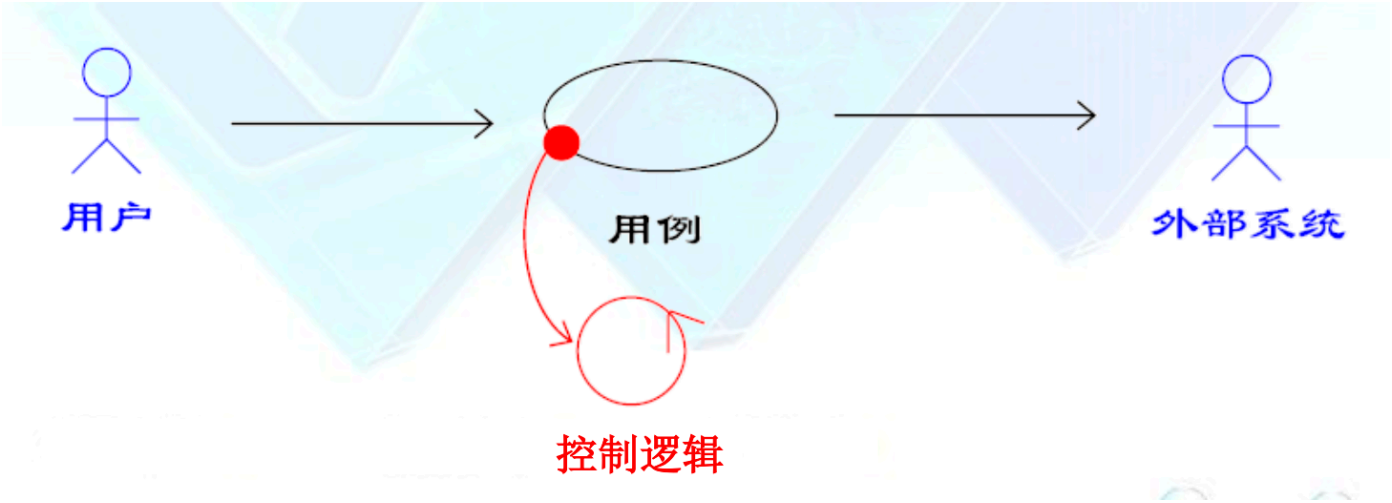
识别控制类：

控制类负责协调边界类和实体类，通常在现实世界中没有对应的食物

负责接收边界类的消息，并将其分发给实体类

控制类与用例之间存在着密切的关系，它在用例开始执行时创建，在用例结束时取消

一般来说，一个用例对应着一个控制类




当用例比较复杂时，特别是产生**分支事件流**的情况下，一个用例可以有**多个控制类**

在有些情况下，用例事件流的逻辑结构十分简单，这时没有**必要使用控制类**，边界类可以实现用例的行为（“登陆”用例）

如果不同用例包含的任务之间存在着比较密切的联系，则这些用例可以使用一个控制类，其目的是**复用相似部分以便降低复杂性**

通常情况下，应该按照一个用例对应一个控制类的方法识别出多个控制类，再分析这些控制类找出它们之间的共同之处

识别实体类：

通过名词驱动的识别方法从用户的需求陈述中找到实体类

实体类通常是用例动作的参与对象，对应着现实世界中的事物

对用户需求陈述进行“语法分析”，找出所有的名词或名词短语， 对其标注下划线

判断一个名次是否为实体类的标准

系统是否需要管理该名词所拥有的信息

系统是否需要管理名次所能发出或接受的动作

合并同义词

将最终得到的每一个名词映射成为一个实体类

动词映射为类的操作，形容词/名词映射为类的属性

识别实体类应当注意的问题：

实体类的识别质量在很大程度上取决于分析人员书写文档的风格和质量

在分析自然语言描述时，应该规范化描述文档中的一些措辞

由于名词可以对应类、属性或同义词，所以开发人员需要花费大量时间进行筛选

角色（Actor不一定是实体类）

除非系统需要在各用例中管理和维护该角色的信息(不是指ID和密码)，否则只需将其作为actor，无需作为实体类

- 选课系统中：教学秘书仅属于actor，无需作为实体类；但学生和老师既是actor也是实体类，因为课程/课表/选课单等实体中均需要使用学生和老师的信息
- 淘宝系统中：淘宝平台管理员/支付宝/物流系统均为actor

对**用户(actor)的权限管理**，是任何系统均具备的通用功能，**无需在实体类中考虑**;将来放在基础设施层实现即可

**step2**：描述分析类的属性

- 对UI类型的边界类：
  - 需要actor输入的各个数据
  - 系统反馈给actor的各个数据
  - 需要临时保存的、用于在边界类和控制类之间传递的临时数据
    例如：登录次数、短信验证码发送时间
- 对API类型的边界类：
  - 需要向外部系统(软件/硬件)传递的数据
  - 需要从外部系统(软件/硬件)接收的数据
- 控制类的属性：
  - 从UI接收的数据
  - 为进行事件流执行所需的临时数据
  - 需要调用的实体类
  - 经过计算之后、需要发送给UI的数据

## Step 2：描述分析类的属性之实体类

- 基本属性：
  - 按照一般常识，找出实体类的基本属性
  - 考虑对象需要系统保存的信息，找出对象的相应属性（例如访问权限、学生学费缴纳情况）
- 状态属性：
  - 识别对象需要区别的状态，考虑是否需要增加一个属性来区别这些状态
- 关联属性：
  - 确定属性表示整体与部分结构和实例连接
- 派生属性：
  - 通过计算其他属性的值所得到的新属性（例如实发工资、扣税）

### Step3：定义分析类的操作

将用例行为分配到相应的分析类之后，系统的一些分析类具有相应的职责，可通过动词分析获得操作

操作的四种类型：

- 以某种方式进行数据的操作（增加、删除、重新格式化、选择）
- 执行计算的操作
- 请求某个对象的状态的操作
- 监听某个对象发生某个控制事件的操作

- 边界类的操作：
  - 输入数据验证
  - 提供供给用户的、可在UI上进行的各类操作（如搜索、结算、设置）
  - 对从控制类返回数据进行各类临时处理而进行的操作（如大写转换）
  - 提供给其他系统的API（如扣款、查询账户余额）
- 控制类的操作：
  - 对从边界类接收到的数据进行各类临时处理而进行的操作（如分词）
  - 向实体类所发出的调用操作（如增、删、改、查）
  - 对从实体类接收到的数据进行临时处理而进行的操作（如排序）
- 实体类的操作：
  - 对属性进行CRUD的操作（Create/Delete/Update/Retrieve）
  - 对状态进行更新的操作
  - 相关的业务操作（如匹配、统计汇总、计算）

用例的事件流描述了一系列的动作（actor做的和系统自身做的）

动作映射到分析类的操作当中的基本原则

- actor自己做的动作，都是边界类的操作，与控制类和实体类无关
- 边界类的操作会调用控制类的操作，并进一步被分解为对各个不同实体类的操作的调用序列
- actor发出的动作通常都比较大，往往涉及到对多个实体类的属性的 增删改查(CRUD)，不能简单的将它直接放置在某个实体类中

实体类的操作可以分为两类

- 对该实体的生命周期的操作，new(), destory()
- 对该实体内部的一个或多个属性的CRUD操作

  注意这里是自己的属性，不是其他实体的属性

对用例中actor所发出的每个动作，需要仔细分析它对哪些实体类的哪些属性做CRUD

外部系统的相关行为是通过接口来进行调用的

判断类的操作是否合理

- 对该实体类的任何一个属性的CRUD，均有特定的操作加以完成

  不是将属性的可见性设定为public，由使用者直接读取或更新操作的值
- 若一个操作CRUD的对象不是该实体类的属性，那么该操作不该出现在这里，把它移到相应的实体类中去

  换言之，一个类只能对自己的属性值进行CRUD操作
- 若发现某个操作op会CRUD多个实体类的属性值，通常意味着这个 操作需要分解为多个小粒度操作(op1,op2,...)，每个opi放在它专门 的实体类中。此时，op本身就不会成为某个实体类的操作，而通常**将它放在某个控制类中，由该控制类分别来调用op1,op2,...**

对当前类的属性的CRUD方法需要定义在当前类下，若需要对多个实体类的属性值进行操作或改变，则可以定义一个控制类，由该控制类进行调用多个实体类的CRUD方法

有一类用例中的操作，没办法分解为小粒度op，尤其查询类的操作

有一类用例中的操作，没办法分解为小粒度op，尤其查询类的操作，所以，这个操作只能由控制类加以完成:。

在领域模型中增加一个实体类“商品清单”，其中包含了 所有的“商品”对象。这样的话，“通过关键字查询”就成了该实体类的一个操作。

- 有一类用例中的操作，没办法分解为小粒度op，尤其查询类的操作
  - 例如：“用户通过关键字查询商品”，返回的是一组商品；该操作绝不会是“商品”类的操作，因为任何一个“商品”对象都不可能包含淘宝上的全体商品
- 所以，这个操作只能由控制类加以完成：
  - 根据用户提供的关键字，直接到数据库中检索出满足条件的商品清单
  - 基于数据库中的每条商品记录，使用“商品”类的new操作构建一个“商品”对象，形成一个List，返回给相应的边界类，边界类对其加以展示
- 另一个办法：在领域模型中增加一个实体类“商品清单”，其中包含了所有的“商品”对象。这样的话，“通过关键字查询”就成了该实体类的一个操作
  - 问题在于：需要在系统初始化时就将该实体类构造出来，否则无法被使用；这明显是不可能的，占用内存空间太大

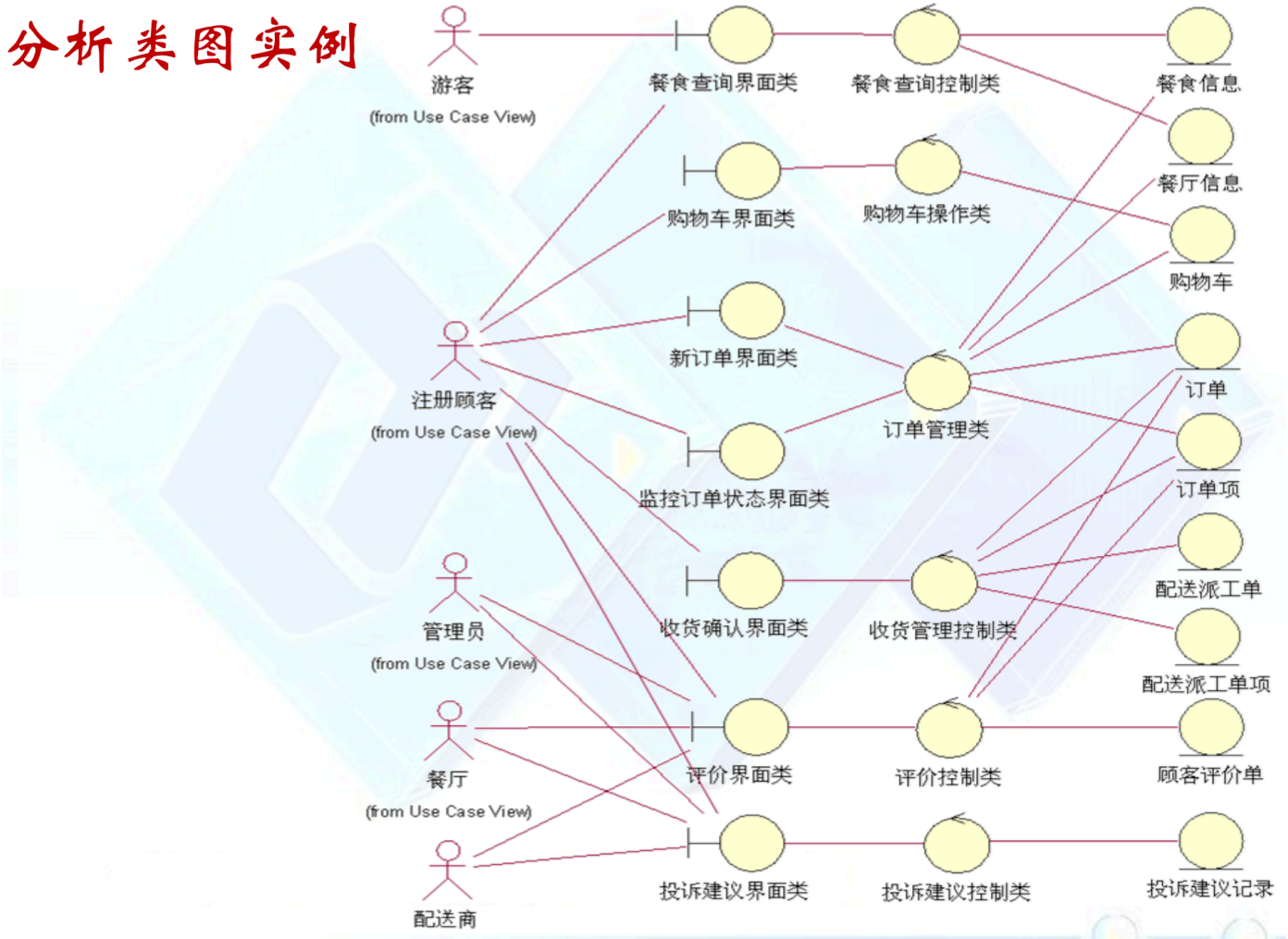
**Step4：建立类之间的关系**

类之间的五种关系：泛化、关联、组合、聚合、依赖

- 例如：
  - “读者”类与“预定”类之间有1:n的关联关系
  - “读者”类与“借书记录”类之间有1:n的关联关系
  - “图书资料”类与“借书记录”类之间有1:n的关联关系
  - “书目”类与“书籍”类、“杂志”类之间是泛化关系

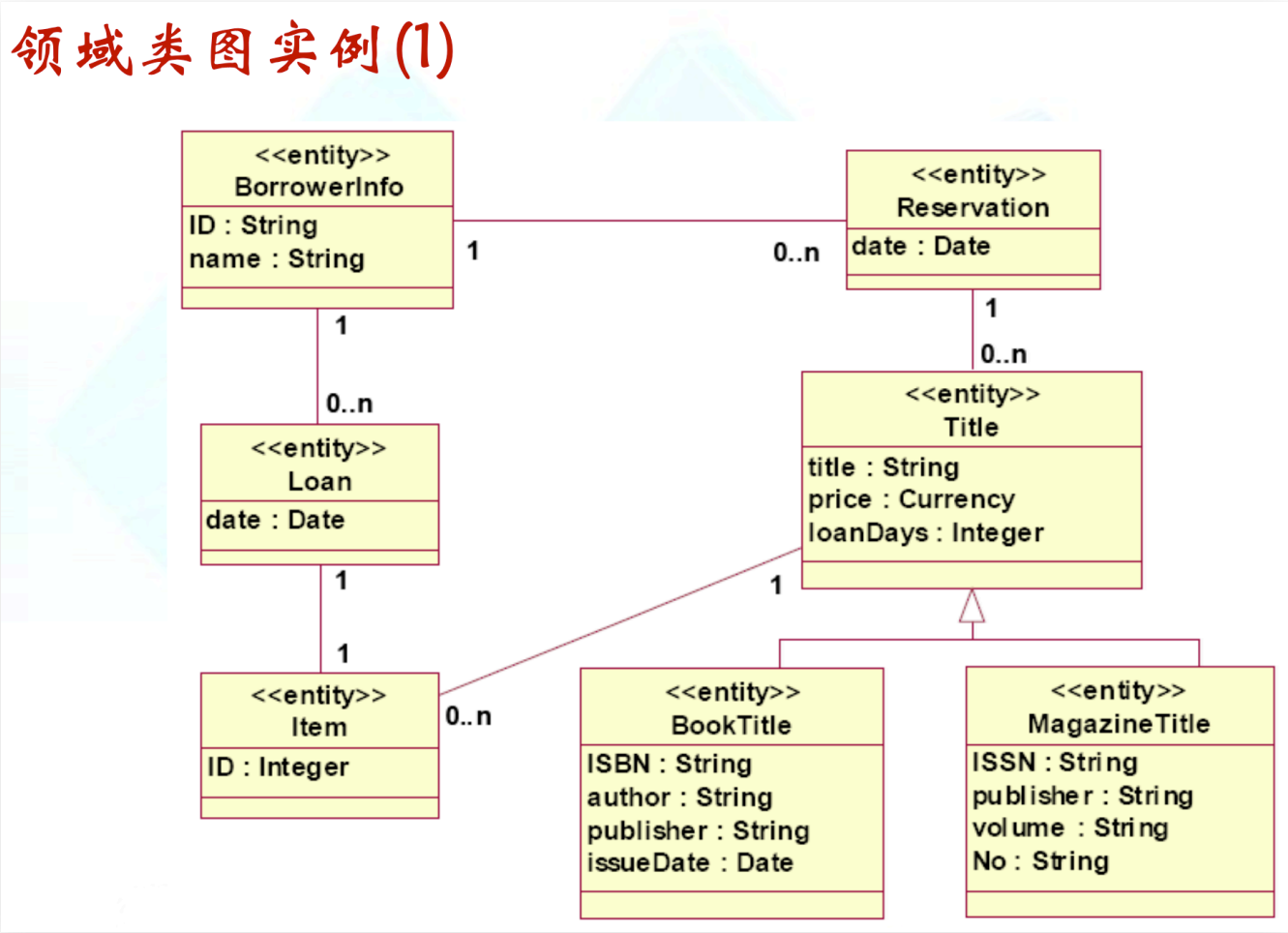
## Step5：绘制类图

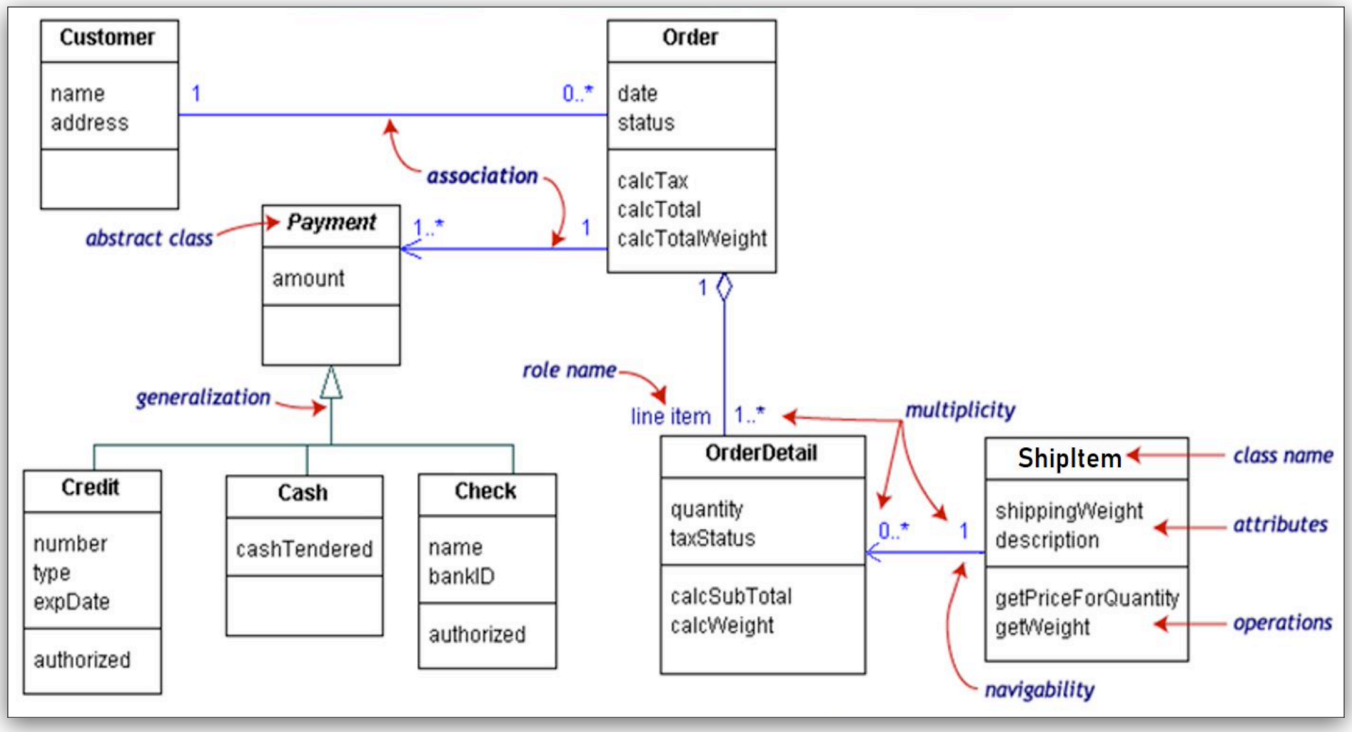
- 分析类图：描述各边界类、实体类、控制类之间的关联关系，无需刻画属性和操作集
- 领域类图：可以不包含边界类与控制类，侧重描述各实体类之间的五种关系，需要给出详细的属性与操作集合在不加说明的情况下，领域类图更有价值




BCE分析类图=Actor+边界类+控制类+实体类

# 领域类图实例(1)



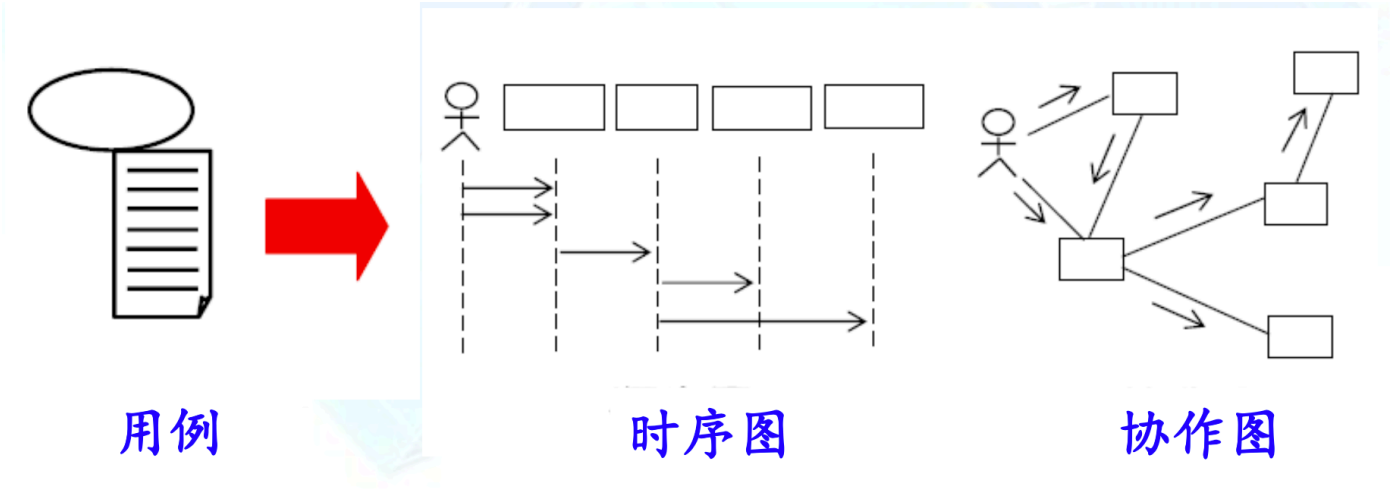




领域类图要标注好相关的关系重数

# 建立动态行为模型

动态行为模型可用两个新的视图进行描述

- 时序图/顺序图
- 协作图



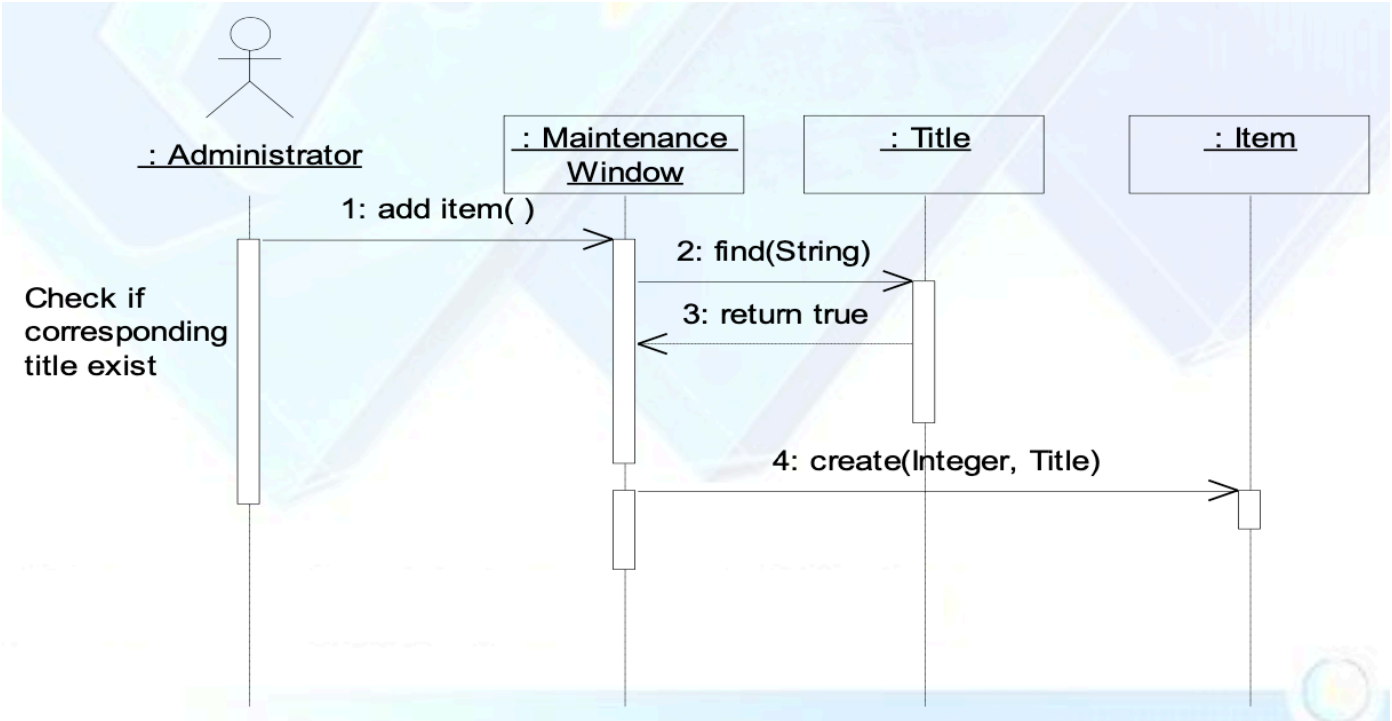

## 时序图

时序图：将用户与分析类结合在一起，实现将用例的行为分配到所识别的分析类当中

时序图是强调消息时间顺序的交互图

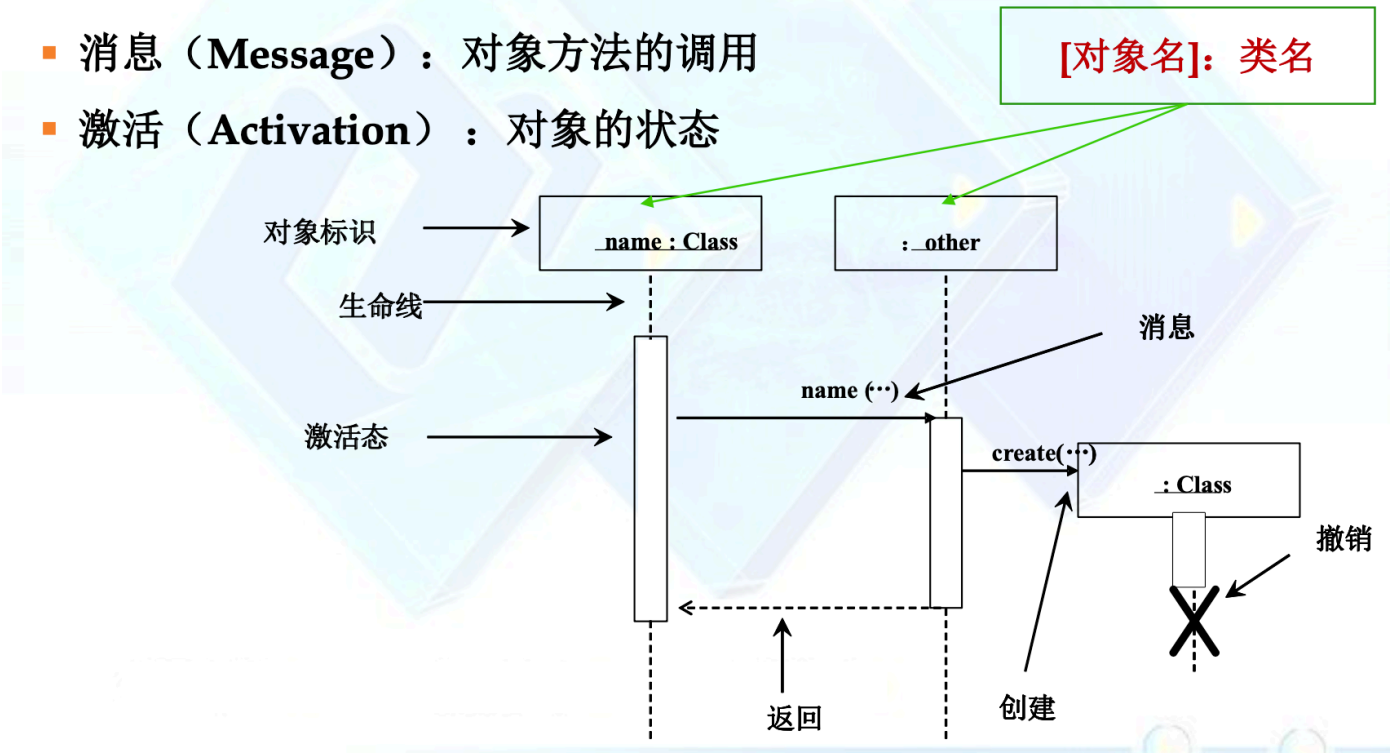
时序图描述了对象之间传递消息的时间顺序，表示用例中的行为顺序

时序图将交互关系表示为一个二维图:纵轴是时间轴，时间沿竖线向 下延伸;横轴代表了在协作中各独立的对象



# 时序图的组成

- 对象（Object）：类的实例
- 生命线（Lifeline）：生命线是一个时间线
- 消息（Message）：对象方法的调用
- 激活（Activation）：对象的状态




消息

消息定义的是对象之间的某种形式的通信，它可以激发某个操作、唤起信号或导致目标对象的创建或撤销

消息是两个对象之间的单路通信，从发送方到接收方的控制信息流

消息可以用于在对象间传递参数

消息可以是信号，也可以是调用

在UML中，消息使用箭头来表示，箭头的类型表示了消息的类型

## 时序模型

时序模型的每一列在本质上都是object，每一个出现的object，其相应的class必须都在分析类模型/领域类模型当中定义过

一张时序图从左到右应该是:actor(用户)、边界类、控制类、实体类、actor(外部系统)

在UML当中，对出现在时序图当中的箭头划分了6种不同的类型

Create/Call/Return/Self-call/Send/Destory

任何类型的箭头(create、call、return、self-call、send、destroy)， 均代表着该箭头需要触发obj2的某个操作

这就意味着，箭头上标识的操作，必须在obj2的操作集合中有所定义

如果在分析类和领域建模阶段没有想全，建立时序模型时可以通过一致性检查发现缺失

## 激活

激活表示该对象被占用以完成某个任务

去激活指的是对象处于空闲状态，在等待消息

（被激活就说明在干活和打工）

在UML中，为了表示对象是激活的，可以将该对象的生命线拓宽成为矩形。

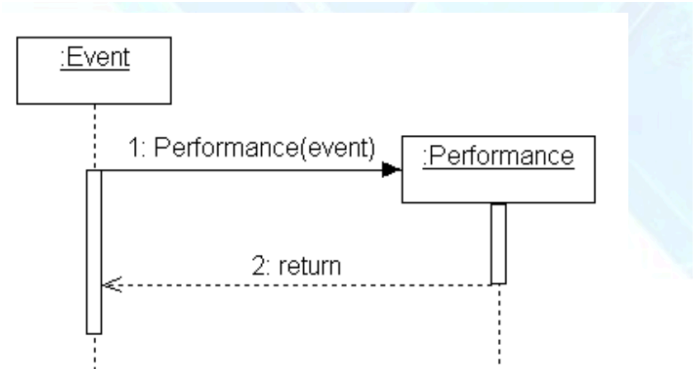
其中的矩形称为激活条或 控制期，对象就是在激活条的顶部被激活的，对象在完成自己的工作后被去激活。

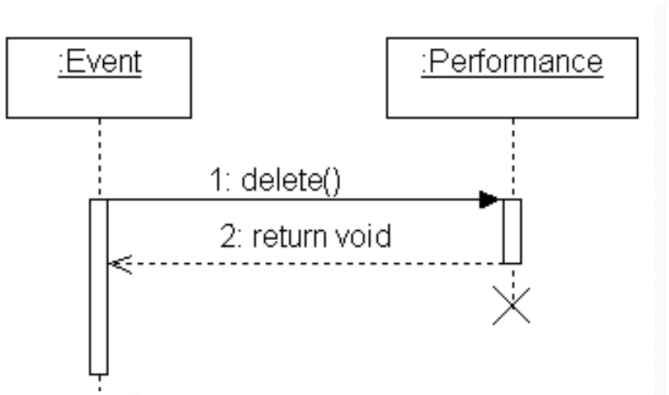
## 对象的创建与撤销

如果对象位于时序图的顶部，说明在交互开始之前该对象已经存在 了;如果对象是在交互的过程中创建的，那么它应当位于图的中间部分。

对象在创建消息发生之后才能存在，对象的生命线也是在创建消息之后才存在的

如果要撤销一个对象，只要在其生命线终止点放置一个“X”符号即可，该点通常是对删除或取消消息的回应




# 时序模型建模过程

- 时序图/顺序图(Sequence Diagram)：将用户与分析类结合在一起，实现将用例的行为分配到所识别的分析类中
- 绘制步骤：
  - 列出启动该用例的参与者
  - 列出启动用例时参与者使用的边界对象
  - 列出管理该用例的控制对象
  - 根据用例描述的流程，按时间顺序列出分析类之间进行消息访问的序列
- 说明：
  - 每个用例对应一张时序图
  - 时序图描述的消息序列需要与用例的事件流保持一致
  - 时序图中出现的每个箭头代表一个操作，需要包含在箭头所指向的分析类中

（时序图和协作图有去，未必需要有回）

### 关于时序模型与用例模型的关系

用例事件流中

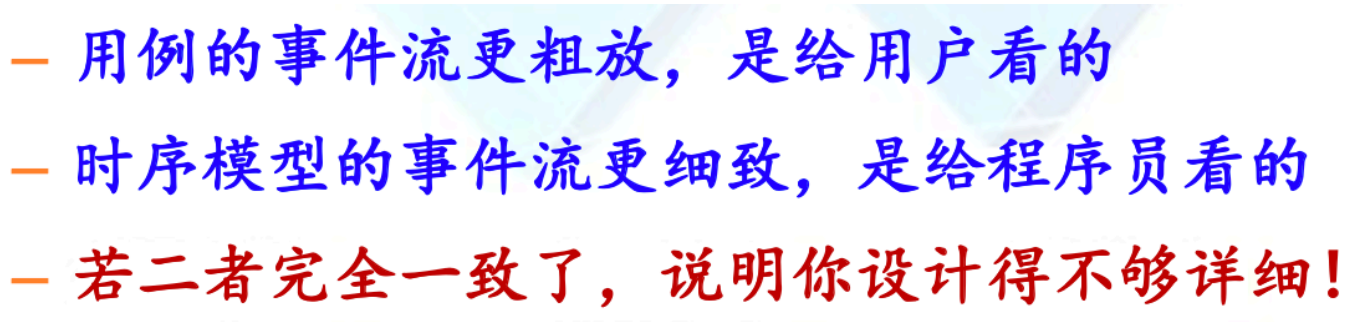
- 用户请求xxx
- 系统响应xxx

在时序图中

- 前者对应于actor向边界类发出的指令
- 后者对应于边界类向控制类、实体类发出的指令与执行的反馈结果（系统响应与控制类发出类似）

用例事件流描述用自然语言，但时序图将其转化为类之间的函数调用与消息传递，是对事件流的“可实现化的描述”

从自然语言描述转变为函数调用和信息传递，对事件流进行了 **可实现化的描述**




### 案例分析

案例:学生选课系统 （见ppt 注意思考方式）

在类间关系当中，注意区分依赖和关联两种不同的类间关系。

依赖是虚线箭头，关联是实线箭头，联系是实现连接。

依赖是参数，关联是属性，聚合是一种特殊的关联，表现为一种整体与聚合的关系

## 05-结构化设计

结构化的系统设计主要考虑：

- 模块的层次化
- 模块之间的接口
- 数据库设计
- 用户界面设计

采用自顶向下的方法进行设计

- 具有系统自动化边界的DFD
- 结构图

## 自动化系统边界划分

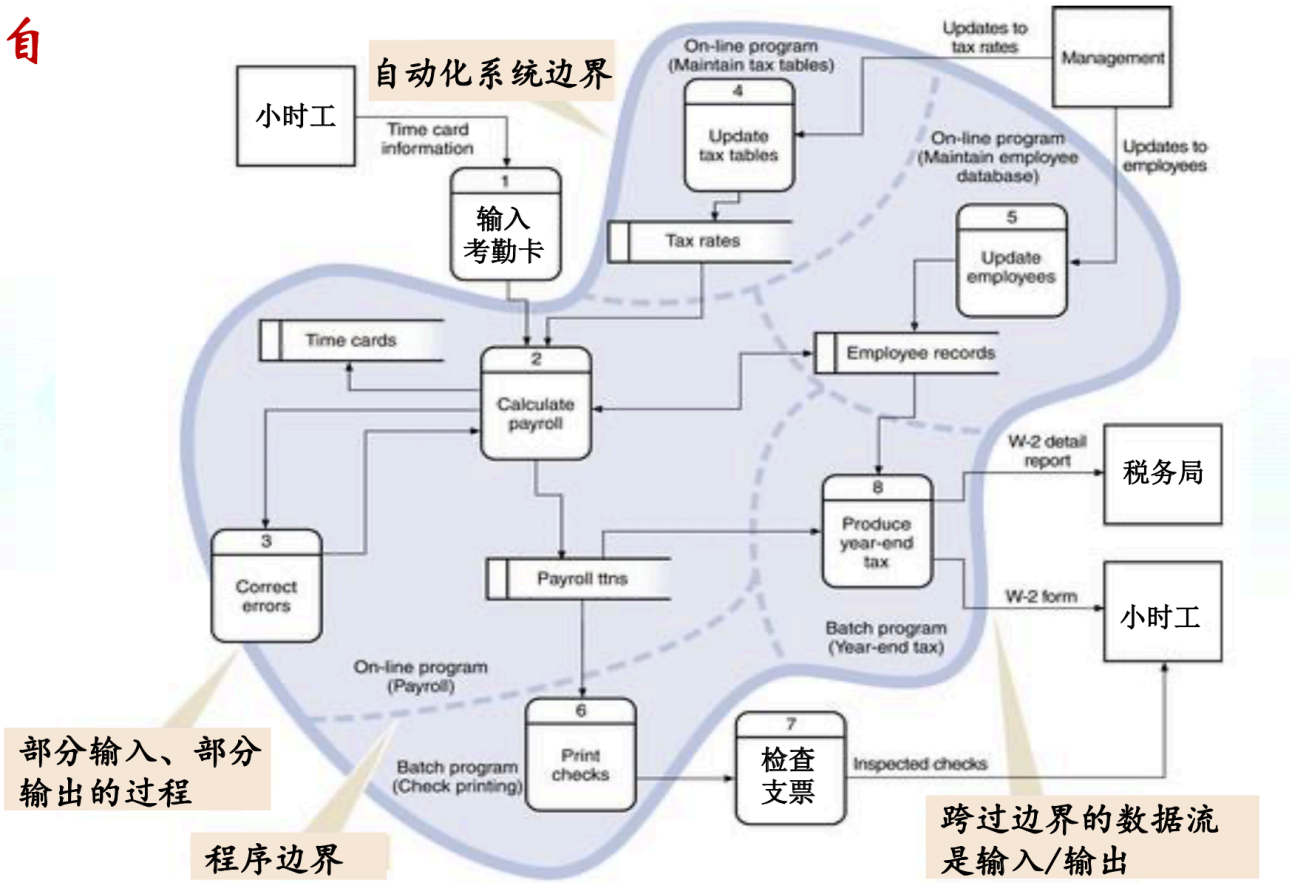
将数据流图划分为手工处理部分和系统能自动完成部分

程序的处理过程可以在系统边界的内部或外部

数据流可以在系统边界内部或外部

穿过系统界线的数据流代表了系统的输入和输出

在最终的系统当中，数据流将乘务用户界面中的表单、报表、供其他系统使用的数据文件等






带自动化系统边界的DFD



## 结构图

以模块为基础，以模块间的调用为关联所构成的图称模块结构图，简称结构图

突出体现的是模块间的相互关系

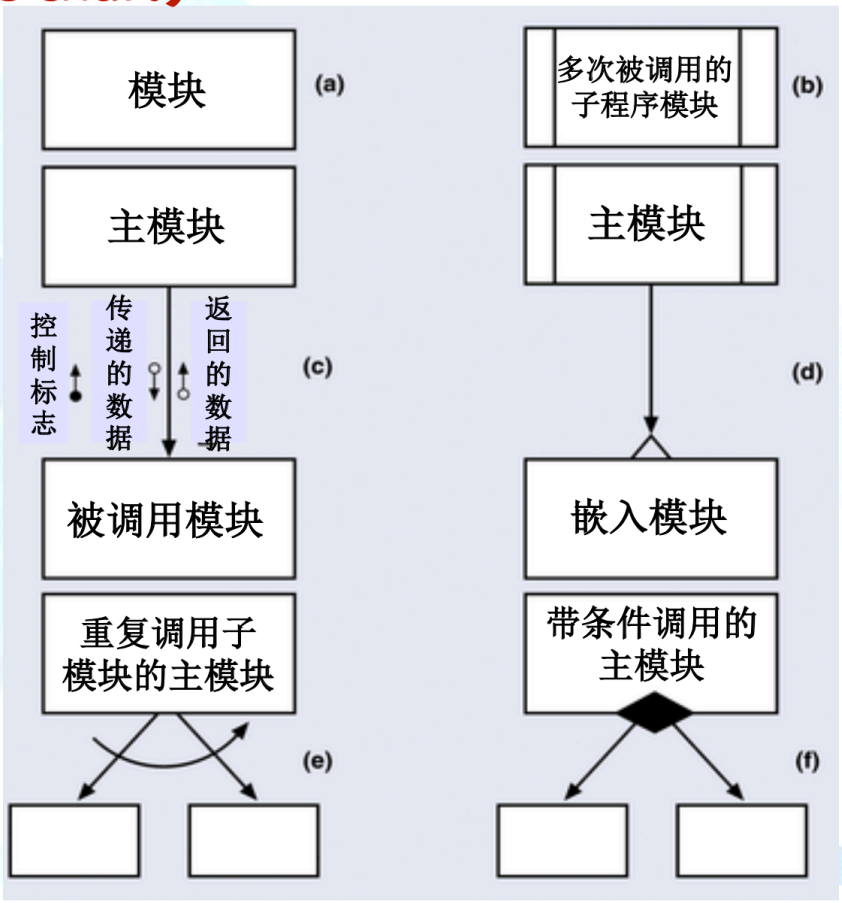
结构图的作用：

- 通过层次的方法来描述系统每部分的功能和子功能
- 展示计算机程序模块间的联系

### 结构图符号

# 结构图（structure chart）

结构图符号




2022/3/29

结构图的创建方法

# 结构图的创建方法

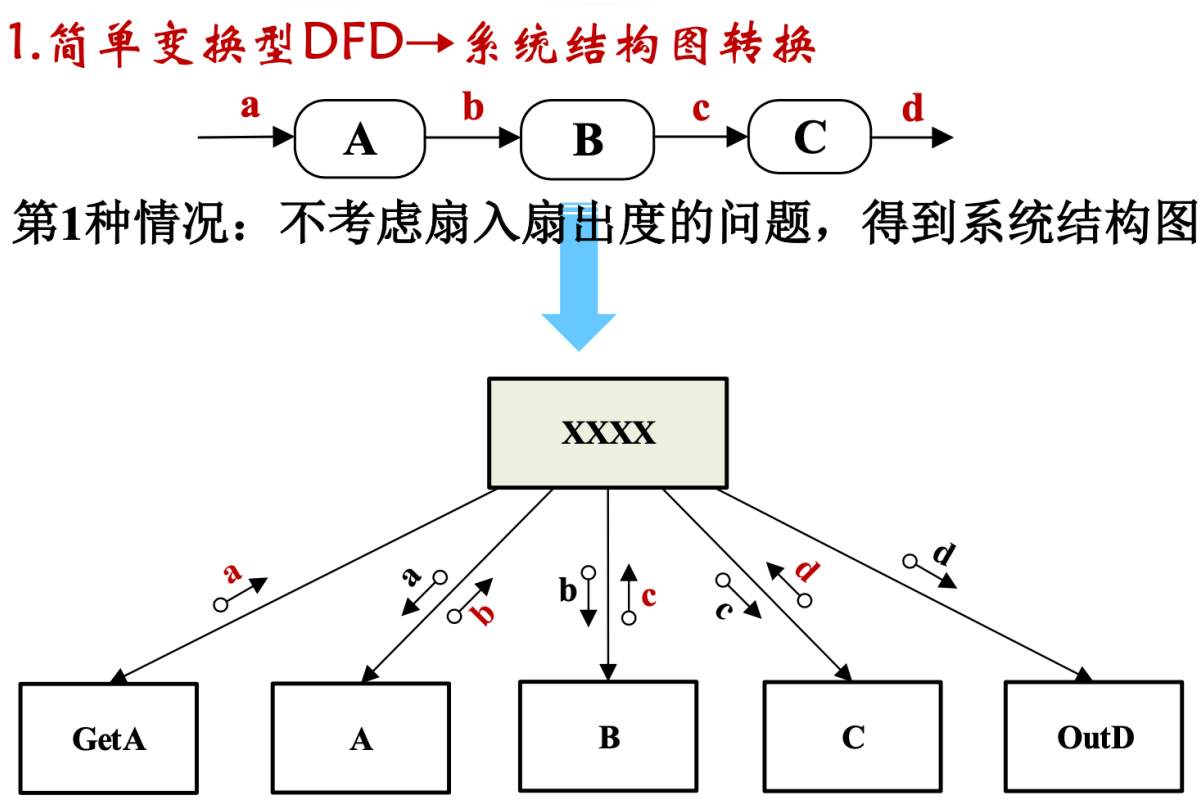
## 从一个DFD片断建立一个结构图的步骤

- 确定主要的信息流
- 找出能代表输入和输出间最基本变化的过程
- 重画数据流图并把输入放在左边，输出放在右边
- 初步建立一个结构图草图
- 加入其它模块实现下列功能
  - 数据输入、数据处理、数据输出
- 使用结构化语言或决策树添加模块间逻辑
- 进一步求精

- 转换分析方法
  - “处理”对输入到输出的转换（I-P-O）
  - 结构图包括输入子树、计算子树和输出子树
  - 用数据流图片断作为输入
- 事务分析方法
  - 含有“处理”分支的情况
  - 某“处理”对输入数据流进行分析，根据分析结果选择不同的“处理”

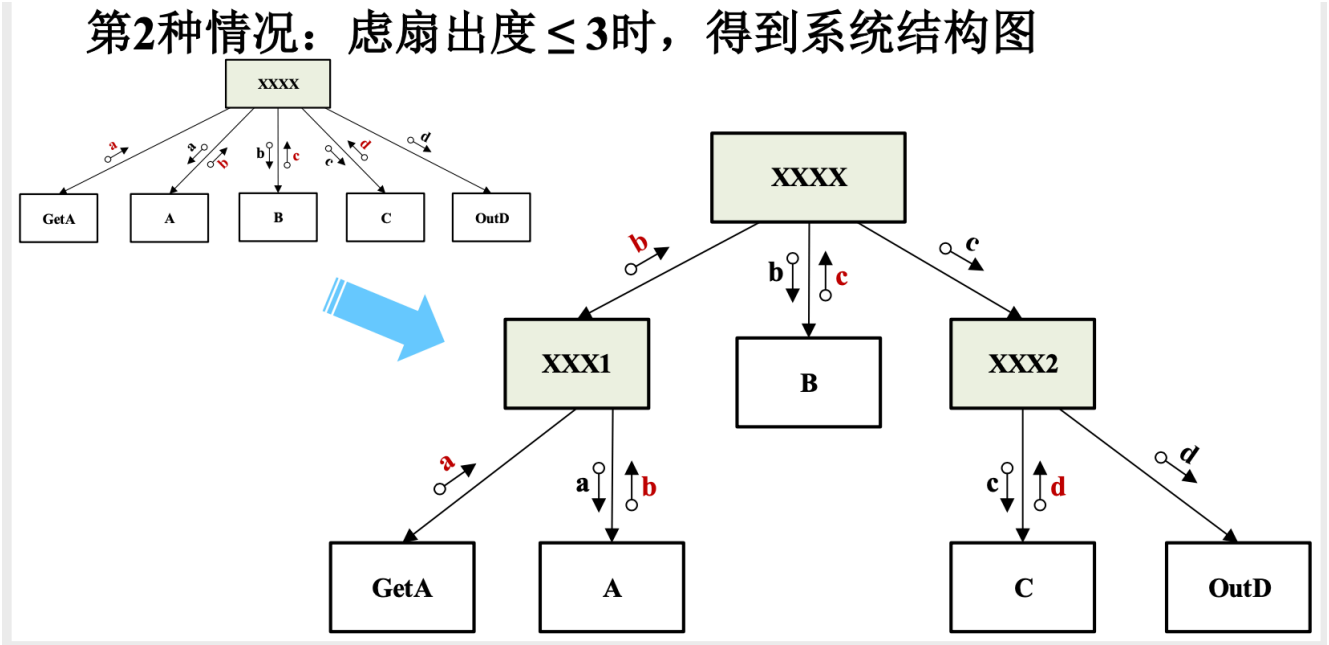
## DFD到系统结构图转换的基本模式

1. 简单变换型DFD -> 系统结构图转换
    - 不考虑扇入扇出度的问题，得到系统结构图




所有的数据流都从一个地方接收和发出

- 虑扇出度 ≤ 3时，得到系统结构图

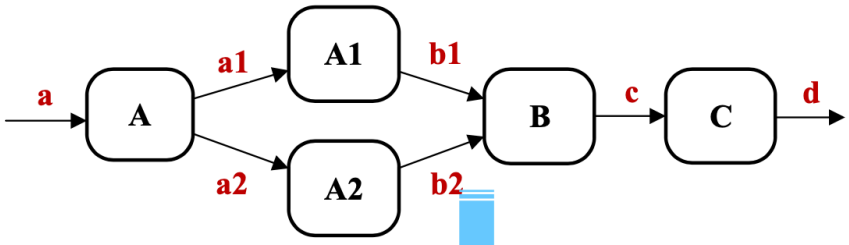


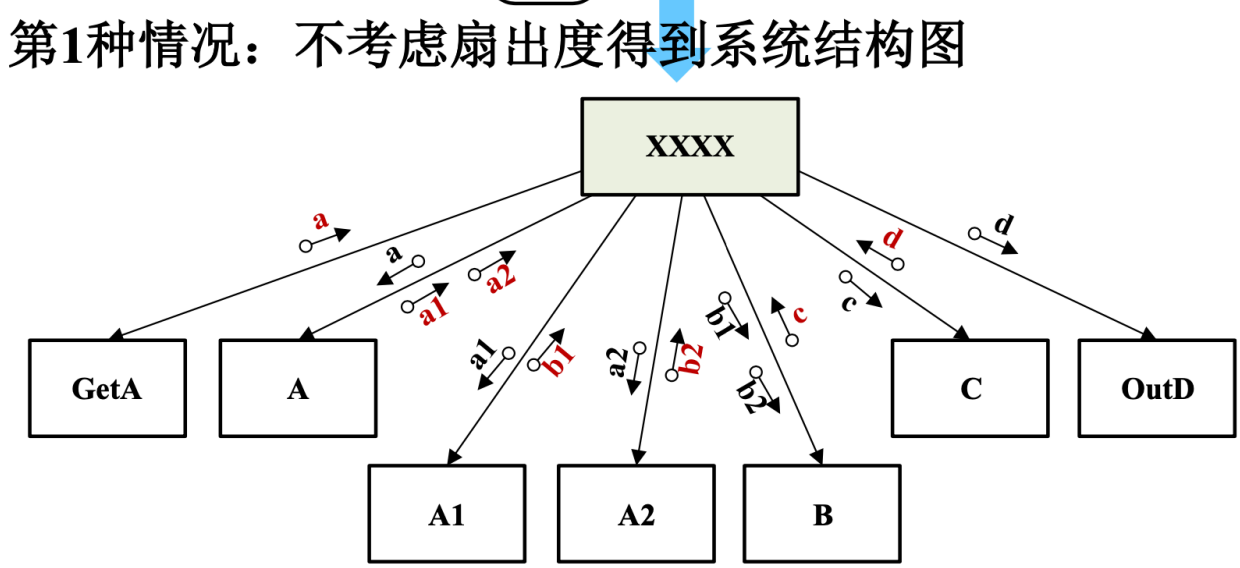

ps：数据流动的目的是从input得到output，中间的数据传输流只是过程

考虑到相关扇出度的要求，将部分的数据传输合并到子模块当中

2. 复杂变换型DFD -> 系统结构图转换
    - 不考虑扇出度得到系统结构图

## 2.复杂变换型DFD→系统结构图转换

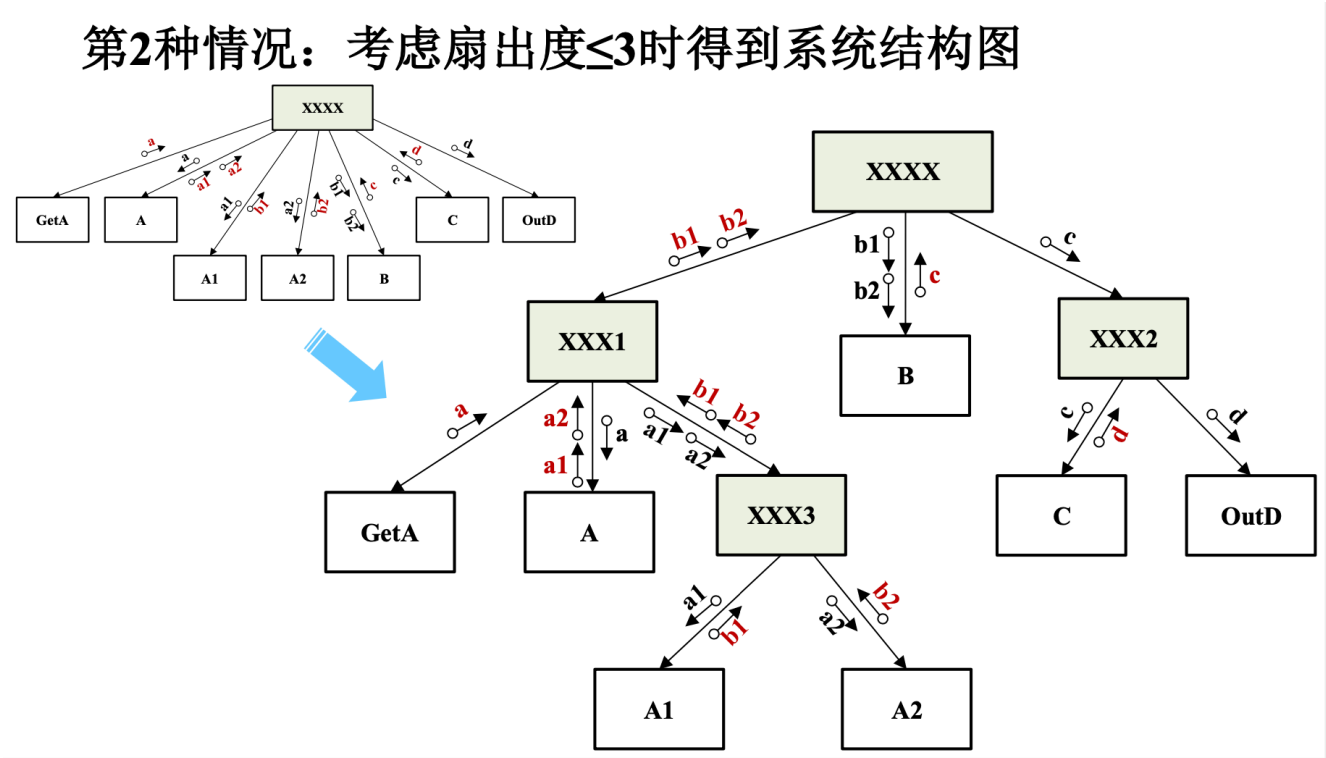



数据流的传输要通过中间媒介来进行

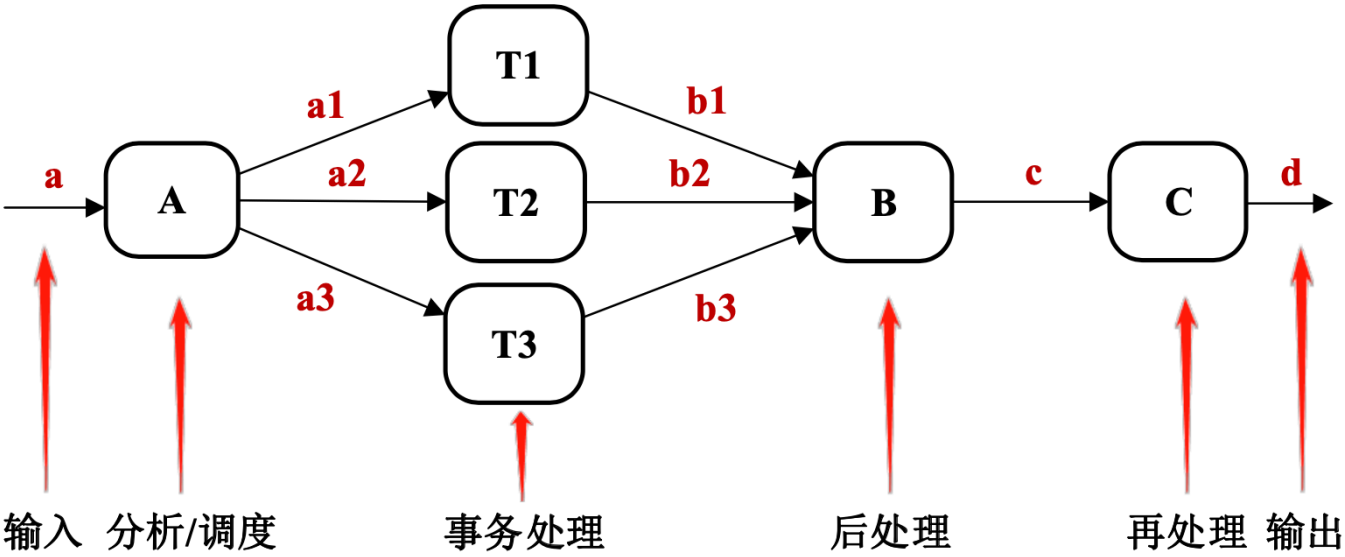
可以引入一个叫负责变换的模块

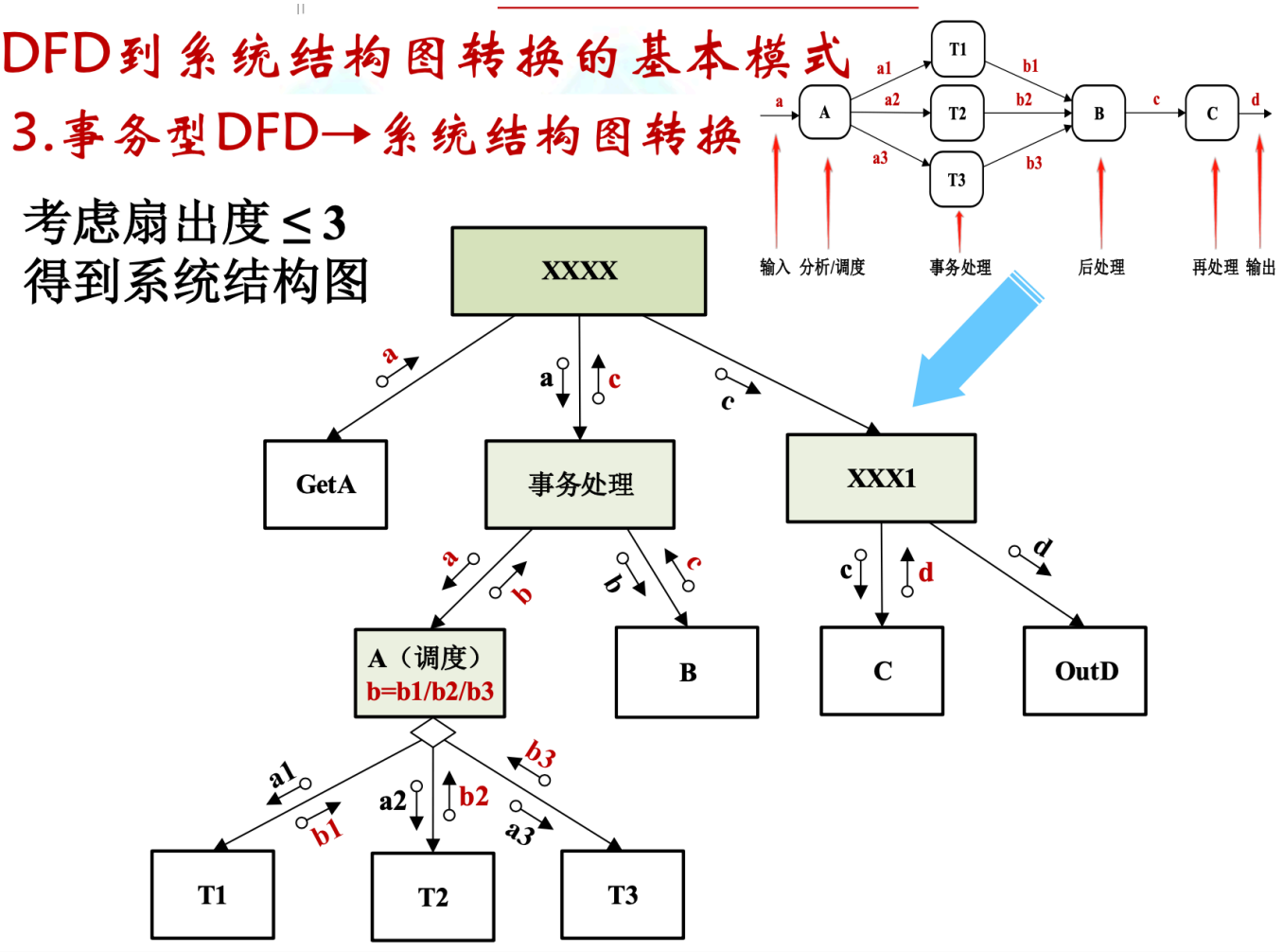
- 考虑扇出度≤3时得到系统结构图




子模块的划分与使用

3. 事务型DFD -> 系统结构图转换

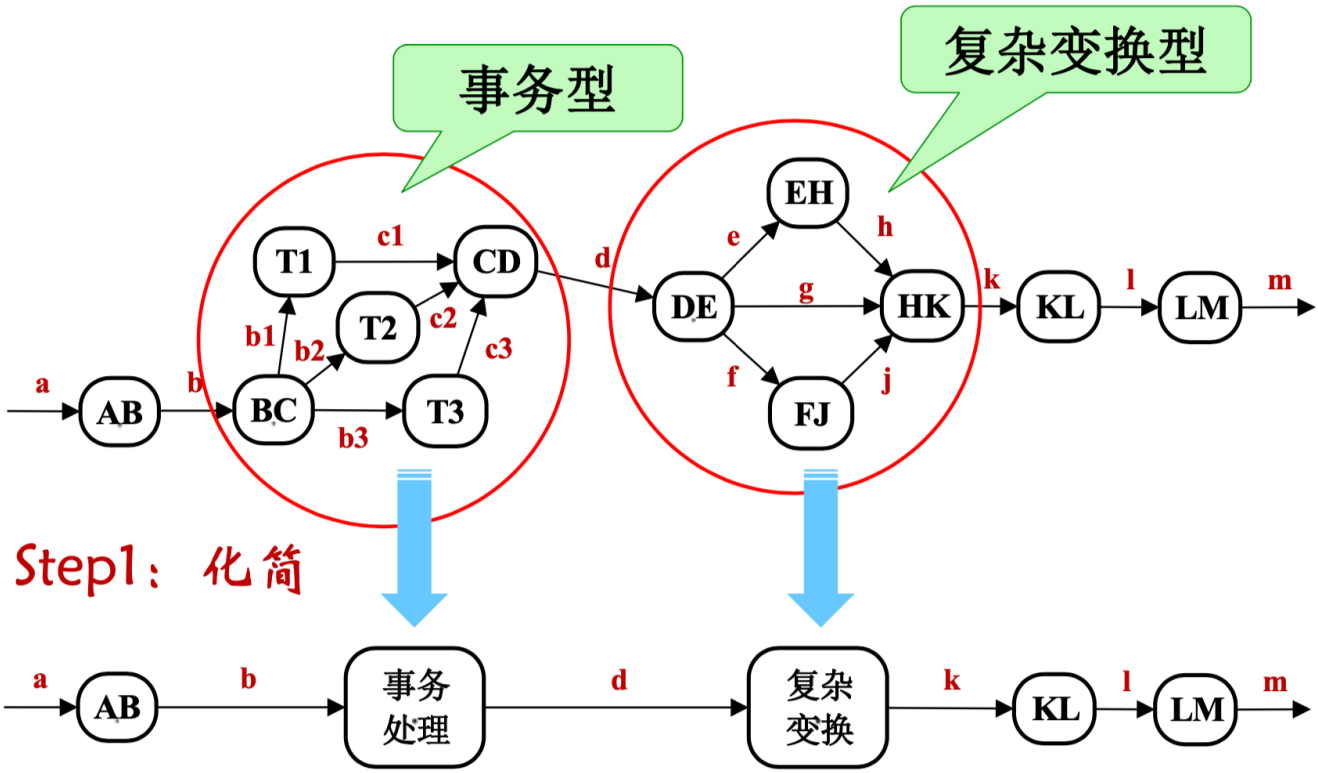



在事务处理当中引入调度模块

4. 事务型与变换型DFD -> 系统结构图转换

## 4.事务型与变换型混合复杂DFD→结构图转换

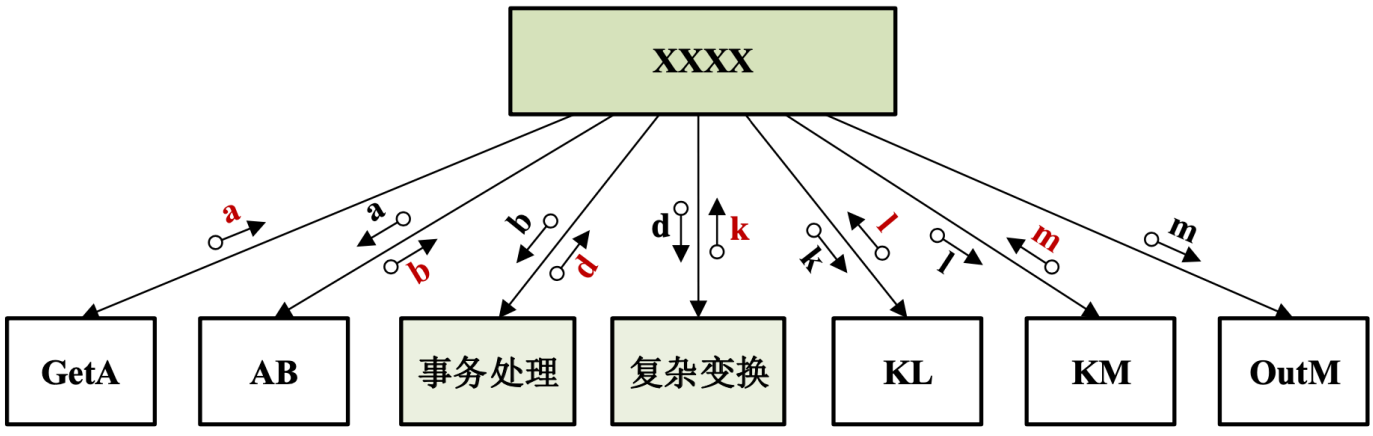



Step1：化简




## 4.事务型与变换型混合复杂DFD→结构图转换

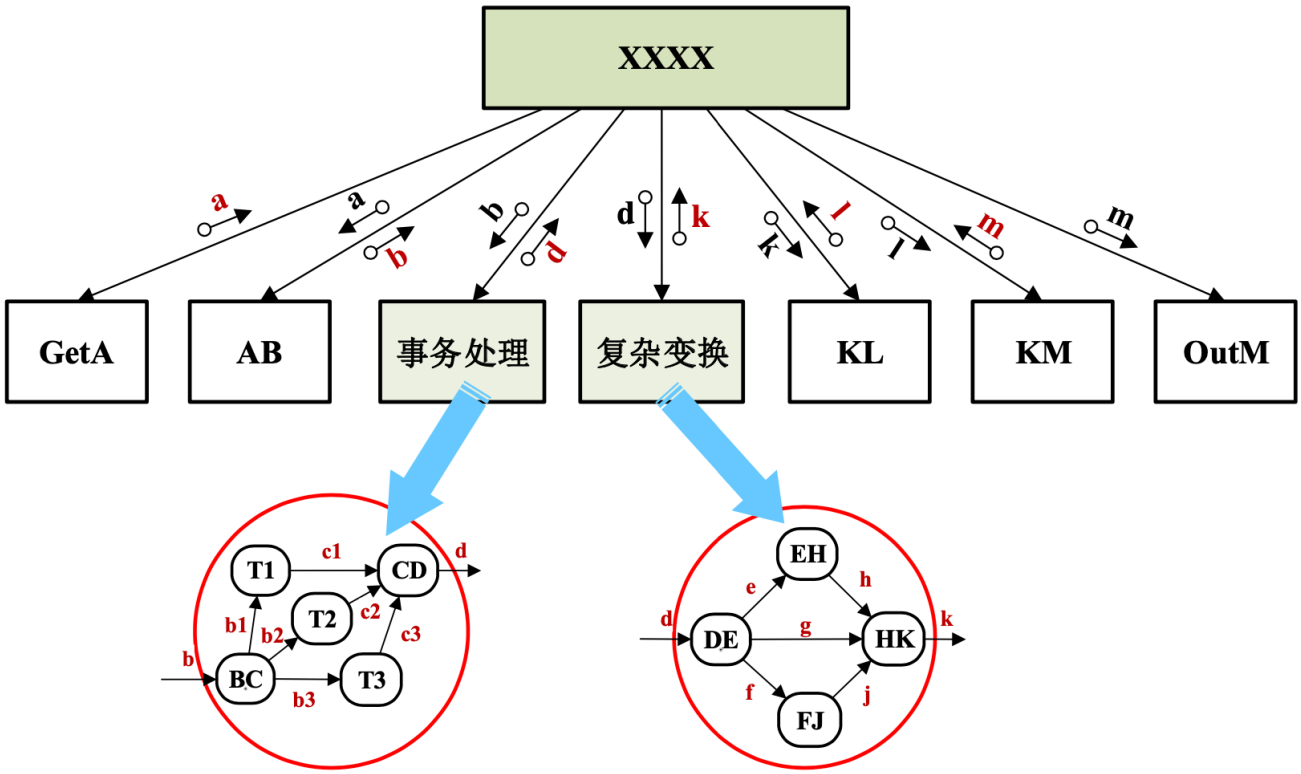
Step2：将化简的DFD→结构图转换



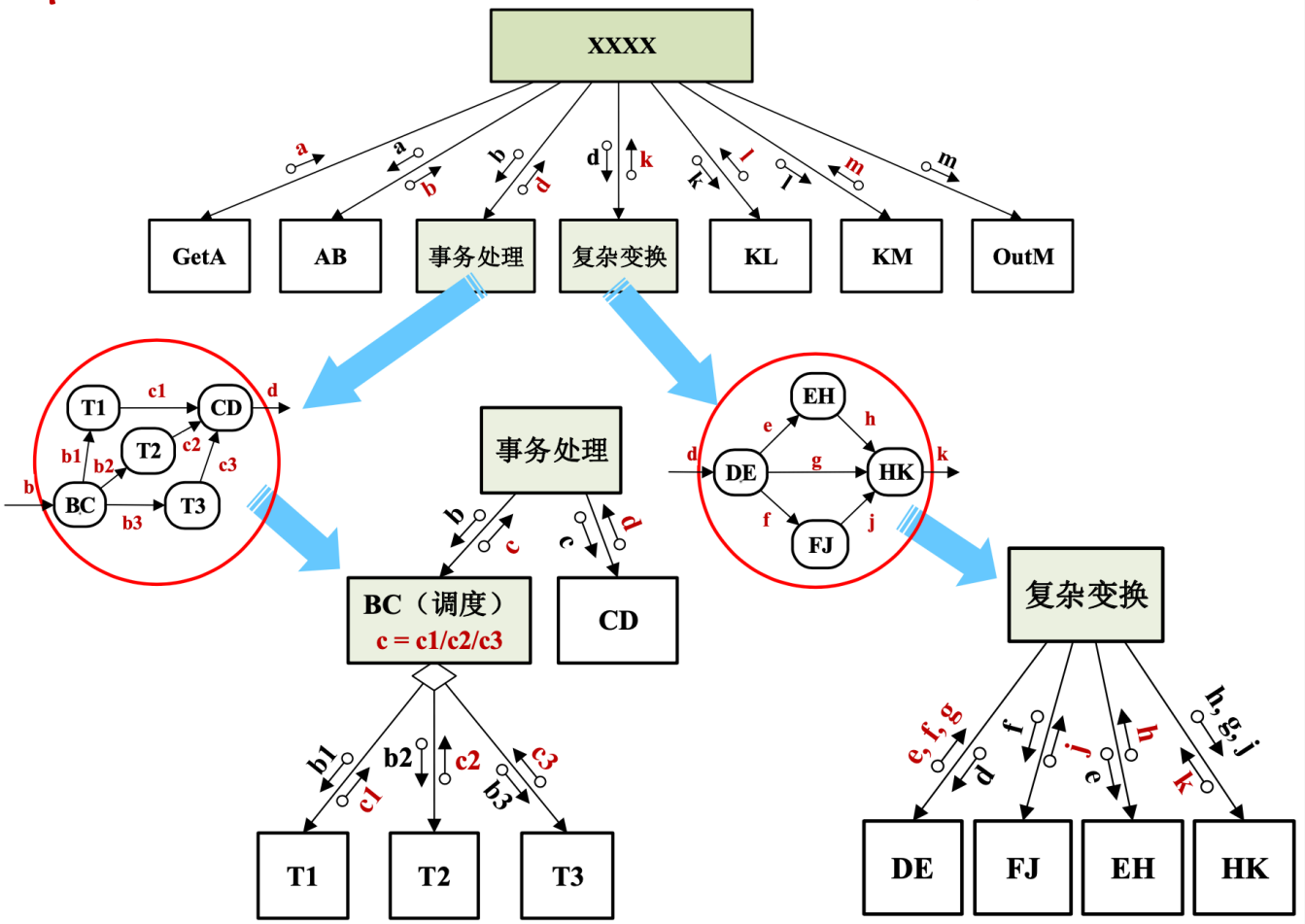

引入事务处理和复杂变换两个层次模块

# 4.事务型与变换型混合复杂DFD→结构图转换

## Step3：拓展事务处理和复杂变换模块的结构图

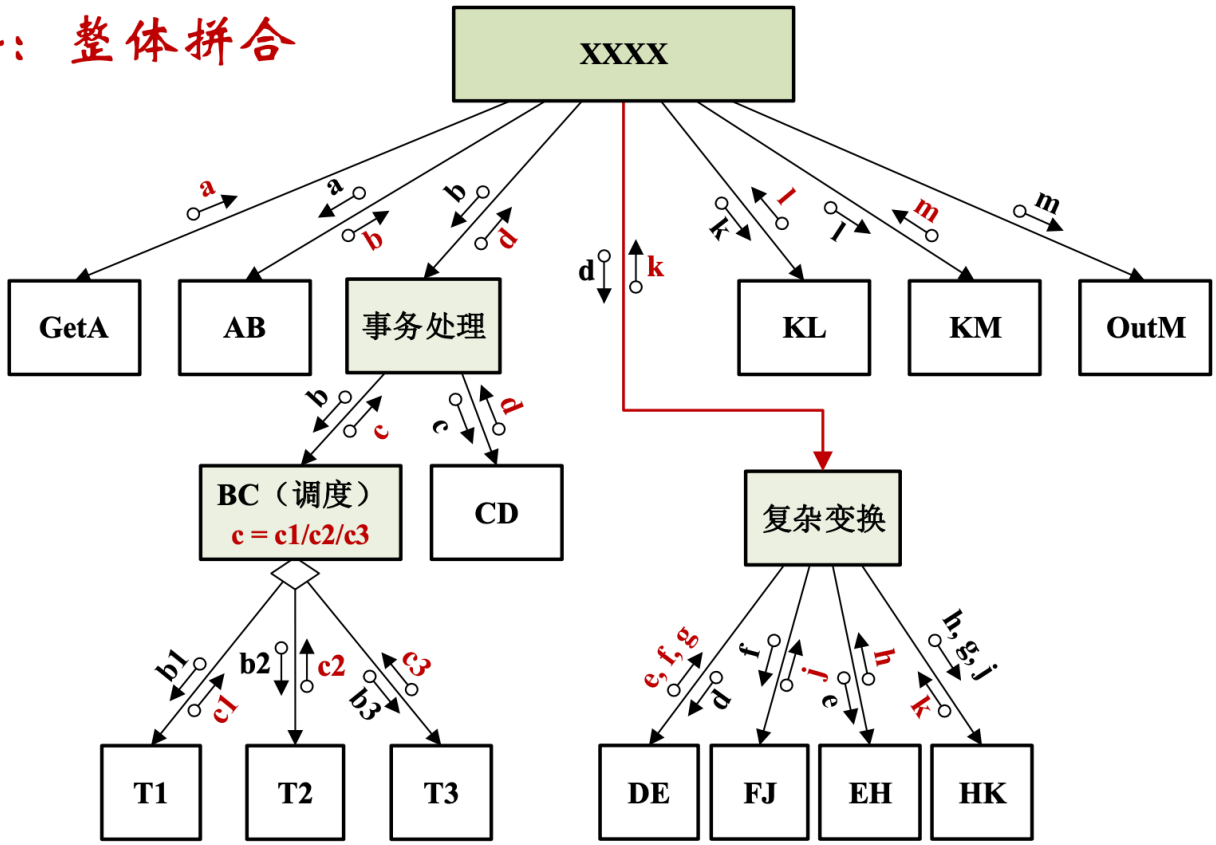


## Step3：拓展事务处理和复杂变换模块的结构图



# 4.事务型与变换型混合复杂DFD→结构图转换

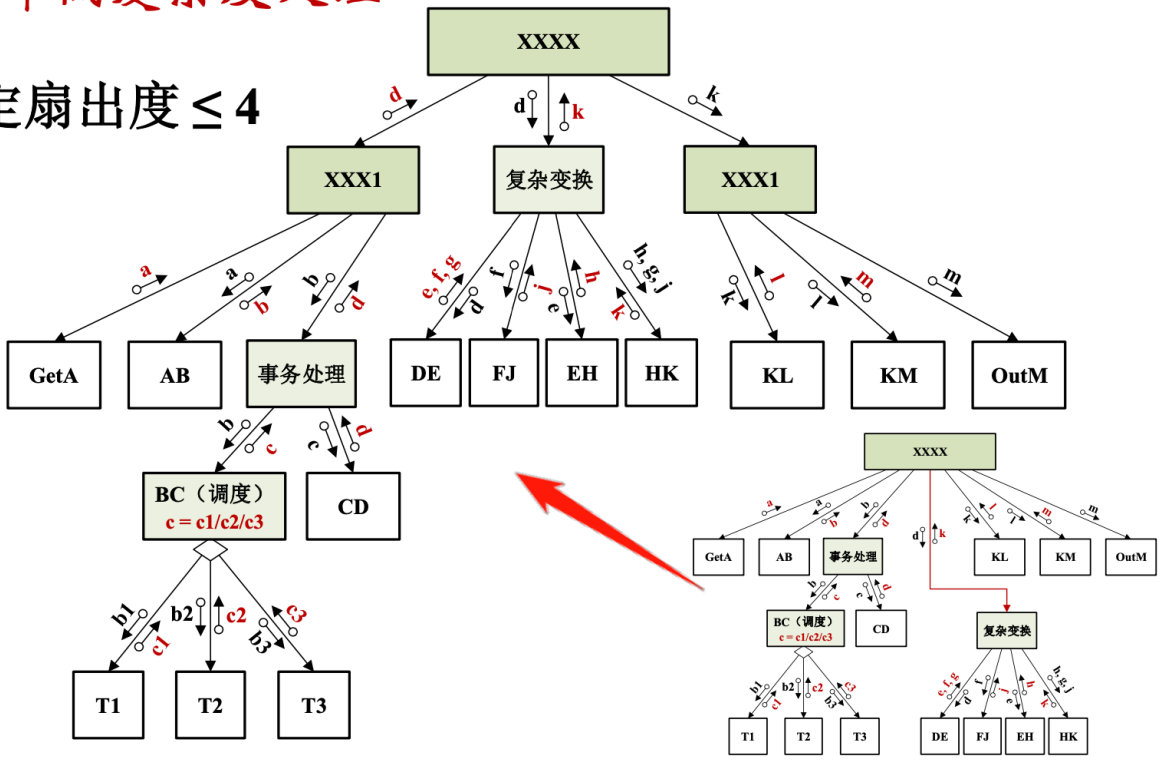
## Step4：整体拼合

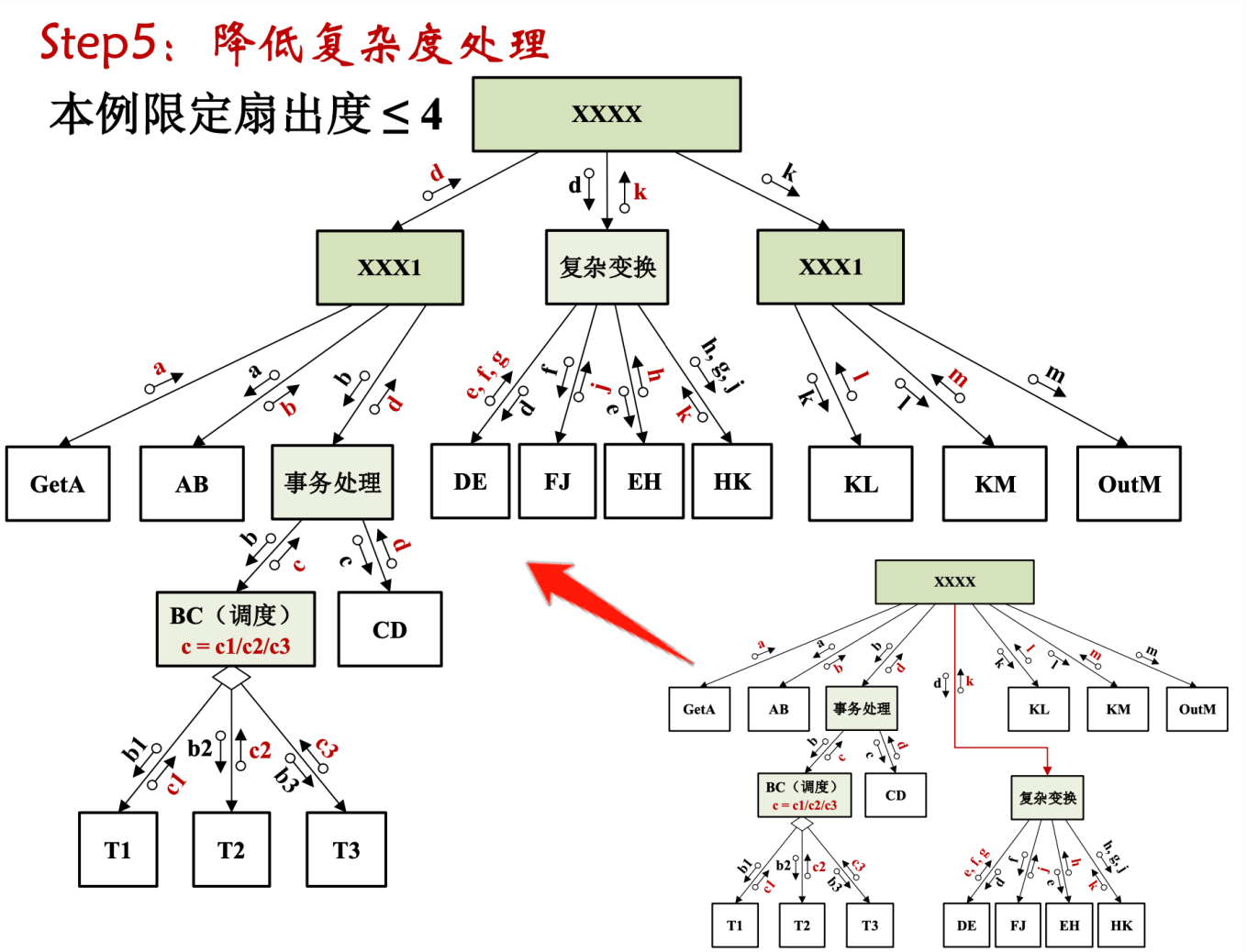



# 4.事务型与变换型混合复杂DFD→结构图转换

## Step5：降低复杂度处理

本例限定扇出度≤4






## 结构化分析与设计实例

在设计顶层DFD时， 可以对相关数据流进行合并操作，再在图的下方补充有“数据说明”相关文字，用于对合并的数据流进行详细的解释和说明。

# 06-面向对象设计

## 面向对象设计概要

传统的结构化方法

分析阶段与设计阶段分得特别清楚，分别使用两套完全不同的建模符号和建模方法

面向对象的设计（OOD）

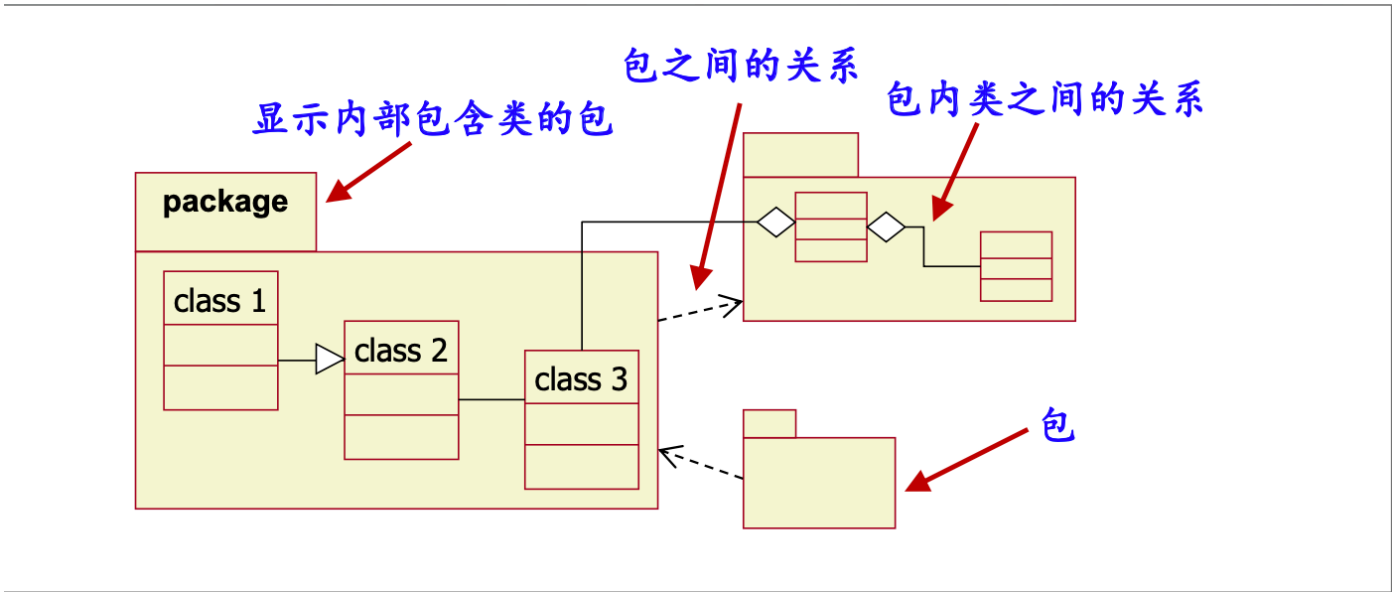
OO各阶段均采用统一的“对象”概念，各阶段之间的区分变得不明显，形成“无缝”连接

因此，OOD中仍然使用“类、属性、操 作”等概念，是在OOA基础上的进一步细化，更加接近底层的技术实现

## 面向对象设计中的基本元素

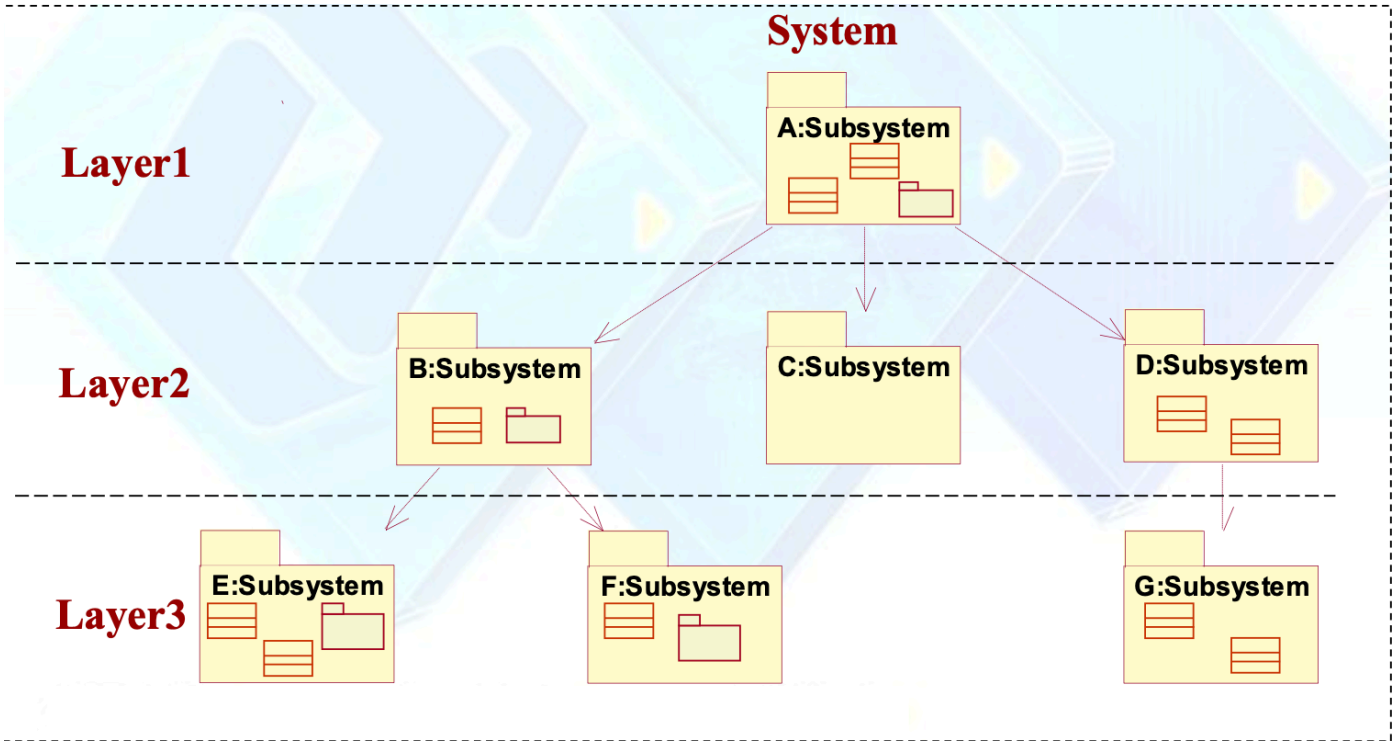
基本单元：设计类（对应OOA中的分析类）

为了系统实现与维护过程中的方便性，将多个设计类按照彼此关联的 紧密程度聚合到一起，形成大粒度的包（package）




一个或多个包聚集在一起，形成子系统(sub-system)

多个子系统，构成完整的系统(system)



## 面向对象的设计的两个阶段

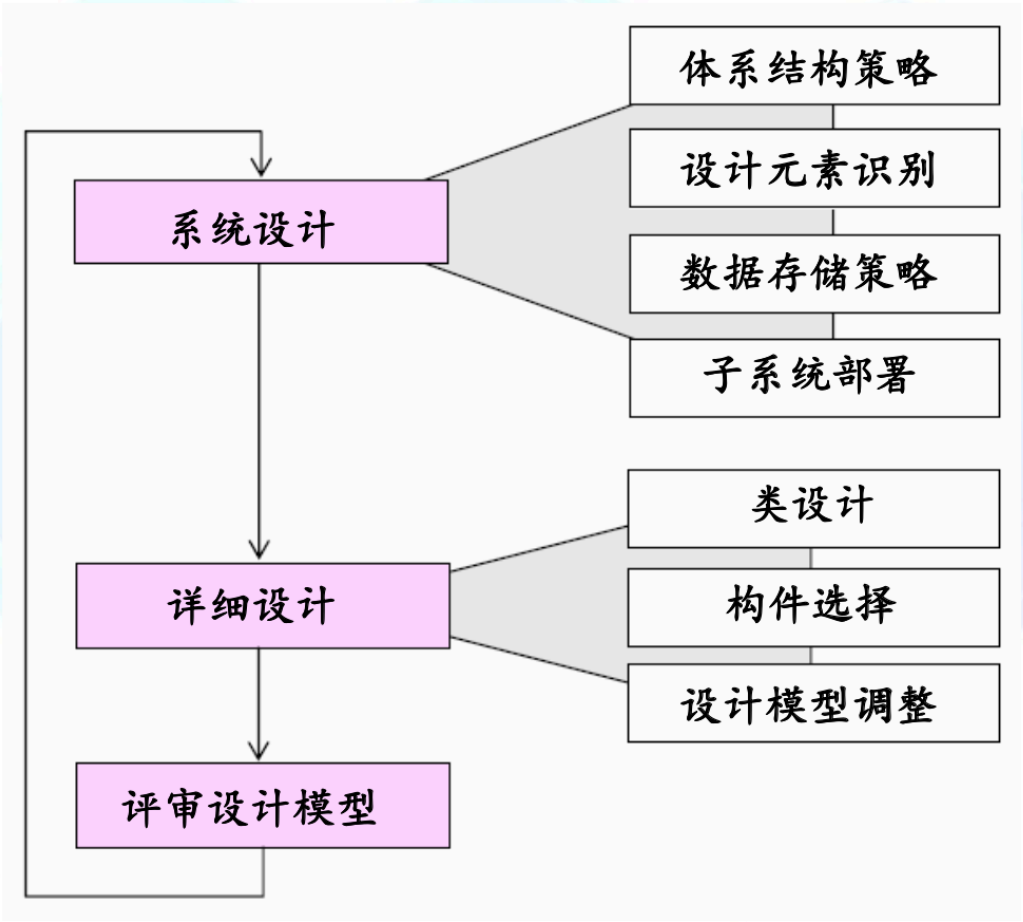
系统设计(System Design)

- 相当于概要设计（即设计系统的体系结构）
- 选择解决问题的基本途径（框架、平台、工具……）
- 决策整个系统的结构与风格（拓扑结构、软件体系结构）

对象设计(Object Design)

- 相当于详细设计（即设计对象内部的具体实现）
- 细分需求分析模型
- 识别新的对象
- 在系统所需的应用对象与可复用的商业构件之间建立关联
    - 识别系统中的应用对象
    - 调整已有的构件
    - 给出每个子系统/类的精确规格说明

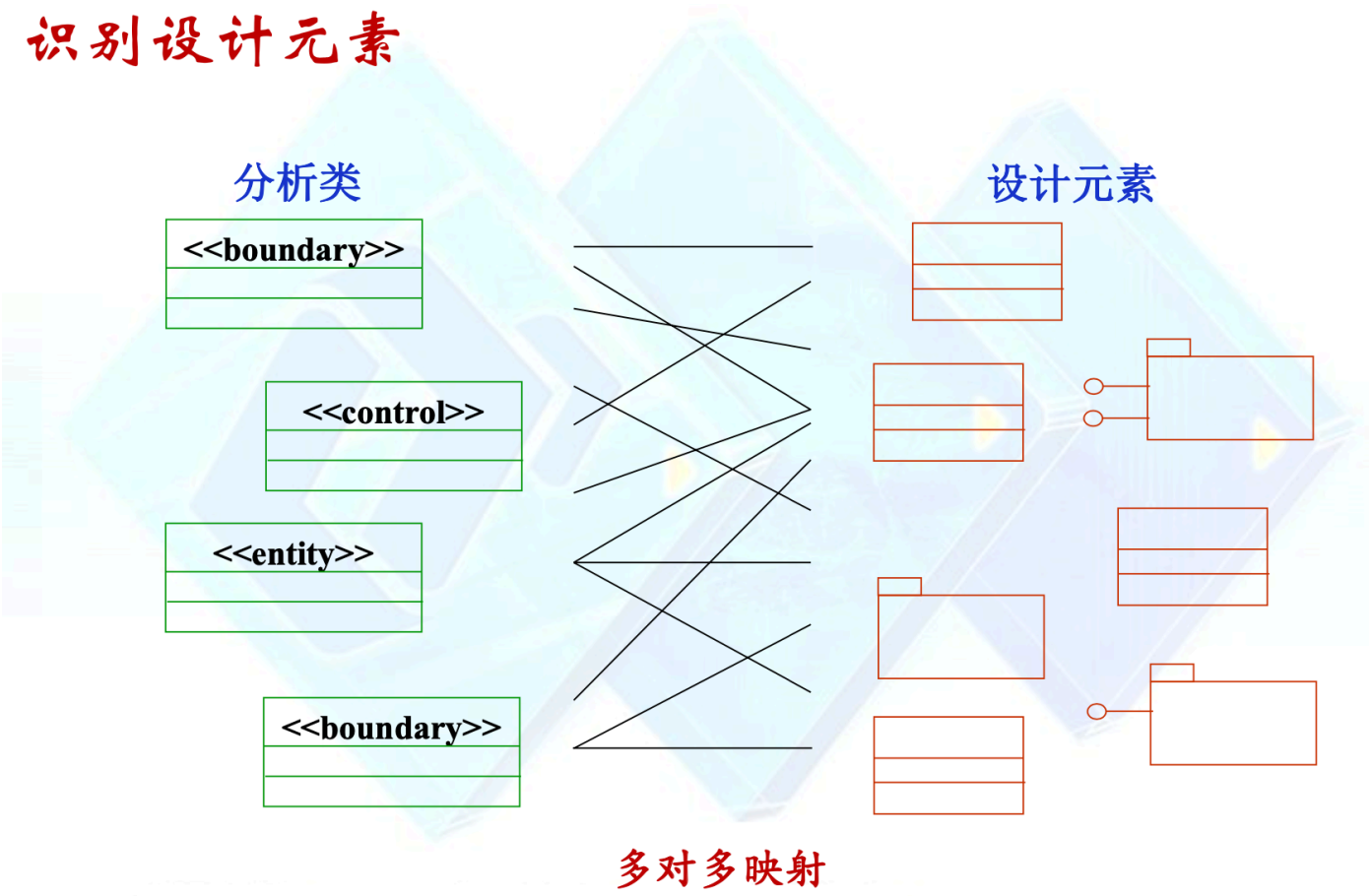
面向对象设计的过程



# 系统设计

- 设计系统的体系结构
- 识别设计元素
- 部署子系统
- 定义数据的存储策略
- 检查系统设计

# 包的设计




## 确定设计元素的基本原则

如果一个“分析类”比较简单，代表着单一的逻辑抽象，那么可以将其一对一的映射为“设计类”

　通常，主动参与者对应的边界类、控制类和一般的实体类都可以直接映射成设计类

如果“分析类”的职责比较复杂，很难由单个“设计类”承担，则应 该将其分解为多个“设计类”，并映射成“包”或“子系统”

将设计类分配到相应的“包”或“子系统”当中

　子系统的划分应该符合高内聚 低耦合的原则

注：边界类可以识别为类，也可以识别为接口

## 绘制包图

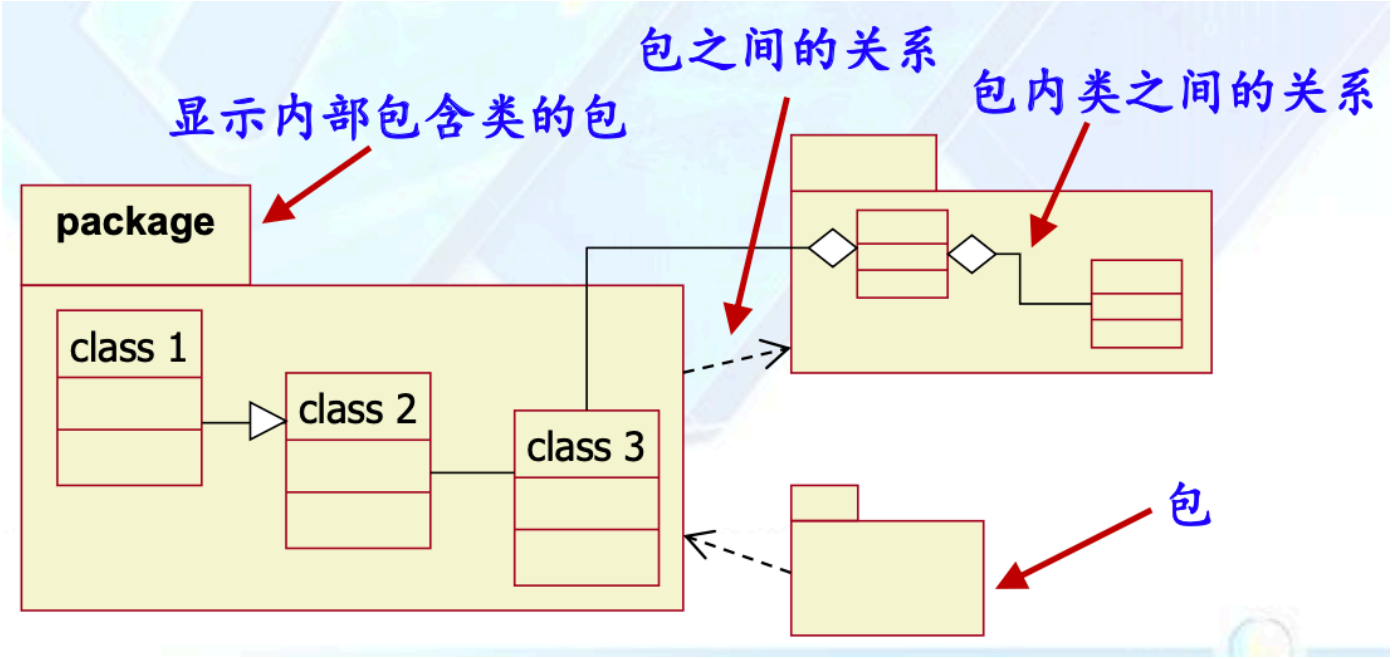
对一个复杂的软件系统，要使用大量的设计类，这时就必要把这些类分组进行组织

在语义上接近且倾向于一起变化的类组织在一起形成“包”

可控制模型的复杂度

有助于按组来控制类的可见性

结构良好的包是松耦合、高内聚的，而且对其内容的访问具有严格的控制




## 包之间的关系

类与类之间存在的“聚合、组合、关联、依赖”关系导致包与包之间 存在依赖关系，即包的依赖(dependency)

类与类之间的存在的“继承”关系导致包与包之间存在继承关系，即 包的泛化(generalization)

包与包之间存在有依赖和泛化关系

## 模型管理视图—包图

包图(Package Diagram)是在 UML 中用类似于文件夹的符号表示的模型元素的组合

UML包图提供了组织元素的方式，包能够组织任何事物

- 类、其它包、用例等
- 系统中的每个元素都**只能为一个包**所有
- 一个包可嵌套在另一个包中
- 类、接口、组件、节点、协作、用例、其他包

注:UML中的包为广义概念，不等同于Java包的狭义概念

使用包图可以将相关元素归入一个系统

一个包中可包含附属包、图表或单个元素

绘制包图的方法

# 绘制包图的方法

- 分析设计类，把概念上或语义上相近的模型元素纳入一个包
- 可以从类的功能相关性来确定纳入包中的类：
  - 如果一个类的行为和/或结构的变更要求另一个相应的变更，则这两个类是功能相关的
  - 如果删除一个类后，另一个类便变成是多余的，则这两个类是功能相关的，这说明该剩余的类只为那个被删除的类所使用，它们之间有依赖关系
  - 如果两个类之间大量的频繁交互或通信，则这两个类是功能相关的
  - 如果两个类之间有一般/特殊关系，则这两个类是功能相关的
  - 如果一个类激发创建另一个类的对象，则这两个类是功能相关的
- 确定包与包之间的依赖关系(<<import>>、<<access>>等)
- 确定包与包之间的泛化关系
- 绘制包图

检查系统设计

- 检查"完整性"
- 检查"边界性"

- 检查“完整性”
  - 是否处理边界条件？
  - 是否有用例走查来确定系统设计遗漏的功能？
  - 是否涉及到系统设计的所有方面(如硬件部署、数据存储、访问控制、遗留系统、边界条件)？
  - 是否定义了所有的子系统？
- 检查“可行性”
  - 系统中是否使用了新的技术或组件？是否对这些技术或组件进行了可行性研究？
  - 在子系统分解环境中检查性能和可靠性需求了吗？
  - 考虑并发问题了吗？

# 对象设计

## 对象设计概述

对象设计

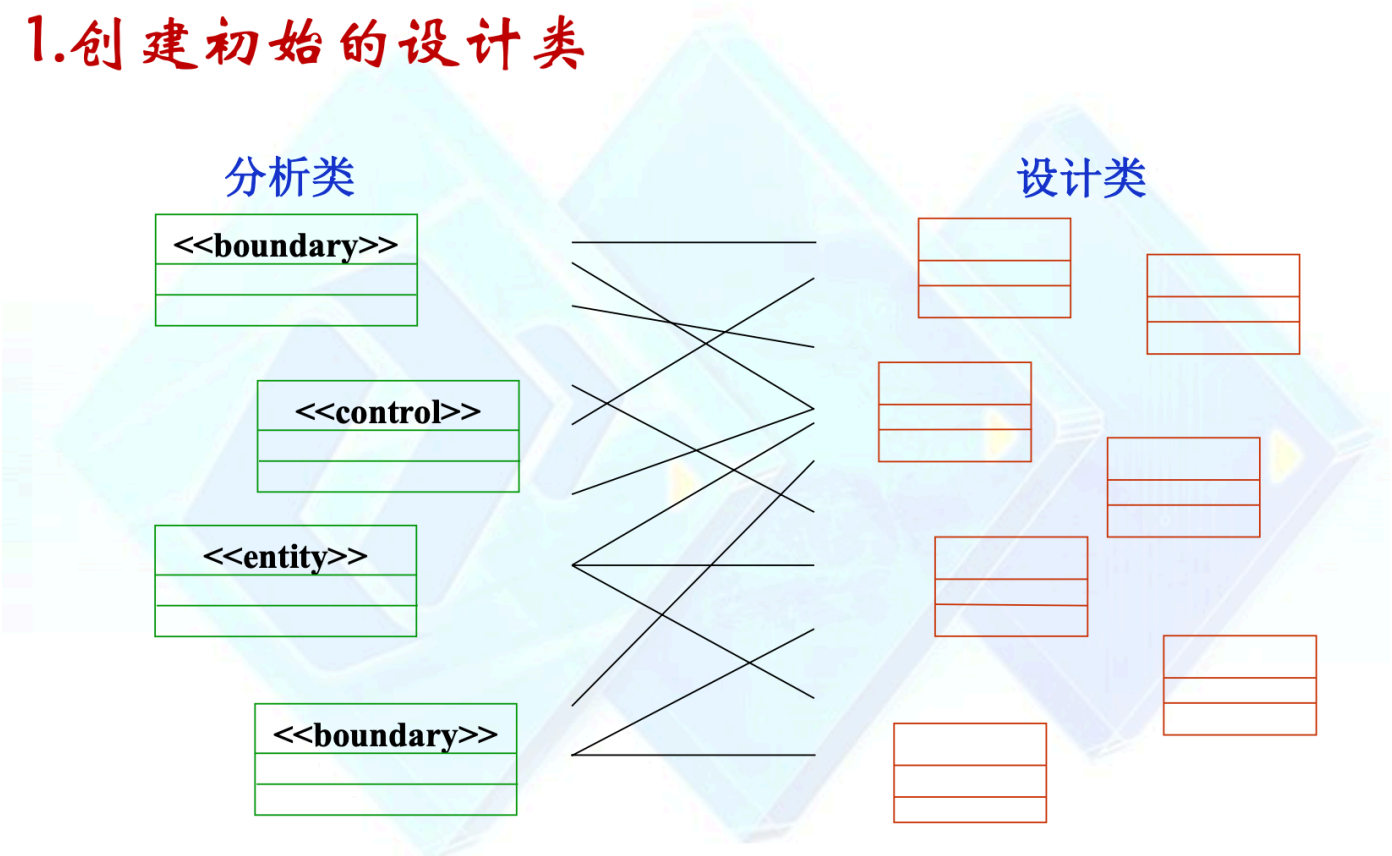
- 细化需求分析和系统设计阶段产生的模型
- 确定新的设计对象
- 消除问题域和现实域之间的差距

对象设计的主要任务

- 精化类的属性和操作
    - 明确定义操作的参数和基本的实现逻辑
    - 明确定义属性的类型和可见性
- 明确类之间的关系
- 整理和优化设计模型

## 对象设计的基本步骤

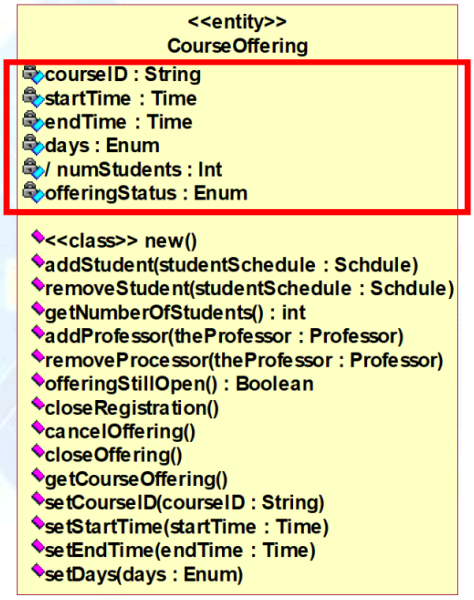
- 创建初始的设计类




- 细化属性

- 细化属性
  - 具体说明属性的名称、类型、缺省值、可见性

    visibility attributeName : Type = Default
  - Public: + 对能看到该类的任何元素都可见
  - Private: - 只对本类的任何元素可见
  - Protected: # 对该类及子孙类的任何元素可见
  - Package: ~ 对同包中的任何元素可见




- 属性的来源：
  - 类所代表的现实实体的基本信息
  - 描述状态的信息
  - 描述该类与其他类之间关联的信息
  - 派生属性(derived attribute)：该类属性的值需要通过计算其他属性的值才能得到，属性前面加“/”表示

    例如：类CourseOffering中的“学生数目” / numStudents : int

注意一下派生属性值在表示时的特殊性！

派生属性(derived attribute):该类属性的值需要通过计算其他属性的值才能得到，属性前面加“/”表示

- 基本原则
  - 属性命名符合规范(名词组合，首字母小写)
  - 尽可能将所有属性的可见性设置为private
  - 仅通过set方法更新属性
  - 仅通过get方法访问属性
  - 在属性的set方法中，实现简单的有效性验证，而在独立的验证方法中实现复杂的逻辑验证

**状态属性**

状态由一个或多个属性来表示（实体类中经常含有状态属性）

每个状态的类型大多使用boolean

大部分状态属性，可以由该类的其他属性的值进行逻辑判断得到

从一个状态值到另一个状态值的变迁，一定是有该实体类的某个操作所导致的

检查状态变迁的条件，可以判断实体类所设计的操作是否完整

## 关联属性

两个类之间有association关系，意味着需要永久管理对方的信息，需要 在程序中能够从类1的object“导航”(navigate)到类2的object

- **订单类中有一个关联属性buyer，其数据类型是“买家”类**
- **买家类中有一个关联属性OrderList，其数据类型是“订单”类构成的序列**

**关联属性不只是一个ID，而是一个或多个完整的对方类的对象**

务必要将关联属性和数据库中的“外键”区分开

不要用关系数据模式的设计思想来构造类的属性

关联属性的目标：在**程序运行空间内**实现object之间的导航，而**无需经过数据层的存取**

## 细化操作

找出满足基本逻辑要求的操作

从actor出发，分别思考需要类的哪些操作

补充必要的辅助操作

细化操作时，要充分考虑类的属性和状态是否被充分利用

对属性进行CRUD操作

对状态进行各种变更

给出完整的操作描述

确定操作的名称、参数、返回值、可见性等

遵从程序设计语言的命名规则(动词+名词，首字母小写)

详细说明操作的内部实现逻辑

复杂的操作过程采用伪代码或者绘制程序流程图/活动图方式

在给出内部实现逻辑之后，可能需要:

将各个操作中公共部分提取出来，形成独立的新操作

## 定义状态

目的

设计一个对象的状态是如何影响其行为的

绘制对象状态图

需要考虑的因素

哪些对象有状态

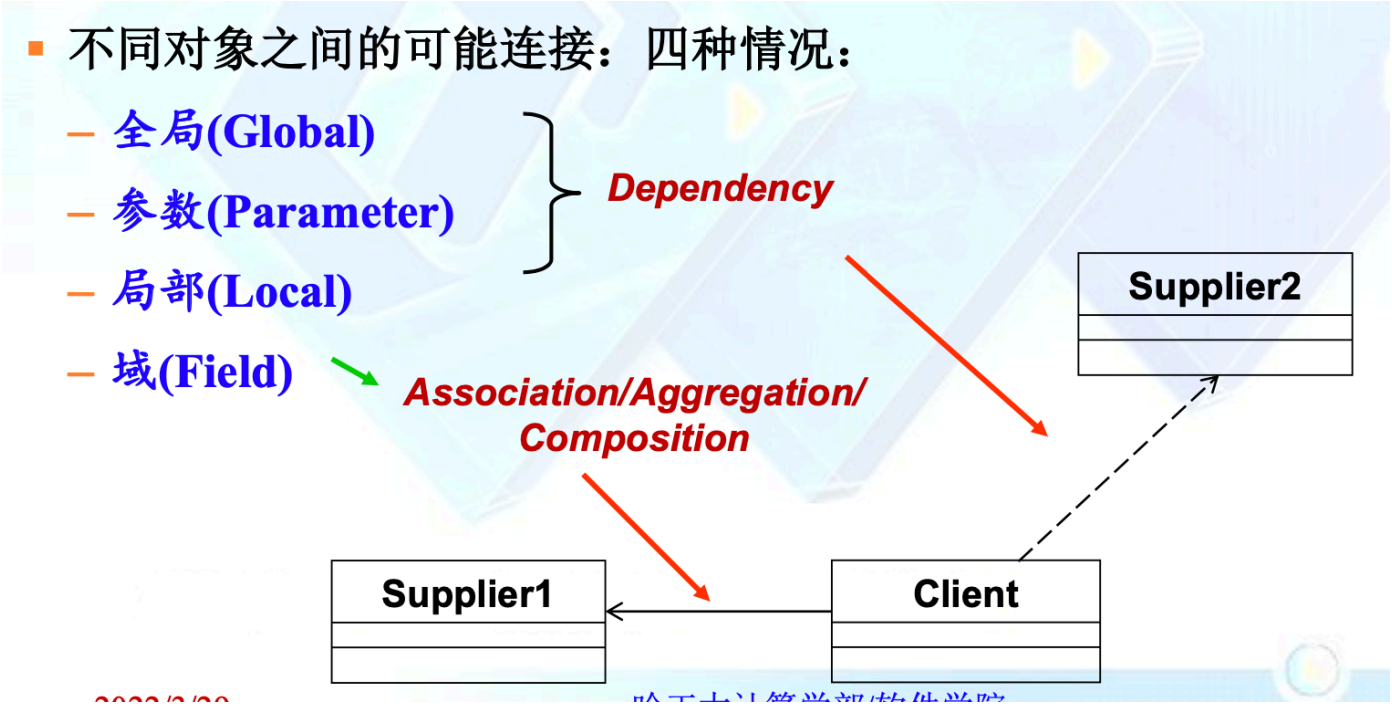
如何发现对象的所有状态

如何绘制对象状态图

## 细化类之间的关系

细化关系：关联关系、依赖关系、继承关系、组合和聚合关系

在对象设计阶段，需要进一步确定详细的关联关系、依赖关系和组合/ 聚合关系等




## 定义关联关系（Association/Composition/Aggregation）

根据“多重性”进行设计(multiplicity-oriented design)

情况1:Multiplicity = 1或Multiplicity = 0..1

可以直接用一个单一属性/指针加以实现，无需再做设计

若是双向关联，在两个类的连接线上的标明关联属性

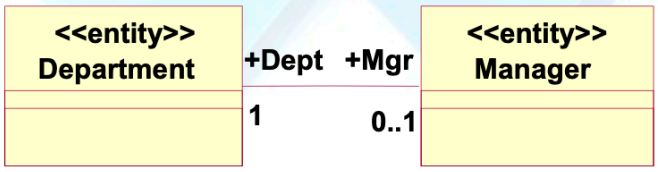
若是单向关联，只在关联关系发出的类中增加关联属性

设计阶段需要将这种 “关联属性”增加到 属性列表中，并更新操作列表

分析阶段则不需要在 属性列表中加入“关联属性”

- 情况1：Multiplicity = 1或Multiplicity = 0..1
  - 可以直接用一个单一属性/指针加以实现，无需再作设计
  - 若是双向关联：
    - Department类中有一个属性：+Mgr: Manager
    - Manager类中有一个属性：+Dept: Department
  - 若是单向关联：
    - 只在关联关系发出的类中增加关联属性

设计阶段需要将这种“关联属性”增加到属性列表中，并更新操作列表
分析阶段则不需要在属性列表中加入“关联属性”

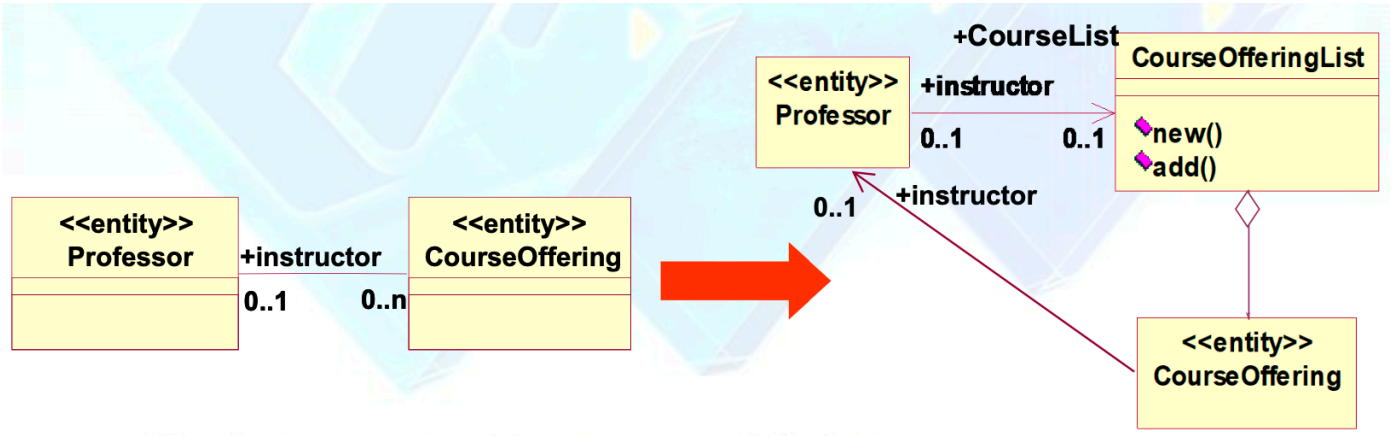



情况2: Multiplicity > 1

无法用单一属性/指针来实现，需要引入新的设计类或能够存储多个 对象的复杂数据结构(例如链表、数组等)

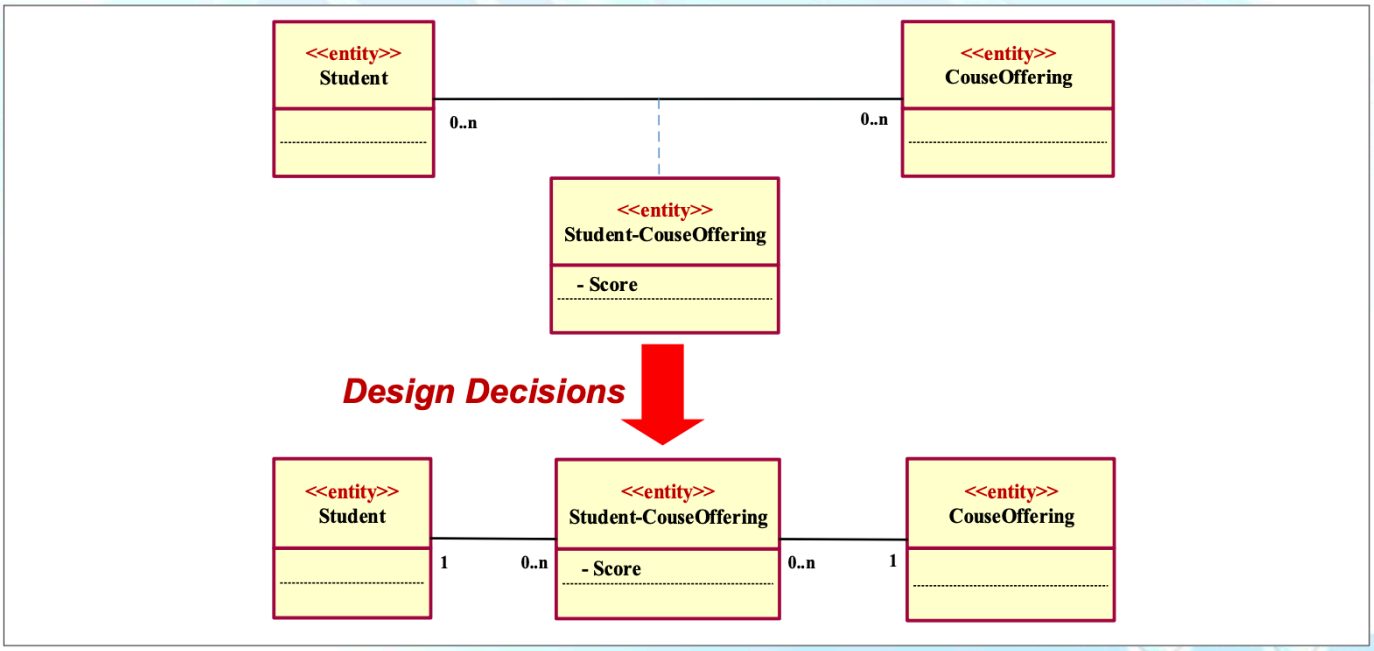
将1:n转化为若干个1:1

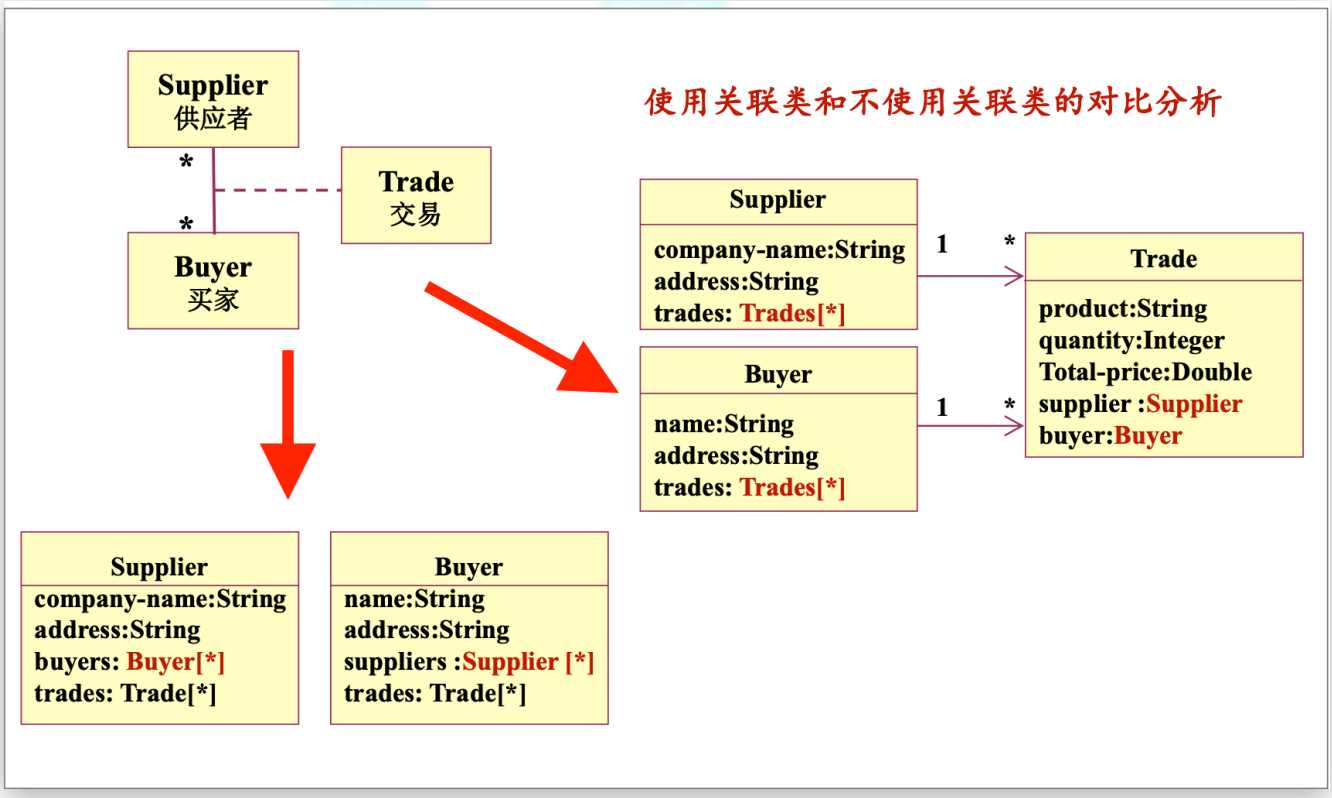
注：无法进行一对多的表示，要将一对多转化为一对一的形式



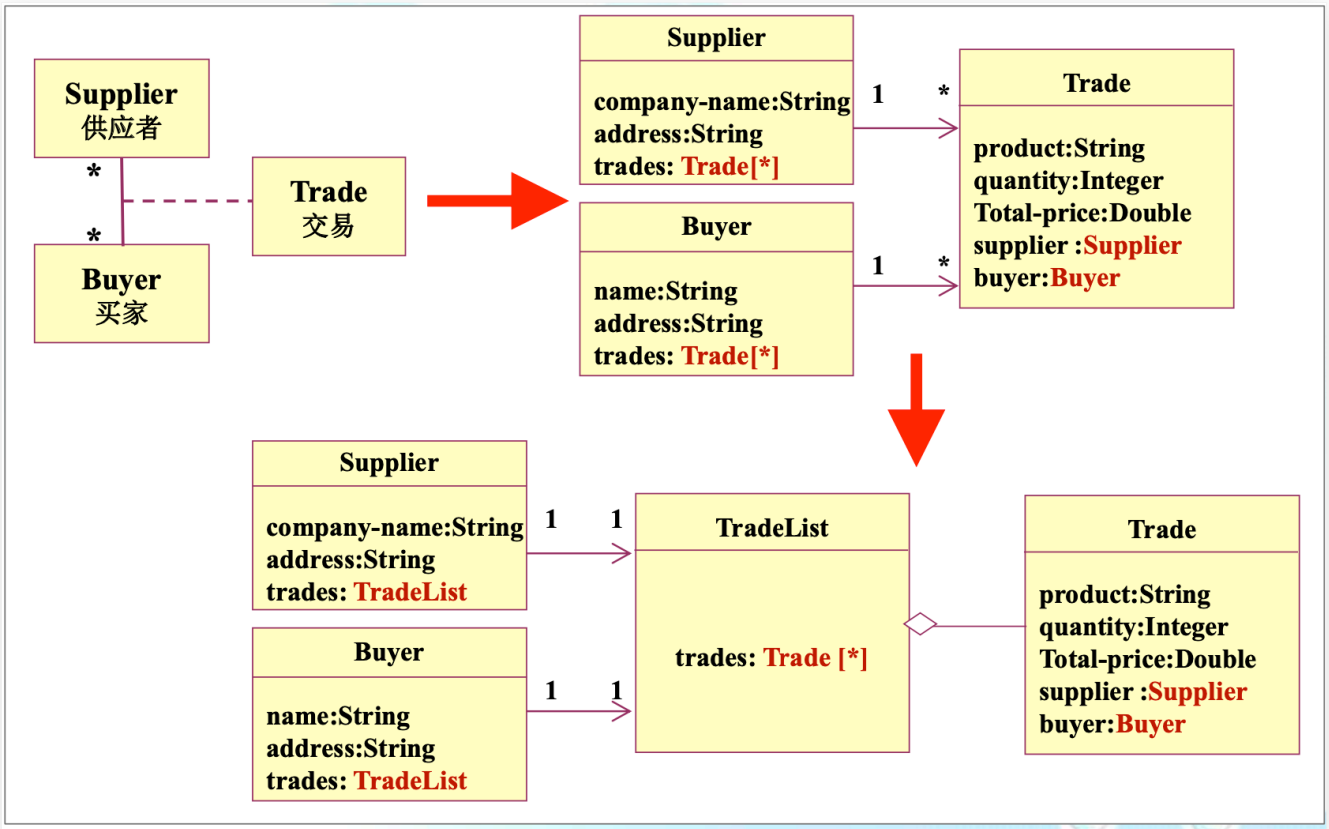

情况3:有些情况下，关联关系本身也可能具有属性，可以使用关联类将这种关系建模

- 举例：学生Student与开设课程CourseOffering之间的关联关系




引入关联类，将一对多的关系转变为多对一的关系




可以再引入一个关联类列表，将一对多的类与关联类之间的关系转变为一对一的关系

（引入 关联类列表）

### 引入“辅助类”简化类的内部结构

辅助实体类，是对从用例中识别出的核心实体的补充描述，目的是使 每个实体类的属性均为简单数据类型

简单数据类型：编程语言提供的基本数据类型(int, double, char, string, boolean, list, vector等，以及其他实体类

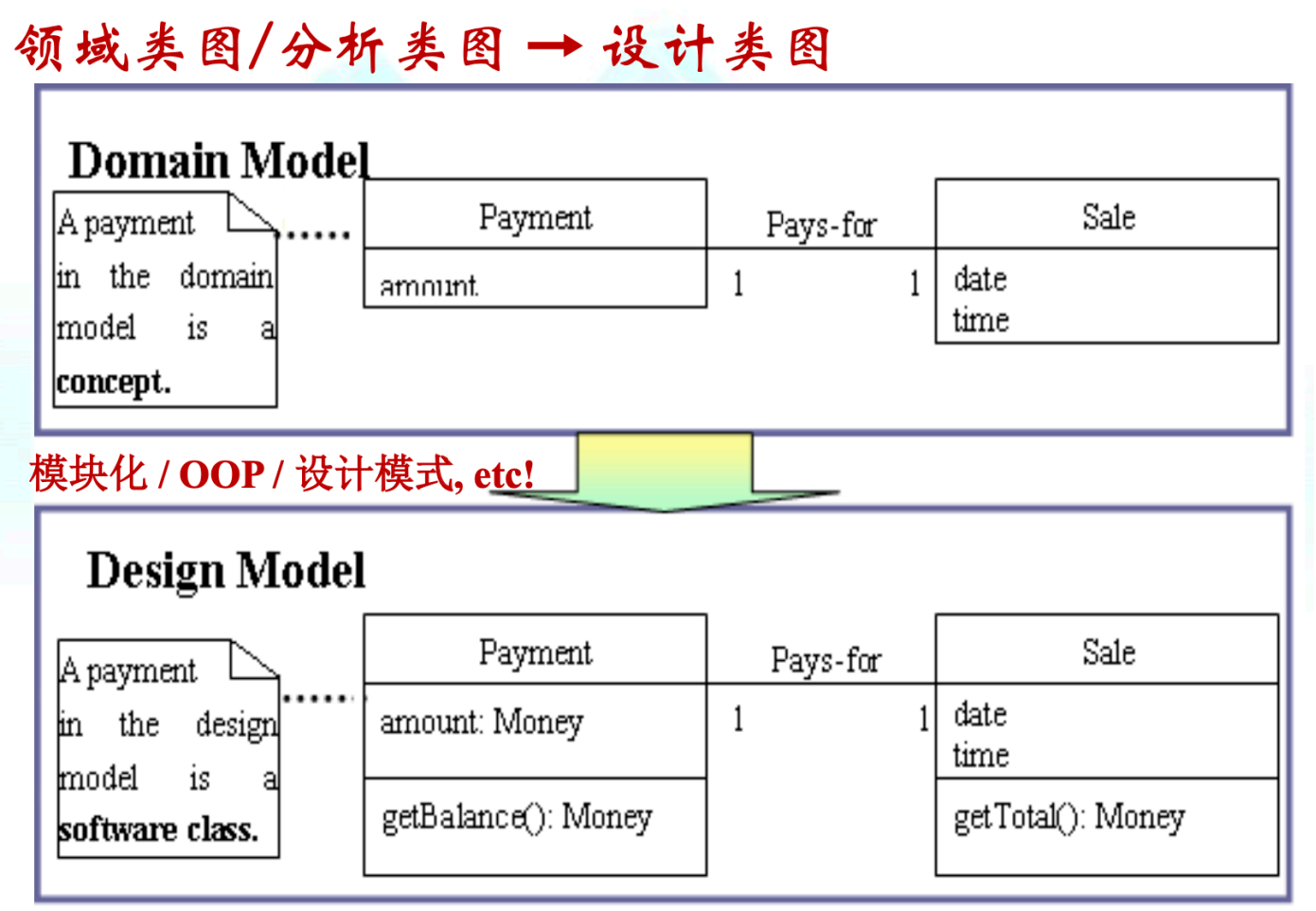
目的：使用起来更容易

- 例如对“订单”类来说，需要维护收货地址相关属性，而收货地址又由多个小粒度属性构成(收货人、联系电话、地址、邮编、送货时间)
  - 办法1：这五个小粒度属性直接作为订单类的五个属性
    实际上，这五个属性通常总是在一起使用，该办法会导致后续使用的麻烦
  - 办法2：构造一个辅助类“收货地址”，订单类只保留一个属性，其类型为该辅助类
  - 在淘宝系统中，恰好还有用例是“增加、删除、修改收货地址”，故而设置这样一个辅助类是合适的
  - 这两个实体类之间形成聚合关系(收货地址可以独立于订单而存在)

创建辅助类，充当复合数据类型的作用，作为一种简单的数据类型，起独立作用。

- 仍以订单类为例，订单的物流记录是一个非常复杂的结构体，由多行构成，每行又包含“时间(datetime)、流转记录(text)、操作人(text)”等属性，故而可以将“订单物流记录”作为一个实体类，包含着三个简单属性，而订单类中维护一个“订单流转记录(list)”属性，该属性是一个集合体，其成员元素的类型是“订单流转记录”这个实体类
- 这两个实体类之间形成组合关系(没有订单，就没有物流记录)
- 当查询订单的流转记录时，使用getXXX操作获得这个list属性，然后遍历每个要素，从中分别取出这三个基本属性即可

领域类图/分析类图 -> 设计类图




## 面向对象设计总结

## OO案例-评教系统

对照ppt 认真理解

# 07-数据库设计

## 数据库系统及关系数据库简介

数据库系统=数据库（DB）+数据库管理系统（DBMS）

数据库（DB）

- 被集中控制和管理的存储数据的完整集合
- 用ERD模型来描述
- 以表（Table）对应ERD中的数据实体（Entity），是数据管理的基本单元

数据库管理系统（DBMS）

- 对数据库的访问进行管理和控制的软件
- 管理数据的逻辑格式定义、存储和访问
- 允许多个用户或应用程序并发访问数据库
- 支持通过SQL语言访问数据库

####关系数据库

关系型数据库管理系统将数据存储成表和关系的结构

表（Table）：关系型数据库管理数据的基本单元，类似于Excel中的sheet，与ERD模型中的实体（Entity）对应

- 元组：表中的行，习惯称之为”记录“
- 域：表中的列，习惯称之为”属性“

每个表中必须包含能唯一识别”记录“的属性或属性组，称为关键属性（Key Fields），相当于记录的索引关键词

数据库的表(Table)之间的关系通过Key Fields来建立

- 主键(Primary Key)：本表中的关键属性
- 外键(Foreign Key)：存储在本表中的其他表的关键属性，用来与其他表建立关联关系(相当于指针)

### 数据库设计的根本任务

- 获取未来系统需要存储的数据实体（一定粒度的数据单元）
- 分析数据实体的内涵，以及数据实体之间的关系
- 设计物理ERD，即用于数据库系统实现的数据表结构的蓝图

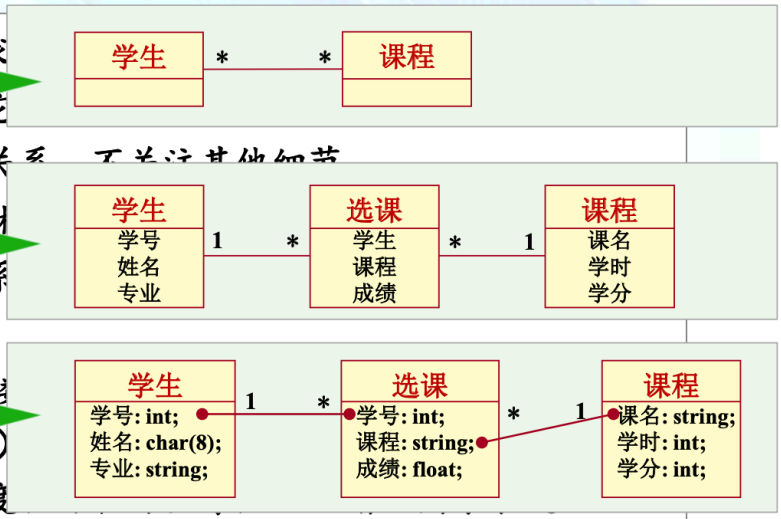
### 数据库设计的基本过程

概念ERD设计 - 逻辑ERD设计 - 物理ERD设计

- **概念ERD设计** – 主要用于需求分析过程，从宏观上识别业务过程和系统层面所存在的数据实体，分析它们之间的关系。概念 ERD数据模型仅需给出存在的数据实体以及基本关系，不关注其他细节。
- **逻辑ERD设计** – 逻辑 ERD 是概念 ERD 的详细版本，明确定义每个实体中的属性，识别实体之间的关系基数，并引入关联实体，但不关注物理实现细节。
- **物理ERD设计** – 物理 ERD 是数据库的实际设计蓝图，将ERD的数据实体转化为数据的数据表（Table），针对具体的数据库系统，定义Table属性的细节（包括属性类型、长度、可否为空等）以及增加用于表达Table之间关系的新属性。

## 数据库设计的基本过程

- **概念ERD设计** – 主要用于需求
  层面所存在的数据实体，分析它
  给出存在的数据实体以及基本关系，不关注其他细节
- **逻辑ERD设计** – 逻辑 ERD 是
  中的属性，识别实体之间的关系
  现细节。
- **物理ERD设计** – 物理 ERD 是
  体转化为数据的数据表（Table）
  性的细节（包括属性类型、长度
  之间关系的新属性。




概念ERD只有实体和关系，没有属性值，关系基数可有可无

逻辑ERD与物理ERD相比，物理ERD多了主外键的约束，二者都比概念ERD多了属性值

| ERD形式 | 概念ERD | 逻辑ERD | 物理ERD |
|---|---|---|---|
| 实体（名称） | √ | √ | √ |
| 关系 | √ | √ | √ |
| 关系基数 | 随意 | √ | √ |
| 关联实体 | | √ | √ |
| 属性（名称） | | √ | √ |
| 属性的类型、长度、说明 | | 随意 | √ |
| 主键、外键 | | | √ |

# 数据库逻辑模型设计

1. 识别所有“自然”数据实体（Entity）
    - 来自DFD中的全部数据存储
    - 来自分析类图中的部分类
2. 为数据实体命名
3. 给出实体的属性
4. 识别实体之间的关联关系及关联重数
    - 关联关系需要命名
    - 关联重数需判别（1:1、1：M、M：N）
5. 建立关联实体来消除M:N重关系（要注意在数据库设计不能存在有多对多的关系）
6. 实体规范化（一般需要满足3NF）
7. 评价ERD质量并做必要的改进

## 复杂ERD分组技巧

根据系统功能对ERD进行分组处理

如果组之间存在相互关联的实体出现，则可以将其 画多次，分别在不同的局部ERD中出现

用局部ERD拆分整体的ERD图，此时允许同一个实体在不同的ERD图绘制多次

# ERD模型及质量评价

- ERD图的设计结果没有标准答案，即不唯一
- 好的ERD的标准
    - 结构清晰
    - 关联简洁
    - 实体个数适中
    - 属性分配合理
    - 没有低级冗余（重复性冗余）
- 至少符合3NF

## 数据库的规范化

通过最小化数据冗余来确保数据库模型的质量

范式的分类

- 范式（Normal Form）的分类
  - 1NF – 没有重复的属性或属性组
  - 2NF – 是1NF 且每个非主属性均函数依赖于主属性(主键)
  - 3NF – 是2NF且非主属性间均不存在函数依赖

- 范式的通俗、易懂的解释：
  - 1NF：同样的东西不能重复拥有！
  - 2NF：领导在，他就在，找到领导就能找到他！
  - 3NF：领导只有一个，不能搞宗派，不能有多级领导！

3NF数据库设计可以做到，但是未必是最好的数据库

为了提高运行效率，必须降低范式标准，适当保留冗余数据

在逻辑模型设计时，遵守3NF，降低范式标准的工作放到物理模型设计时考虑，即增加冗余字段

（降低范式标准，增加冗余字段，提高运行效率）

## 物理数据库设计及建立

根据ERD建立物理数据库

1. 为每个ERD实体创建一个二维表（即数据库Table）
2. 根据ERD中的实体属性，在对应的Table中定义对应的字段（Field），并给出适当的数据类型和取值范围
3. 定义每个Table的主键（PK）
4. 针对1:M的关联关系的子表添加外键（FK），即对应主表的主键

   外键是在一对多的关联关系当中进行使用
5. 定义完整性约束

定义完整性约束的3个层次

- 域的完整性：设定字段的取值范围
- 参照完整性：用PK、FK、表级触发器来实现
- 用户定义完整性：定义一些业务规则，用存储过程和触发器来实现

参照完整性的自动执行

- 当建立一个包含外键的记录时，DBMS 确保该外键同时作为主键出现在另一个相关表的记录中
  - 如果关联其他记录，对方必须存在
- 当删除一个记录时，DBMS 确保在其它相关的表中不会出现和该记录主键值相同的外键
  - 如果正在被其他记录关联，则不能删除
- 当主键值改变时，DBMS保证相关表中没有外键与该主键具有相同的值
  - 改变记录主键值时：①确保不冲突；②全局自动更改

## 物理数据库提高效率的技巧

- 降低范式，提高冗余；少用触发器，多用存储过程
- 体外运算

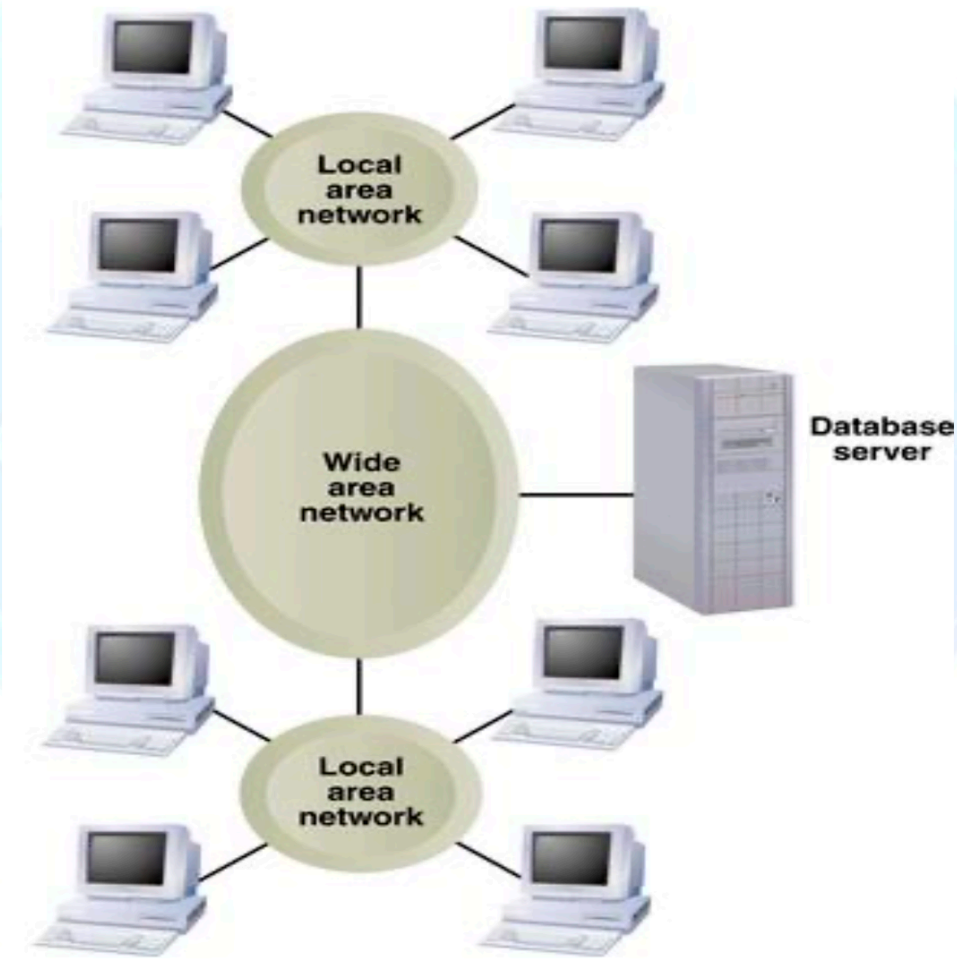
  以文件系统方式进行计算处理，最后入库追加到表当中去
- 水平/垂直分割表

- 当表中记录太多时则水平分割：以该表主键PK的某个值为界线，将该表的记录水平分割为两个表
- 当表中字段太多时则垂直分割：将原来的字段分为二组，分别建立二个1：1的表

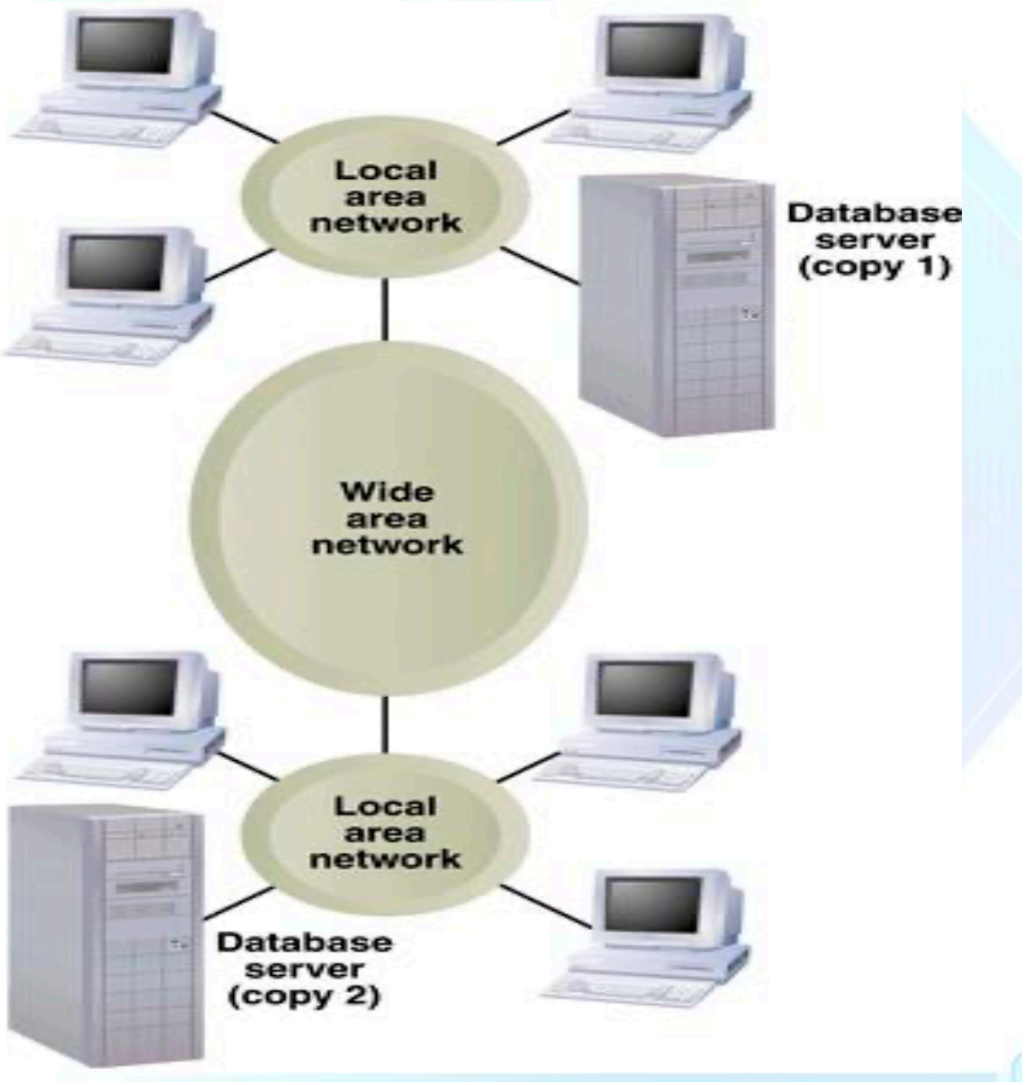
- 对数据库管理系统DBMS进行系统优化，即优化各种系统参数，如缓冲区的个数
- 在使用面向数据的SQL语言进行程序设计时，尽量采用优化算

## 描述分布式数据库的不同的体系结构模型

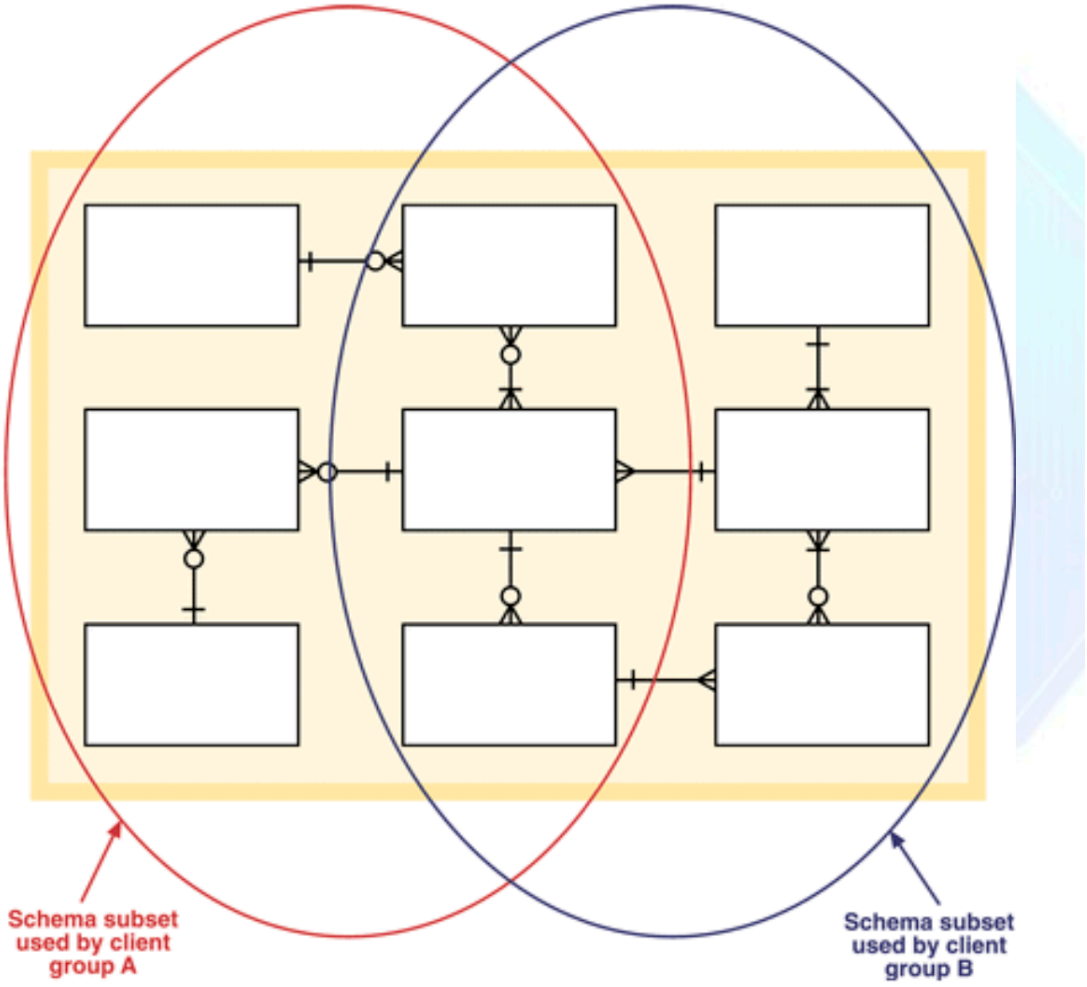
# 单数据库服务器体系结构



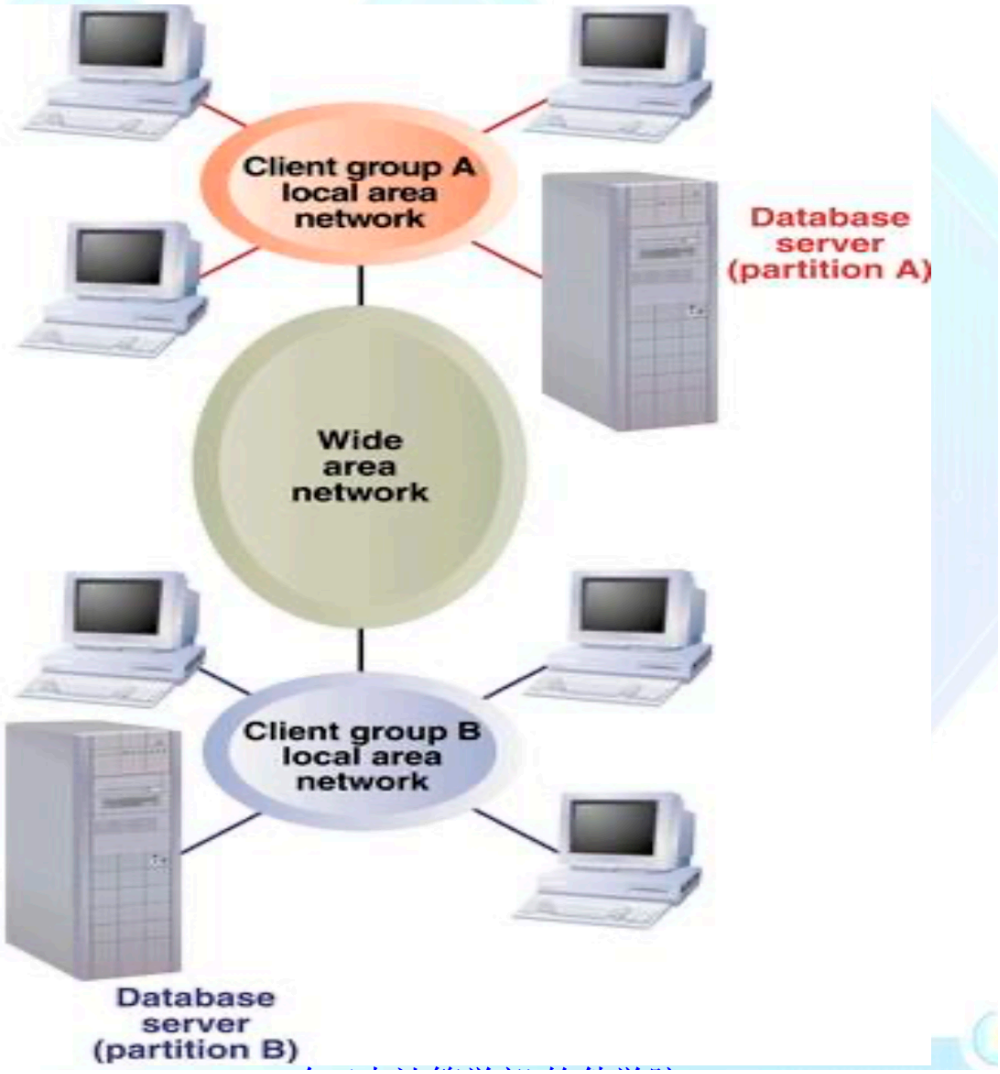
# 带备份的数据库服务器体系结构



# 将数据库模式划分为多个客户访问子集



# 分区数据库服务器体系结构




## OO实体类图映射到ERD

### 类和关系数据库表的关系

oop以class为基本单位，所有的object都是运行在内存空间当中的

如果某些object的信息需要持久化存储，那么就需用到database，将object的属性信息写入关系数据库

在系统执行某些功能的时候，需要首先从database中将信息取出，使 用各实体类的new操作构造相应的object，在程序运行空间中使用(充 分利用继承/组合/聚合/关联/依赖关系在各object之间相互导航)

持久性数据：对象只是存储在内存当中，而某些对象则需要永久性存储

### 数据存取策略

如果使用OO数据库，那么数据库系统应提供一个接口供应用系统访问数据

如果使用关系数据库，则需要一个子系统来完成应用系统中的对象和数据库中数据的映射于转换

## OO设计中的数据库设计

核心问题：对需要永久性存储的数据，如何将UML类图（实体类图）映射为数据库模型（实体关系模型ERD）

本质：把每一个类、类之间的关系分别映射到一张表或多张 关系型数据库的表中

解决：UML -> RDB

两个方面：

- 将类映射到表
- 将关联关系映射到表

## 将对象映射到关系数据库

- 简单类型

一类一表，表中的字段对应类的属性，表中的每一行数据记录类的实例

- 复杂类型（实体类图 -> ERD映射）
    - 1:1关联关系
        - 建立1个表

            增加PK属性

            复制2个类中的所有属性
        - 建立1:1关系的两个表

            每个表分别增加PK属性

            在每个表中分别复制对应类中的所有属性

            2个表的PK建立关联，从而建立参照完整性，任何一个表作为主表均可
    - 1:*关联关系
        - 建立1:*关系的2个表

            每个表分别增加PK

            每个表分别复制对应类中的所有属性

            表2中增加FK属性，并与表1的PK属性关联，从而建立起参照完整性，主表为基数为1的那个表
    - * : * 关联联系

        3个表分别添加PK

        表1、表2的其他属性分别复制类1、类2的所有属性

        关联表中增加2个FK属性，分别与表1、表2的PK关联，从而建立参照完整性，表1、表2均为主表

        关联表中经常会增加若干其他属性值
    - 循环* : *关联关系
        - 这3个类是紧密关联的，即任何2个类的关联行为都不能缺少第3个类的参与

            建立表1、表2、表3分别直接对应类1、类2、类3

            建立1个关联表，需要创建3个ForeignKey，分别跟表 1~3的PrimaryKey关联，建立参照完整性

            关联表中一般会增加若干其他属性

        - 这3个类仅仅是两两相关联，即该类图包含3个“典型类型三” 实例:学生 课程 教师

          的情况，即表1-表2、表2-表3、表3-表1，需要创建3个关联表

          每对关联的处理方法同“典型类型三”。
    - 组合/聚合关联关系

      聚合/组合可以理解为特别紧密的关联关系

      在映射到数据库表设计时，参照1:1或1:*关系处理即可
    - 继承关系
        - 分别建立父类和子类对应的共3张数据表，形成1:*的映射类型;

          其中父类表保留父类的全部属性，或者将部分属性(父类与子类是1:1关系的属性)下移到子类对应的表中
        - 分别建立子类共2张数据表，将父类的属性全部下移到各个子类所对应的数据表中

          会产生一定量冗余数据，浪费存储空间，表的访问效率会降低

          当父类中独立的属性较少时，该方案可行
        - 只建立1张数据表，将子类的属性上移到父类所对应的数据表中， 该表包括父类和各子类的全部属性，需增加区别子类的标志性属性

          会产生大量的空白数据和冗余数据，比方法2更浪费存储空间, 表的访问效率会大大降低

          只有子类中属性很少的情况下，该方法才可用

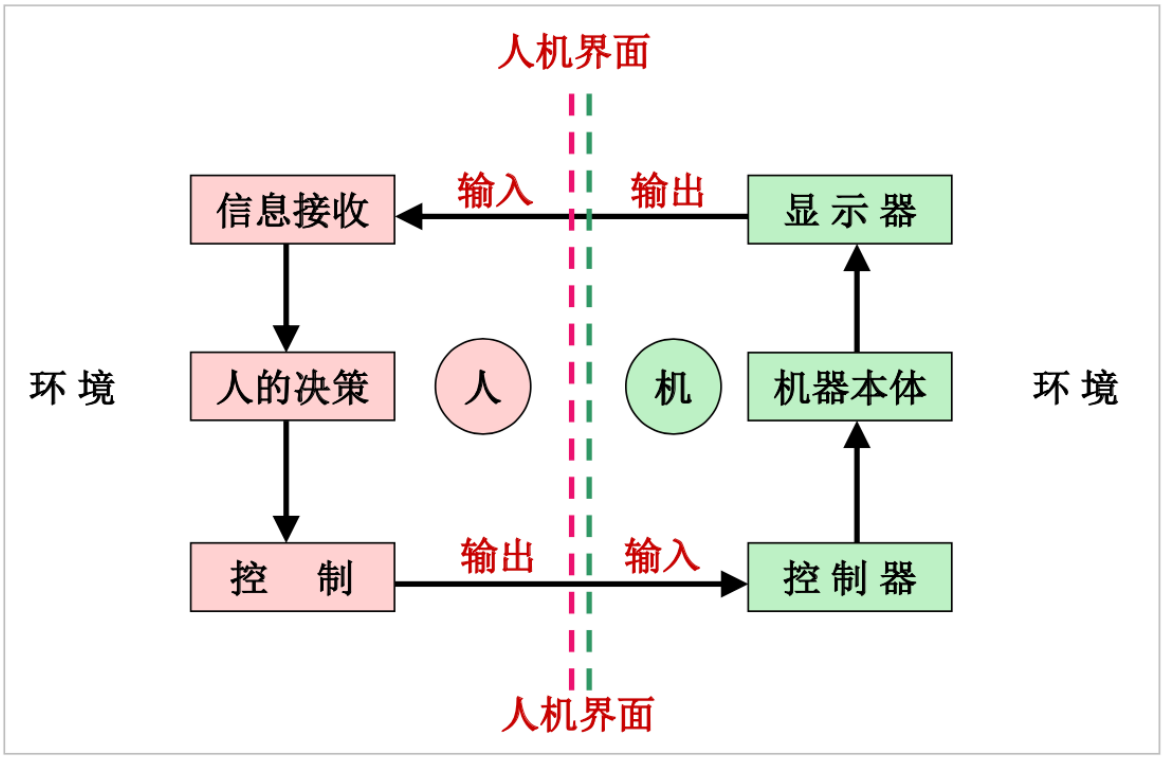
## 关系型数据库句规范化

# 08-用户界面设计

## 用户界面设计的基本概念

- 人机系统模型

# 1. 人机系统模型




人机系统模型

就用户界面而言，其指的一般是软件人机界面

CUI - GUI - MUI （字符用户界面 - 图形用户界面 - 多通道用户界面）

（看看ppt就行，太杂了）